Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

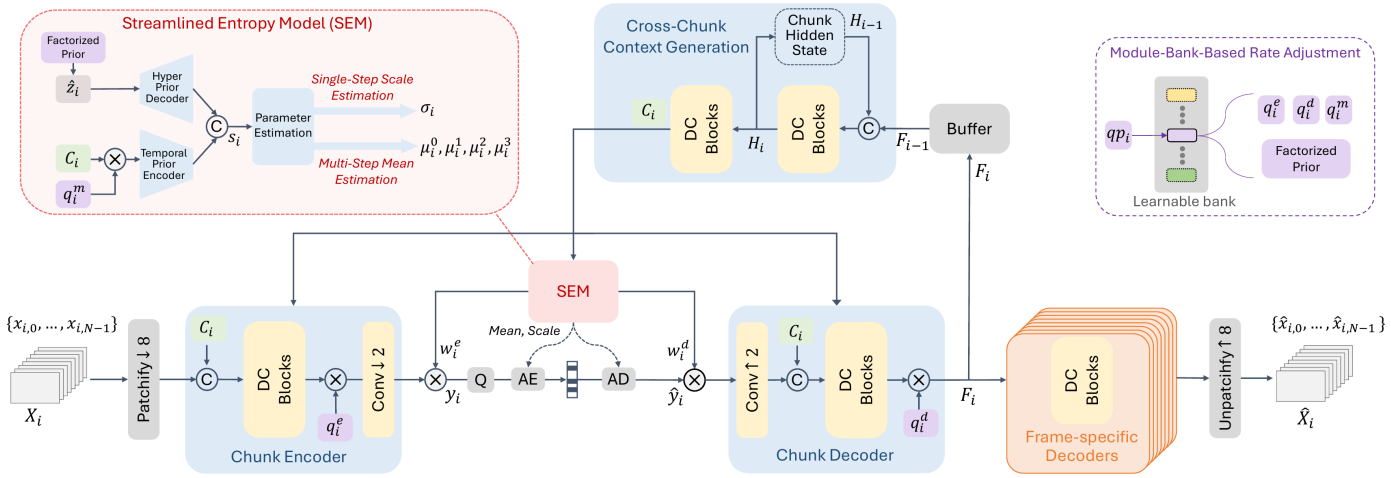

Figure 1. The decoder we modify. Figure 3 of DCVC-UF [14], reproduced. Everything up to the reconstruction stays frozen in this work: the patch embedding, the chunk encoder, the entropy model and the coded payload. FLEX-UF replaces the frame-specific decoders on the right with a ladder of exits taken per tile.

Abstract

Learned image decoders spend the same computation on every region of a frame, whatever that region contains. We show that decoding compute can instead be allocated spatially, by content, with the encoder, the entropy model and the transmitted representation left untouched. We add a multi-exit ladder to the intra decoder of DCVC-UF, so that tiles of one frame leave a shared reconstruction trunk at different depths. On 53 CTC intra frames, one from each sequence, under a 0.1 dB quality constraint this removes 29.9% to 15.6% of decoder multiply-accumulates across the rate range, 22.1% on average, at a BD-Rate cost of 0.88%. At 0.2 dB the mean saving is 34.4% and at 0.3 dB it is 38.2%, within 0.9 points of the ladder's 39.1% architectural ceiling.

Adaptive decoding works only inside a finite window. Tiling sets a rate-dependent quality floor below which no allocation is feasible, and a saturation point above which every tile already sits on the cheapest path. Rescaling the budget between those two limits collapses five rate curves that differ by 15.5 points onto one within 1.7. The dominant cost of adapting is set by per-tile decoding depth rather than by the spatial extent of boundary contamination. Finally, the bits the entropy model has already spent on a tile predict how deep that tile must decode better than a trained 144 K-parameter router does, by up to 6.9 points, with no parameters and nothing added to the file. A compressed representation carries not only what to reconstruct, but how much computation the reconstruction needs.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity given as a single number, so many GMAC per frame or so many milliseconds. That number is a constant. A learned decoder runs the same graph on a page of text as on a cloudless sky, although the sky is much the easier picture.

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop as soon as the prediction is confident [3, 15, 20], and the literature around them is by now mature. Super-resolution adopted the same idea spatially. ClassSR [12] routes image patches to networks of different capacity by difficulty, and APE [1] exits patches at different depths of one network. Learned compression has taken a different route. Slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream* rather than per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder, under a constraint that makes the answer deployable: **the encoder is frozen and the coded payload is unchanged.** Whatever we do has to consume the bitstream the released encoder already produces. That rules out re-training the analysis transform, changing the entropy model, or altering the latent. It leaves one place to spend adaptivity, the synthesis transform.

Our method, FLEX-UF, attaches exits at intervals through the twelve residual blocks of the DCVC-UF intra decoder; we call that ordered set of exits the *ladder*. The first j blocks run over the whole frame. The rest run per tile, over a set of tiles that shrinks with depth, and the shrinkage is where the saving comes from. Each exit hands its feature to the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Getting this to work depended less on the ladder than on two problems that have little to do with early exit as it is usually studied.

Tiling has a large price.

Spatial adaptivity needs tiles, and once a frame has been cut into tiles, every 3×3 convolution at a tile border reads invented values. The tile-boundary artefact that follows is what we call the *seam*, and under the default padding rule it costs 0.677 dB on this decoder at high rate. That is several times the entire budget the method works to, and it is paid before any tile has saved a single operation. In Section 4 we treat padding as an *estimator* of the unseen neighbour and measure four of them. The ordering we get is not the obvious one.

A distortion budget only works inside a band.

Tiling costs something even when nothing exits early, so there is a *floor*, and budgets below it admit no allocation at all. The ladder also has a shallowest usable exit, so there is a *saturation* point above which every tile already takes that exit and more quality buys nothing. Between the two, in what we call the *band*, the budget genuinely trades quality for

compute. We measure both ends at every rate in Section 5.4, where the 0.1 dB budget uses 45% of the band at low rate and 9% at high rate.

Contributions.

(i) An early-exit ladder for a learned image decoder that picks a depth per tile, which is spatial adaptivity at the granularity of a tile. The encoder, the entropy model and the coded latent are untouched in both modes we report. *Signalled* FLEX-UF leaves the latent unchanged and adds 89 bits per 1080p frame of routing metadata, or 0.021% of a typical bitrate, and saves 29.9% to 15.6% of decoder MACs for 0.1 dB on the common test set at a BD-Rate cost of 0.88%. *Bitstream-identical* FLEX-UF adds nothing at all, decodes a byte-identical file, and reaches 26.8% to 5.1%.

(ii) The band a distortion budget works in: a floor below which no allocation is feasible, a saturation point above which none improves, both in closed form, and seven propositions verified numerically rather than asserted. Measuring the budget in units of that band accounts for most of the rate dependence, on two independently trained checkpoints.

(iii) A zero-learned-parameter control that adaptive-inference work is rarely measured against. Routing on the bits the entropy model has already spent per tile costs nothing, adds nothing to the stream, and beats our trained head at every rate, while agreeing with the oracle on fewer tiles than the head does.

2. Related work

Early exit.

BranchyNet [20] and MSDNet [3] established the pattern of intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking. We train all exits jointly, after Scardapane et al. [18], and distil between exits as in Phuong and Lampert [17]. The caveat in [19] is that too large a student-teacher gap hurts the shallowest exits, so our distillation runs between *adjacent* exits. All of this work exits on a confidence signal computed from the network's own output. A decoder has no such signal. There is no class posterior, and the quantity that would decide is the error against a source the decoder cannot see (Section 3.5).

Spatially adaptive inference.

ClassSR [12] sorts super-resolution patches into easy/medium/hard and runs a different network on each; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. We share the per-region premise with all three, and we inherit the same tiling problem, though the cost of tile borders is rarely quantified in that line of work. To our knowledge the estimator view of border padding and its measured ordering (Section 4) is new. The closest prior work is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Method	Varies	Where	Latent	Side data	Dec. only
SlimCAE [22]	width	per stream	new	–	no
Slimmable video [23]	width	per stream	new	–	no
EVC [22]	mask / pruning	per model	new	–	no
Spatial competition [27]	which codec	per region	new	yes	no
DCVC-RT [33]	architecture	fixed	new	–	no
FLEX-UF signalled	**decoder depth**	**per region**	**same**	89 b	no
FLEX-UF bitstream-identical	**decoder depth**	**per region**	**same**	**none**	**yes**

Table 1. Where this sits. What each method varies, at what granularity, and what that costs the stream. A single new-bitstream column cannot represent our own two modes, so the last three ask what actually changes: whether the coded latent is the same, what side data travels with it, and whether the decoder alone can do it.

Method	kMAC/px	Par. (M)	BD-Rate (%)
CHARM [51]	496	58.5	+0.86
STF [45]	511	99.9	-2.06
ELIC [44]	574	36.9	-3.22
WeConvene [50]	2343	107.2	-6.98
MambaVC [49]	814	47.9	-8.72
DCVC-DC intra [52]	542	45.5	-9.18
TCM [46]	1824	76.6	-10.70
FLIC [48]	1096	71.0	-13.20
MLIC++ [47]	1283	116.7	-15.15
HPCM-Base [43]	919	68.5	-15.31
HPCM-Large [43]	1261	89.7	-19.19

Table 2. What a learned image decoder costs. Eleven codecs as published, measured on one machine by Li et al. [43]: BD-Rate against VTM-22.0 on Kodak, arithmetic per pixel and parameters. The spread in cost is an order of magnitude, and every entry pays its cost on every pixel of every image.

Method	GMAC	Par. (M)	BD-Rate (%)	Adaptive
DCVC-DC [52]	–	18.3	+14.5	no
DCVC-FM [13]	2642	18.3	-21.3	no
DCVC-RT [33]	385	20.7	-21.0	no
DCVC-UF (LD) [14]	170	9.7	-9.5	no
DCVC-UF (HT-S) [14]	211	81.2	-31.6	no
DCVC-UF (HT-L) [14]	343	120.5	-42.2	no
FLEX-UF intra decoder	**219 → 170**	–	**+0.88**	**yes**

Table 3. The family this work modifies, from DCVC-UF [14]: MACs per 1080p frame, parameters, and BD-Rate against VTM-17.0 low delay. Comparable within this table and not against Table 2, which uses a different anchor and a different test set. The last column is the one this paper is about: whether the decoder spends more computation where the picture needs it. Our row is the intra decoder of the last DCVC-UF entry, at the 0.1 dB budget.

Together the two tables say where the field spends its decoder budget. Complexity has moved a long way in both directions: TCM [46] and WeConvene [50] buy rate with an order of magnitude more arithmetic per pixel than CHARM [51] or STF [45], and DCVC-UF [14] moves the other way, cutting a 1080p frame from DCVC-FM's 2642 GMAC to 170. Two things are constant down both tables. The cost is a property of the model, chosen once and paid on every image; and it is uniform over the frame, so a flat sky and a face are decoded at the same price. Those are the two the ladder in this paper changes, and it changes them without touching the model that was shipped.

Complexity control in learned compression.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model. The choice is per stream and it changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Rate–distortion–complexity has since become an explicit third axis. Gao et al. [35] tune spatial context usage to trade decode cost against rate, Ho et al. [36] survey where conditional residual coding sits on that surface, and Zhang and Gao [37] route *whole frames* to one of several jointly-trained coding paths, spending less computation on cheaper frames. Every one of these changes the model, and with it what the encoder emits. Our axis is orthogonal. The model and the bitstream are both fixed, and only *how much of the decoder runs where* varies.

The closest neighbour.

Blard et al. [27] also partition an image into regions, also choose per region by a rate–distortion cost computed at the encoder, and also transmit a mode map. The differences are in what the regions choose among and in what the map buys. Their regions choose among several *separately trained* codecs, so the decoder must hold all of them and its complexity is that of one codec regardless of the choice; the map buys rate, not compute. Ours choose a prefix length of a single trunk, so the weights are shared by construction and the map buys compute at fixed

rate. Because their alternatives are unrelated networks, no decoder-side predictor could stand in for the encoder's search. Our exits are prefixes of one another, and that nesting is what lets a decoder-side predictor stand in at all (Section 3.1).

Signalling versus prediction.

Being *told* a mode decision rather than inferring it is the norm in standardised video coding. HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both, and treat the signalled variant as the conventional design (Section 3.1).

Deferring to an oracle under a budget.

Letting a predictor decide most cases and handing a small budget of the hardest ones to something exact is the shape of selective prediction [38] and learning to defer [39, 40], and of the budgeted variant of the latter [41]. We build on that shape in Section 5.7. Two things differ, and both make our case easier. The expert here is the encoder's own search, so it is exact and always available at no test-time cost. What is scarce is the *bits* needed to say what it decided. The selection rule is not learned either. Because the objective is separable over tiles, the optimal set of size s at a fixed multiplier is exactly the s largest regrets, which the encoder can compute. Who to defer and how to learn it is the open question in that literature, and it has a closed form here. What remains is the question we measure, which is how concentrated the regret is.

Allocating a budget over units.

The construction we use is not new and we do not present it as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points, a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has: a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs (Section 5.4). The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling and a binary search on the multiplier. That is the identical structure in a domain with no rate axis, and we read it as evidence that the floor/saturation characterisation is worth stating generally.

How much is there to gain?

Bounding what adaptive inference could achieve is itself a line of work. Hasan et al. [34] derive an oracle bound on efficiency at fixed accuracy, given per-model resource and accuracy, and report 43–121× on ImageNet and 7–81× on HellaSwag. Their bound has a ceiling and no floor, because the largest model in their family reaches the reference accuracy by definition. Spatial adaptivity introduces a floor. Cutting a frame into tiles costs quality even when every tile runs to full depth, so the reference is unreachable at *any* compute and the feasible set of budgets is an interval rather than a ray. Section 5.4 characterises that interval and both of its ends.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact. The remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is the closest to the exact remedy we describe in Section 4.4, and the difference is accounting. It pads every convolutional layer and does not report the cost. We pad only the 0.29% of each block that has any spatial extent, which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it. To our knowledge the estimator view of the padding rule, the measured ordering of four estimators, and the dependence of the penalty on per-tile depth (Section 4) have not been reported.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, and cites channel reductions that yield linear rather than quadratic speedups. Section 5.9 is an instance of that claim inside one loop. Removing 35.3% of the operations buys 29.1% of the time, and 1.2 points of the difference come back from reordering the loop with the arithmetic untouched.

3. Method

3.1 Setting and notation

We set out here the decoder we work on, the notation the rest of the paper uses, and the three configurations we compare.

The decoder.

We work on the intra decoder of DCVC-UF [14]. The analysis side stays frozen throughout: the patch embedding, the chunk encoder, the entropy model and the coded payload are never touched (Figure 1). What we replace is the synthesis trunk that turns the decoded latent into a picture. We call the unmodified public decoder the *released decoder*, and one full-frame run of it is the unit of both cost and quality below.

Tiles and exits.

A frame is cut into square tiles, 256 px on a side unless we say otherwise, and each tile is carried through part of the trunk on its own. Tiles are indexed by t and there are N of them, N=40 at 1080p. The trunk itself is a stack of twelve blocks; we cut it into K groups and attach an exit to each, so exits are indexed by k and exit K−1 is the whole trunk. The first j groups run once over the whole frame and the remaining K−j run per tile, so j is the *split depth*. A tile that leaves at exit k runs groups j to k and skips everything after it. Writing c_k for what a tile costs at exit k, a frame that assigns exit k_t to tile t costs the mean of c over its tiles. Tiles are cut on the trunk's feature grid, which is coarser than the pixel grid, but we quote tile sizes in pixels.

Distortion and the budget.

D(t,k) is the error tile t incurs if it leaves at exit k. It is measured on the path we deploy at inference, a tiled decode, against the released decoder's full-frame decode of the same latent, so what tiling costs is inside it; we call that path the *deployed path* and the table built on it the *deployed table*. Tapping the exits of one full-frame forward pass gives a cheaper *full-frame table* that is not the same quantity, and Section 5 measures the difference. Every result is quoted at a *distortion budget*, the decibels a decoded frame is allowed to fall below the released decoder on that same latent, and the headline budget is 0.1 dB. To allocate is to choose an exit for every tile so that the frame lands on its budget as cheaply as possible. A budget only buys something inside its band, because tiling costs quality before any tile exits early and the shallowest usable exit bounds what can be saved; Section 5.4 measures both ends. Rate is set by a quality index running from q0, the lowest rate, to q63, the highest, and we report five of them: q0, q16, q32, q48 and q63.

symbol	meaning
t, N	tile index and tiles per frame; N=40 at 1080p
k, K	exit index and number of exits, k in 0..K−1; K=6
j	split depth: groups 0..j−1 run full frame, j..K−1 per tile; j=2
c_k	cost of a tile at exit k, as a fraction of one released full-frame decode
D(t,k)	error of tile t at exit k, tiled decode against the released full-frame decode of the same latent
λ	multiplier trading c_k against D(t,k), bisected to the budget
β	the same trade applied to the router's logits
q	quality index, q0 the lowest rate and q63 the highest

Three configurations.

All three choose the same object, one exit per tile, on the same weights and the same coded payload. Configurations **A** and **B** differ only in where that choice is made, and **C** sits between them.

A decides at the encoder. The encoder holds the source, so it knows D(t,k), picks each tile's exit by the Lagrangian of Section 3.5, and transmits the resulting *exit map*, one exit index per tile, which is the analogue of the mode map a standard codec signals. It entropy-codes to ~89 bits per 1080p frame, or 0.021% of a typical bitrate. It costs the encoder about one extra decode per frame and the decoder nothing.

B decides at the decoder. A 144 K-parameter head reads decoded data only and predicts the same map (Section 3.5). Nothing is transmitted and the file stays byte-identical to the released one. It costs the decoder 0.163% of its own multiply-accumulates, which is charged inside every B number we report. Because that head reads nothing the encoder cannot also read, the encoder can run it too, and so knows tile by tile where it will be wrong.

Two of these are deployment modes and we name them so that no claim in this paper is ambiguous about what is sent. *Signalled* FLEX-UF is configuration A: the coded latent is unchanged and a small exit map travels beside it. *Bitstream-identical* FLEX-UF is configuration B: nothing is added, and the decoder reads a file that is byte for byte the one the released encoder produced. Every saving quoted in this paper is labelled with the mode it was measured in.

C interpolates between the two. The encoder signals a fraction ρ of the tiles, the ones where leaving B alone is most costly, and B decides the rest, so ρ=0 is B and ρ=1 is A. Any decoder-side predictor can take B's place there, and Section 5.7, which measures C, tries a second one. C transmits a mask rather than a full map, costs the encoder A's search plus one run of the predictor, and costs the decoder whatever that predictor costs.

3.2 Where the computation is

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are 89.4% of that, which is why we build the ladder across them. Inside one block at C=384 channels, the only operator with any spatial extent is a 3×3 depthwise convolution, and it costs 9C against the block's $8C^2+9C$. That is **0.29%.** We come back to the fraction several times below. Tile borders damage the picture through it, and it is what makes the exact remedy of Section 4.4 affordable at all. It is also why we kept the adapters pointwise.

3.3 The exit ladder

Two consequences follow from the ladder, and both are easy to state wrongly. Exits shallower than j are indistinguishable from each other, because the first j groups run for every tile regardless, and the usable ladder therefore has K−j distinct costs. The second consequence is the *architectural ceiling* S_max = 100(1−c_j)%, reached when every tile takes exit j. For our shipped setting (K=6, j=2, 256 px tiles) S_max = 39.1%. We show in Section 5.4 that this bound is *reached* in practice at low rate, so it behaves as an operating point and not as an asymptote.

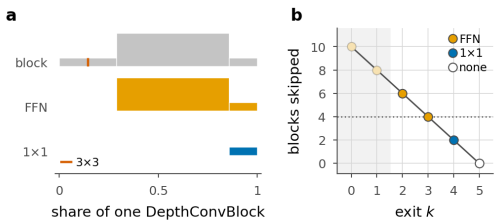

Figure 2. Inside an exit adapter. a, one DepthConvBlock and the two adapters, drawn to scale: bar length is a share of the block, bar height the channel width written. Neither adapter contains the block's only 3×3 (the hairline rule), so neither adds receptive field or seam penalty. **b**, blocks skipped per exit, coloured by adapter; the rule switches to the FFN at four skipped blocks. Lengths counted with hooks off the modules themselves.

3.4 Exit adapters

An early exit hands the shared head a feature the head was not fitted to, and the adapter is the correction. For exits that skip little we use a residual 1×1, Ad(f) = f + Wf with W zero-initialised. For exits that skip four blocks or more we use the pointwise expand/activate/contract pair of the block's own FFN. Both are *pointwise by design*. A 3×3 inside an adapter would add seam damage at the tiles that took an early exit, and those are the tiles least able to afford it. Zero-initialising the last layer makes every adapter exactly the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins and every shallow exit starts from “the decoder as it is”.

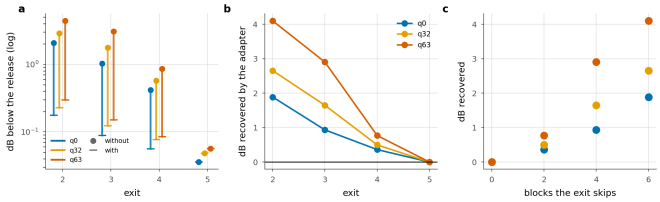

Figure 3. What the adapters are worth. a, each exit with and without its adapter. **b**, the dB the adapter recovers. **c**, the same against the number of blocks the exit skips, which is the design rationale measured.

	q0		q32		q63	
Exit	with	without	with	without	with	without
2	0.177	2.065	0.228	2.884	0.297	4.398
3	0.089	1.023	0.123	1.772	0.151	3.054
4	0.056	0.416	0.077	0.574	0.084	0.853
5 †	0.036	0.036	0.048	0.048	0.056	0.056

Table 4. What the adapters are worth. dB below the released decoder with every tile at that exit, with the trained adapters and with each set back to the identity it was initialised to. † the deepest exit has no adapter by construction and is the control.

Zeroing the adapters measures what they learned, since the identity is exactly what they were initialised to. Without them the shallowest exit costs 4.40 dB at q63, forty-four times the budget, and the adapter buys 4.10 dB of that back. We find the gain grows with rate and with the number of blocks the exit skips, so the design rationale here is a measurement rather than an argument, and the deepest exit moves by exactly zero. We would not describe the adapters as a refinement of the ladder, because without them it has no usable exit.

3.5 Allocation

The ladder and its costs are fixed by this point, and one decision is left. Every tile has to be given an exit, drawn from the usable ones, k ≥ j, and we want the set of choices that keeps the frame inside its distortion budget for the least compute. There are K−j choices per tile and N tiles, so searching assignments one by one is out of the question. What makes the problem tractable is that the tiles do not interact: each contributes its own error D(t,k) and its own cost c_k, and the only thing they share is the single budget the frame has to meet.

Problems of that shape are solved with a Lagrangian. Put a price λ on compute, add λ c_k to the error of exit k, and the budget constraint drops out; what is left is N separate one-tile problems, each a minimum over K−j numbers. Shoham and Gersho [28] showed that sweeping the price this way traces the lower convex hull of what the units can achieve together, and Ortega and Ramchandran [29] made the construction standard in image and video coding.

What λ does is set the exchange rate between compute and error. At λ=0 compute is free and every tile takes whichever exit has the smallest error. As λ rises, compute becomes expensive relative to error, and a tile drops to a shallower exit as soon as what that exit saves is worth more than the error it adds. Compute falls and error grows as λ grows, neither of them turning back, so one value of λ puts the frame exactly on its budget and we find that value by bisection. The rule each tile follows is

$$k^*(t) = \arg\min_k \, [\, D(t,k) + \lambda \, c_k \,] \tag{1}$$

with the minimum taken over k ≥ j. We call k*(t) the choice of the *Lagrangian oracle*: it is made with the true error of every exit in hand, which no decoder has, but it is optimal only within the family of allocations one multiplier can reach. That qualifier is not cosmetic. Section 5.7 exhibits allocations that are cheaper at the same distortion than anything this rule produces, so *oracle* here means best under a single multiplier and not the global constrained optimum. Where we mean the latter we say so. Both quantities are available *to the encoder*, which holds the source, so configuration A can run this search exactly.

What the search costs the *encoder* depends on how the table of D(t,k) is built, and the two ways of building it are further apart than they look. Take one tiled decode at 1080p as the unit. The deployed table of Section 3.1 needs one tiled decode per exit, which comes to 4.6× the unit rather than K times it, because a shallow exit is cheaper to run than a full one. The full-frame table taps every exit off a single full-frame pass and comes to 1.4×. Section 5 shows that the full-frame table must not be used to *report* quality, but it can still be used to *rank*. Running the argmin on it, and the deployed path only to check the budget, we find the two searches agree on 85% of tiles and land within 0.4 points of saving of each other (19.0% against 18.6%, both under budget). The exact search therefore costs 4.6 decodes and a practical encoder need not pay it; ranking on the full-frame table is the one extra decode per frame we quote for A.

The decoder cannot run the argmin. D(t,k) is the error against the source, and the source is the one thing a decoder never receives. The gap is one of information: the errors the argmin compares depend on a picture that is not in the bitstream, so no decoder-side model recovers them, however large it is. What a decoder can do is estimate which exit the Lagrangian oracle would have picked, and configuration B therefore *predicts*. Its head reads what the decoder already holds, the feature at the split point, the decoded latent y-hat, the entropy model's scales and the quality index. For each tile t it emits a vector of logits z_t, one entry per exit, and the tile takes

$$\hat{k}_t = \arg\max_k \, [\, \log \mathrm{softmax}(z_t)_k - \beta \, c_k \,] \tag{2}$$

The first term is the head's score for exit k, on a log scale. The second is the price on compute that λ carries in the Lagrangian: raising β takes more away from the deep exits than from the shallow ones, so more tiles are handed to cheap exits and the frame again gets cheaper and worse. We bisect β against the budget just as we bisect λ.

Why a logarithm. The oracle's rule adds a distortion to a price. The head does not produce a distortion; it produces a distribution over exits, and the quantity that plays the same additive role is the surprisal −log p, which is what the head is trained to make small on the oracle's label. Writing the rule with log softmax puts the two terms in one currency: what exit k costs in belief against what it costs in compute, with β the exchange rate between them. It also makes the comparison invariant to a constant added to every logit, which the softmax removes, so βc_k is the only thing tilting it. Using the probability itself would set a number bounded in [0,1] against a cost measured in decodes, and would compress every distinction between confident exits into a range near one, where β would have to move by orders of magnitude to change any decision.

The bracket for the bisection is $[-2\times10^4, 2\times10^4]$, which is wider than it looks like it needs to be. A trained head is confident: its log-probability gaps reach the hundreds, and β has to be able to outweigh them before the allocation moves at all. A narrower bracket we tried first, [−50, 50], never reached the all-deepest allocation and so never bracketed the budget from below.

The head, exactly. Three things enter. The first is the feature at the split point, 384 channels at one eighth of frame resolution, which is the last thing every tile shares. The second is the decoded latent ■ concatenated with the entropy model's scales σ, the predicted Gaussian width used to code each latent position: σ is already computed during the decode and is, position by position, an estimate of how hard that position was to code. The third is the quality index. Each of the first two is projected by a 1×1 convolution, to 48 and 32 channels, and then reduced to one vector per tile by taking the mean and the standard deviation over the tile. That gives 96 + 64 + 1 = 161 numbers per tile, which pass through LayerNorm and a three-layer perceptron of width 256 with SiLU activations, ending in K logits. The whole head is 144,030 parameters and 0.162% of the decode it is deciding about, almost all of it in the 1×1 on the stem, which is the only part that runs per pixel. The perceptron runs once per tile, forty times for a 1080p frame, so its width is nearly free.

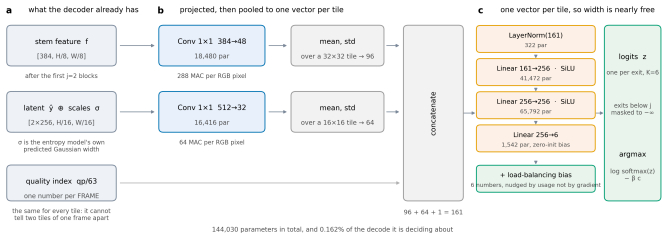

Figure 4. The router head. a, the three things it reads, all already held by the decoder. **b**, each projected by a 1×1, then reduced to one vector per tile by its mean and standard deviation. **c**, a three-layer perceptron, one pass per tile. Every shape and parameter count is read off the module when the figure is drawn.

Two details matter for reproducing it. Exits below the split depth do not exist, and their logits are masked to −∞ rather than to a large negative constant; nothing in the objective penalises a constant offset, the raw logits drifted to around -10^4 during training, and a -10^4 mask then became the *largest* entry in the row. The balancing term is a per-exit bias added to the logits before the decision and updated by usage rather than by a gradient, which is what makes it loss-free.

One head for every quality index. The index is an input, not a selector. During training it is drawn uniformly at random for each batch, so one set of weights covers all 64 operating points, and at test time the same head runs at every rate. It enters as a single number per frame, which means it can tell the head which operating point it is at and cannot, even in principle, distinguish two tiles of the same frame. That is why it stays live in every variant of the input ablation: the variants then differ in per-tile information and in nothing else, and the variant with only the quality index is the control that measures what agreement is reachable with no per-tile information at all.

The head is trained against a *frozen* decoder, so the exits it chooses between do not move while it learns. Its target is the oracle's choice k*(t) and the loss is a cross-entropy to that target, with each tile weighted by its *regret*, meaning by how much the bracket in the Lagrangian grows if the head's exit is used in place of the oracle's. A tile whose two best exits are nearly tied then counts for less than one where the wrong choice is

expensive. A router like this can collapse onto a single exit during training, so it carries a balancing term; ours is loss-free [16] and is biased toward the oracle's own exit distribution instead of toward uniform. At a high λ the oracle genuinely does send every tile to one exit, and forcing spread there would force mistakes. Section 5.5 measures what B gives up against A.

3.6 Training

All exits are decoded every step and the objective is

$$\mathcal{L} = \mathcal{L}_{\text{RD}} + w_a \mathcal{L}_{\text{anchor}} + w_d \mathcal{L}_{\text{distill}} \tag{3}$$

where L_RD is the released rate-distortion loss averaged over exits, weighted per exit by fixed α_k and scaled by λ_rd, the codec's own rate-distortion weight, which is a different multiplier from the λ of the Lagrangian above. The anchor term L_anchor pins the deepest exit's reconstruction to the released decoder's, so the reference every saving is quoted against cannot drift away underneath the measurement. The distillation term L_distill supervises the adapters in feature space, matching the feature at exit k to a stop-gradiented copy of the feature at exit k+1. We work in feature space and not in pixels because the head is a fixed map from feature to RGB, so matching the deeper feature is the stronger constraint, with 384 dense channels of target instead of 3. Each exit imitates its neighbour, exit k following exit k+1, and not the deepest exit, for the reason given in [19].

We train the adapters *through the tiled decode path they are deployed in*. Training them full frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB, so the sign of the effect flips.

4. The price of tiles

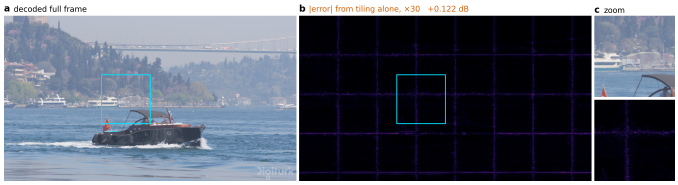

Figure 5. The artefact, before anything is done about it. Bosphorus at 1080p, quality index 63, zero padding at the tile border. **a**, the frame decoded in one piece. **b**, the same bitstream and weights decoded in 256 pixel tiles, every tile at full depth: the difference between the two, amplified 30 times. **c**, the boxed region of each.

A 3×3 depthwise at feature position (x,y) computes a weighted sum over its neighbours. Decoded full frame, those neighbours exist. Decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the affected region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is

$$1 - \left(\frac{F-2b}{F}\right)^2$$

At our shipped F=32, b=8 that comes to 0.750. Three quarters of the tile is affected, so what we are looking at is a structured error over most of the tile and not a thin border at its edge.

That fraction tells us which pixels are affected and not how badly, and it turns out to be the wrong predictor of the penalty. We swept the split depth, which sweeps b from 12 to 0 with b=0 as an exact zero-seam control, and measured

$$\text{seam} \propto b^{\alpha}, \qquad \alpha \approx 2 \tag{5}$$

with α = 2.38 at q0, 2.22 at q32 and 1.93 at q63, and r = 0.98–0.995 in log-log. Fitted with one free scale, the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. Fixing the exponent at exactly two costs a factor of two, 31–38%, so the square is the right shape without being quite the right law. The exponent has a reading. The count of affected pixels grows like perimeter times depth, and the error accumulated in each of them grows with how many convolutions reached it. Once every pixel has been touched the area law saturates and the penalty does not, which is where the area law fails as a predictor.

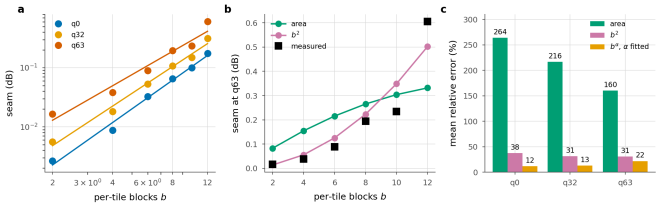

Figure 6. The seam against per-tile depth. a, the measurement with the fitted power law. **b**, both models against q63, one free scale each. **c**, mean relative error. The area fraction saturates once the border reaches every pixel; the penalty does not.

4.1 Padding is an estimator

Border padding is an *estimator* of the unseen neighbour, and the seam is its error. The table below measures four of them, with early exit switched off so that tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	**0.071**	**0.123**	**0.218**
Linear extrap.	0.198	0.286	0.384
AR(1) per channel [11]	0.061	0.107	0.197

Table 5. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better. Replication, a zero-order hold, beats linear extrapolation by a wide margin, and the fitted AR(1) wins on quality while costing +10.7% of decode wall-clock.

A higher-order estimator is worse. Extrapolating the local gradient past a boundary amplifies whatever noise sits on that boundary, and assuming local constancy does not. The gap is wide, 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, and it costs 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, so we drop it.

4.2 What actually removes the seam

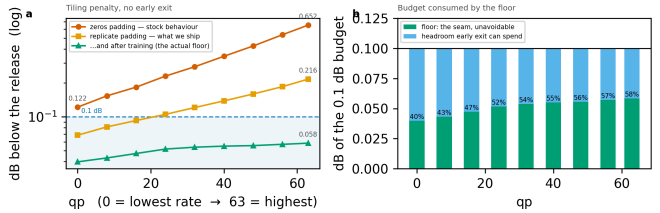

Figure 7. The tiling penalty across the whole rate range, every tile at full depth. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Panel (b): the floor is charged *inside* the distortion budget and consumes a growing share of it.

Tile size is free in arithmetic. A tiled decode's MAC count does not depend on it at all, and the damage scales with the tile perimeter, as the table below shows. What tile size costs is routing granularity, 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 6. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the power law above predicts, and costs no computation.

The largest factor of all is the easiest one to miss. On the untrained ladder, replicate padding leaves 0.218 dB at q63; after training, the same setting leaves 0.072 dB. So more than two thirds of what the estimator could not fix is absorbed by weights learning to live with it. That single change removes more of the seam than any module in this paper does.

4.3 A learned deblocking filter, and why we reject it

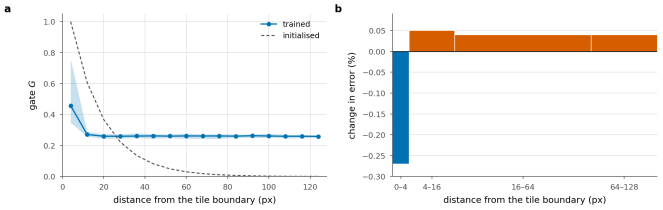

Figure 8. A learned deblocking filter, gated by position within a tile and applied once to the stitched frame, for 0.95% of the decode. **a**, the trained gate against distance from the boundary, beside its initialisation. **b**, the change in error by that distance; bar width is the share of pixels, so bar area is the contribution to the frame. It wins on the 6.2% of pixels nearest a boundary and loses on the 94% beyond.

What a standard codec does about a partition boundary is deblock it, with a filter applied after reconstruction [9, 21]. The tile lattice is known exactly at training and at inference, so ours can be *told* where to look instead of having to infer it

$$\mathrm{Rep}(f) = f + G[\,x \bmod P,\ y \bmod P\,] \cdot \mathrm{PW}(\mathrm{WSiLU}(\mathrm{DW}_{3\times3}(f))) \qquad (6)$$

with PW a pointwise convolution, DW a depthwise one, WSiLU the weighted SiLU of the DCVC line [33], G a P×P gate shared over channels, P the tile pitch in feature samples, and G initialised at $\exp(-d/\tau)$ in the distance d to the nearest tile boundary with τ a fixed decay length. It costs 0.95% of the decode.

It does not earn that. Splitting the per-pixel error by distance from the nearest tile boundary, we find the filter gains 0.27% in the 0–4 px ring, which is 6.2% of pixels, and loses 0.04–0.05% everywhere else. Give it a *perfect* gate, with zero correction in the interior and the boundary gain unchanged, and the ceiling on what it could earn is ≈0.0008 dB, for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode, and this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Only 0.29% of each block has spatial extent, so the exact fix is affordable. Give the 3×3 its real neighbours across the tile border, a *halo exchange* narrowed to the one operator that needs it; local padding [31] does the same at every layer. The cost is +0.032% of the decode, thirty times less than the deblocking filter. The fix is also exact. At uniform depth a tiled decode with the exchange is bit-identical to a full-frame one, once the comparison is not confounded by the filter, which runs on the stitched frame and has no full-frame counterpart. Measured: 1.13e-2 with the filter on, **exactly 0** with it off, against 6.06e-2 for replicate padding.

The exchange also removes most of the floor. Switched on at inference, it drops the floor from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, which is 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63.

The reason is structural, and it is written into the condition on the exactness claim. The exchange is exact when every tile is at the *same* depth, and routing deliberately violates that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation is free to give adjacent tiles any depths it likes. The exchange makes a tile depend on its neighbours' features, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is an arrangement the weights have never seen, and that mismatch costs more than the seam it removed.

The natural reading of the exactness result is that the exact fix should simply be adopted, and that reading is wrong. We report the negative result for that reason, and because the tension between the halo exchange and routing belongs to the combination itself, so any spatially-adaptive decoder will inherit it. One caveat keeps the finding from being final, the same one that applies to the deblocking filter. This model was trained with padding, and a run trained *with* the exchange could reverse the result. The burden of proof lies with that run.

5. Experiments

Setup.

We fine-tune the decoder on 512×512 crops from OpenImages with the encoder frozen ($\max|\Delta| = 0$ is asserted every run). One λ_{rd} is drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights covers the whole rate range. We evaluate on one intra frame from each of the 53 sequences of the common test set (CTC: UVG, MCL-JCV and HEVC classes B, C, D and E), one intra frame each.

Measurement protocol.

Two details of the protocol move the numbers enough that we state them here. The per-tile distortion table has to be built on the *deployed* decode path, that is, one tiled decode per exit, rather than by tapping the exits of a single full-frame forward pass. Tapping is the natural implementation and it is correct for training. But with a full-frame reference on the other side of the ratio, the tiling penalty cancels, and the quality reported is that of a decoder nobody ships. On our model the gap is +0.035 dB at the deepest exit and +0.008 dB at the shallowest. It is not a constant offset; it grows with how many blocks ran per tile, so we cannot correct for it after the fact. The second detail is that once the allocation is chosen, we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty sits inside every number. Savings are fractions of the released decoder's cost. Our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points.

5.1 Main result

Figure 9. What the saving looks like. The same bitstream decoded by the released decoder and by ours at the 0.1 dB operating point, 30.5% fewer multiply-accumulates. The crop is the tile that gave up the most quality, chosen automatically, so it shows the method's worst case on this frame rather than a flattering one.

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	29.9	25.6	20.9	18.3	15.6	**22.1**
0.20 dB	39.1	37.8	33.7	31.7	29.6	**34.4**
0.30 dB	39.1	39.1	39.1	37.9	35.8	**38.2**
0.50 dB	39.1	39.1	39.1	39.1	39.1	**39.1**

Table 7. Decoder MACs saved (%) against the released decoder, per quality index and distortion budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere; past that point a looser budget buys nothing.

One word on the budget before the numbers. A tenth of a decibel is an engineering convention, chosen because it is small against the spacing of the rate points and because codec work has long used differences of this size as a working tolerance. It is not evidence that the difference is invisible, and we make no perceptual claim for it. The 0.2, 0.3 and 0.5 dB rows are there so a reader who disagrees with the choice can read off another one.

The table carries the headline numbers. At 0.1 dB the method saves 29.9% at the lowest rate and 15.6% at the highest, and 22.1% on average, at a BD-Rate cost of 0.88%; that is, the compute is worth about the same as a 0.88% increase in bitrate. We do not read the fall with rate as an artefact of the ladder. High-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate as well; both effects push the same way.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	111
FLEX-UF, all tiles deepest	458	101.0	114
FLEX-UF, 0.1 dB budget	353	77.9	78
FLEX-UF, architectural ceiling	263	58.1	–

Table 8. Decoder complexity at 1080p. Our deepest exit costs slightly more than the released decoder because it still pays the deblocking filter; that 1.0095, not 1.0, is what every saving in this paper is *not* divided by. Wall-clock is the sorted loop of Section 5.9.

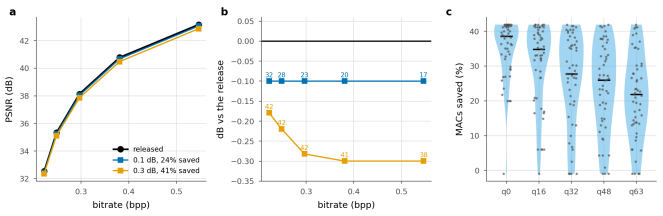

Figure 10. The plane a codec is read on, and the spread behind the mean. a, operating points against the released curve. **b**, the same with the quality axis expanded; labels are compute saved. **c**, per sequence at a matched point near 0.1 dB; one dot per sequence, bar is the median.

Panel c shows the distribution that the headline averages over, and it is wide. At q0 the median sequence saves 38.6%, with an interquartile range of 32.0–41.5, while the worst saves −1.0%. The worst cases are the low-resolution sequences of Section 5.3, where two tiles leave nothing to allocate. Reporting the mean alone would hide both ends.

5.2 Is per-tile adaptivity necessary?

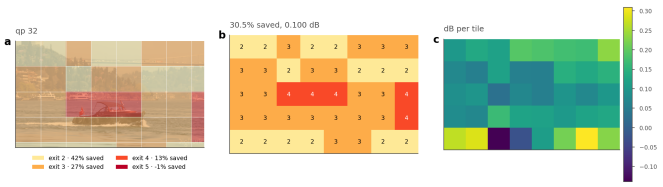

Figure 11. Where the decoder spends. Bosphorus at q32, a 0.1 dB budget. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up, against its *own* full-depth reference. Water and sky leave at the shallowest exit; the boat and the shoreline run deep.

q	Allocation	ΔPSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	39.1
	uniform, exit 3	0.093	24.2
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	29.9
	rate-ranked (free)	0.107	29.9
	oracle	**0.100**	**29.9**
16	uniform, exit 2†	0.219	39.1
	uniform, exit 3†	0.118	24.2
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	25.6
	rate-ranked (free)	0.106	25.6
	oracle	**0.100**	**25.6**
32	uniform, exit 2†	0.282	39.1
	uniform, exit 3†	0.147	24.2
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	20.9
	rate-ranked (free)	0.107	20.9
	oracle	**0.100**	**20.9**
48	uniform, exit 2†	0.333	39.1
	uniform, exit 3†	0.176	24.2
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	18.3
	rate-ranked (free)	0.107	18.3
	oracle	**0.100**	**18.3**
63	uniform, exit 2†	0.396	39.1
	uniform, exit 3†	0.206	24.2
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	15.6
	rate-ranked (free)	0.110	15.6
	oracle	**0.100**	**15.6**

Table 9. Adaptivity against two controls at the 0.1 dB budget. † marks uniform depths whose distortion exceeds the budget, so they are not admissible allocations at all. “Random” draws each frame's map from the oracle's own exit histogram and shuffles it across tiles, which preserves the average cost and the mix of depths while discarding the content dependence.

The obvious alternative to routing tiles is to run every tile at the same shallower exit and accept the loss. The table puts that option, together with two stronger controls, at matched compute.

The uniform rows answer the question directly, and the answer sharpens with rate. At the lowest rate a static decoder can reach exit 3 inside the budget and save 24.2%, against the Lagrangian oracle's 29.9%. At q48 and q63 the only uniform depth that fits is the deepest one, which *costs* 1.0% instead of saving anything. At those two rates, then, the comparison is between 18.3% and 15.6% against nothing at all, not against something smaller. Averaged over rates, the best static allocation saves 9.7% where the adaptive one saves 22.1%.

The two shuffled rows separate effects that are easy to conflate. We keep the oracle's own exit histogram, so the mix of depths and hence the average cost are unchanged, and assign it to tiles at random. Quality falls sharply. What the allocation buys is knowing *which* tiles can afford to run shallower; running some fraction of them shallower is worth nothing by itself. The oracle-histogram bit ranking keeps the same histogram and orders it by a signal the decoder already holds, namely the bits the entropy model spent on each tile. This is the cheapest router we can think of. It has no parameters and no training, and the number is

available before the trunk starts. It is also the natural analogue of the confidence rules used by early-exit classifiers [3, 20], which likewise read a signal off the network's own output.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute, while the oracle-histogram bit ranking costs 0.110 dB. That is 75% of the oracle's advantage over chance, at 0.68–0.79 agreement with the oracle's map. A learned router therefore has to beat a calibrated bit rule that is already three quarters of the way there. We would not have run that comparison without this control, and we suggest that any adaptive-inference paper report it. Given the Lagrangian oracle's histogram, what we measure here is *ranking* and nothing else. Section 5.6 removes that crutch and turns the same signal into a complete routing rule, which matches the trained head across the whole rate range.

5.3 Resolution, and the granularity of a tile

Figure 12. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ. The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	33.7	26.2	20.2
UVG	1920x1080	40	7	30.6	23.1	15.5
HEVC_B	1920x1080	40	5	31.1	18.2	9.7
HEVC_E	1280x720	15	3	29.1	18.4	15.5
HEVC_C	832x480	8	4	18.0	1.7	2.5
HEVC_D	416x240	2	4	8.8	2.5	2.5

Table 10. Saving by test class (%), at a 0.1 dB budget set globally over all 53 sequences rather than per class. “Tiles” is how many 256 px tiles a frame of that resolution contains. The saving tracks tile count much more closely than it tracks content.

Completing the test set exposed a dependence that the 1080p-only subset had hidden. The table groups the saving by class. At 1080p, where a frame is 40 tiles, we save 33.7% (MCL-JCV), 33.1% (UVG) and 33.8% (HEVC B) at the lowest rate, three classes of very different content within three points of each other. At 832×480 a frame is 8 tiles and the saving is 18.0%; at 416×240 it is 2 tiles and 8.8%. At high rate the small resolutions fall to 2.5% against 20.2% for 1080p.

The natural explanation is granularity. With two tiles per frame there is almost no allocation left to make, and the Lagrangian degenerates towards a uniform choice. That suggests a fix. Choose the tile size relative to the frame, since the MAC count does not depend on tile size at all and only the seam does.

We tested that fix and it mostly does not work. We compare the 256 px tiling against a 128 px one at q0, which multiplies the tile count by 3.4. MCL-JCV (1080p) goes from 36.3 to 34.3, HEVC B (1080p) from 33.8 to 32.1, HEVC C (832×480) from 20.9 to 17.6, and HEVC D (416×240) from 11.3 to **13.7**.

Smaller tiles help only at the smallest resolution, where the count goes from two to eight. Everywhere else they cost between 1.7 and 3.3 points, including at 832×480, where the count rises from 8 to 28 and the saving still falls. Beyond a modest number of tiles, the extra seam outweighs the extra granularity. We report the comparison as indicative, since the two tile sizes come from different training runs and tile size is therefore confounded with training. But the direction is consistent, and it is enough for us to say that a resolution-adaptive tile size is not the easy win the granularity argument suggests.

5.4 The band a distortion budget works in

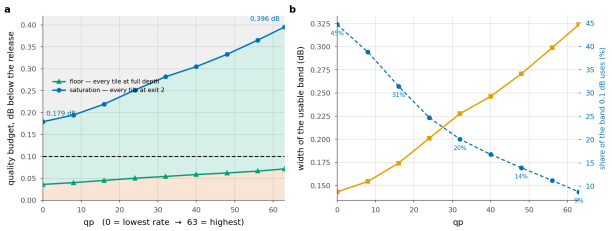

Figure 13. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile already takes the cheapest exit. The 0.1 dB budget uses 45% of the band at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D_min	0.036	0.045	0.054	0.062	0.072
Saturation D_sat	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 11. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

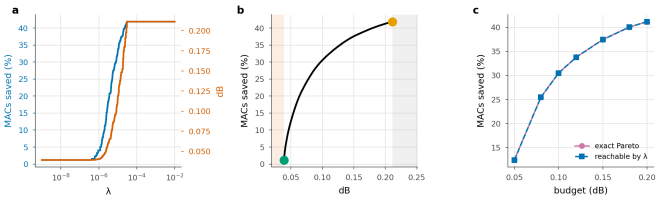

Figure 14. The structure of the allocation, on one frame. **a**, compute and distortion are monotone in the multiplier. **b**, the frontier between floor and saturation; shaded regions are infeasible or wasted. **c**, what the Lagrangian reaches against the exact Pareto set, enumerated by dynamic programming.

One assumption is worth naming before any of this. The argument treats the frame's distortion as a sum over tiles, while the deblocking pass runs on the stitched frame and therefore sees the whole exit map at once, so the tiles are not strictly independent. We measured the size of that coupling over 60 allocations and the largest mismatch between the separable prediction and a real decode is 5.5×10^{-5} dB, four orders of magnitude below the budget. We proceed as if the objective were separable and treat that as measured rather than assumed.

The construction has enough structure to state as propositions, and we check each one numerically instead of asserting it (scripts/verify_theory.py, seven of seven). The allocation decouples per tile; compute is non-increasing and distortion non-decreasing in λ; the sweep traces the lower convex hull and therefore cannot reach an interior point of the achievable set; λ=0 reaches the least distortion the ladder can produce; beyond a finite λ, computable in closed form, the allocation is the constant map to exit j at cost exactly c_j; and the ceiling is a function of the split depth alone.

The third of those has a practical edge. The sweep reaches only hull vertices, so a budget that falls between two of them cannot be met exactly. We measured what that costs by enumerating the *exact* Pareto set with dynamic programming over tiles, which is feasible because there are only four distinct exit costs. The sweep reaches 95 allocations against a Pareto set of 635, and the loss from convexity is at most 0.05 saving points. For this problem the standard construction is essentially optimal.

It matters *why* that loss is small, since the reason is structural and points at what would make it smaller. The reachable costs form a lattice, and its

spacing is set by how much the mean cost moves when the multiplier crosses a breakpoint. If m tiles each move to the next exit there, the spacing is at most m·max(c_k+1 − c_k)/T saving points. The convexity loss cannot exceed the largest spacing, because a budget between two levels is served by the lower one. Here m=2, since tiles with identical rows switch together, and the bound is 0.745 points; the measured spacing sits exactly on it. Nothing in the expression depends on content or rate, and the bound falls as 1/T: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine granularity is what makes the Lagrangian relaxation lossless in practice, and no property of the images is doing that work.

Two numbers bound what any budget can do. The **floor** is the distortion of a tiled decode with every tile at full depth, which is pure tiling penalty: 0.036 dB at q0, rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it reaches the ceiling, and a larger one reaches nothing more.

One consequence of that is worth stating plainly. At q0 the ceiling is reached at 0.179 dB, so the architectural bound behaves as an operating point rather than as an asymptote. Past it, the limiting factor is the *ladder* and not the budget, which is an argument for a finer ladder before a looser budget. The same two numbers pick out a narrow regime: budgets in [0.179, 0.194) dB saturate the lowest rate and nothing else, a range 15 thousandths of a decibel wide.

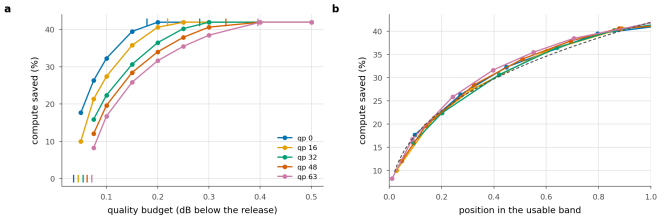

Figure 15. The band is the rate dependence. a Saving against the budget, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own band. Five curves become one; dashed is the one-parameter power law fitted to all of them.

And the band is almost all of the rate dependence. At a matched decibel the five rates are 15.5 points apart. Rescale the budget axis onto each rate's own band, with the floor at 0 and saturation at 1, and they collapse onto a single master curve, 1.7 points apart on average and 2.2 at worst. The answer to "how much does a 0.1 dB budget buy at this rate" is therefore, to within a couple of points, "where does 0.1 dB sit in this rate's band". The floor and the saturation point are both in closed form and both cheap to measure. What they leave over is small enough that a deployment could calibrate the two ends and read the rest off one curve. That curve is a one-parameter power law, saving ≈ C·u^0.38, with C the architectural ceiling and u the position in the band. We fit it in log space over all five rates and get $R^2 = 0.989$, worst residual 2.0 points. The claim we make is the *rescaling*: measuring the budget in units of each rate's own band is what collapses the curves. The exponent is an empirical fit to that collapse and not a law. Inverse fits to the same frontier disagree in it by up to 40%, so we report it as a description of these curves and read nothing into its value.

The collapse appears robust across two checkpoints. We repeated the measurement on a different training run, BEST, a separate recipe taken at a different epoch, and the same thing happens. The rates are 16.9 points apart at a matched decibel and 1.6 after rescaling, and the collapsed curve is again a power law, $R^2 = 0.984$. The *exponent* does not carry over: 0.33 there against 0.38 here, so it describes a trained decoder and not the architecture. What replicates is the collapse. Within a checkpoint, the rate dependence of the trade-off is the rate dependence of the band.

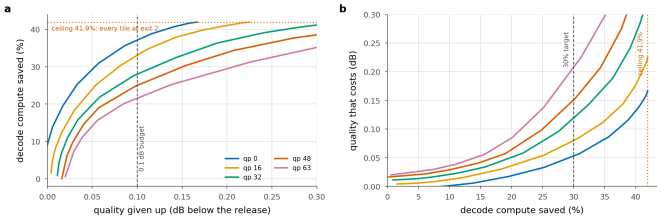

Figure 16. The trade-off, whole. a What a budget buys, per rate; the ceiling is 39.1% and q0 reaches it at 0.179 dB. **b** The same relation inverted, so it reads as what a saving target costs. The three budgets reported elsewhere are three points on this curve.

5.5 Signalled versus predicted allocation

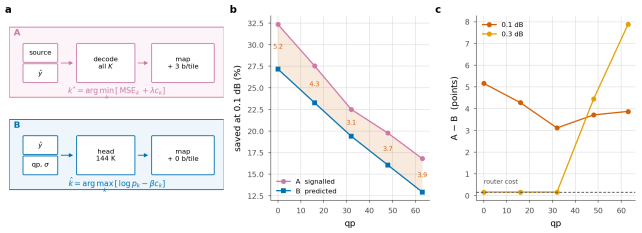

Figure 17. Who decides. a, the two decision paths side by side: the encoder's search over all K exits per tile against the head's forward pass over decoded data. **b**, saving at 0.1 dB; shading is what a bit-exact bitstream costs. **c**, that cost is smallest at q0\ and grows in both directions.

	0.1 dB	0.3 dB				
q	A	B	gap	A	B	gap
0	29.9	26.8	**3.1**	39.1	39.1	**0.0**
16	25.6	18.4	**7.3**	39.1	39.1	**0.0**
32	20.9	13.2	**7.7**	39.1	39.1	**0.0**
48	18.3	8.3	**10.1**	37.9	37.0	**0.8**
63	15.6	5.1	**10.5**	35.8	34.1	**1.7**

Table 12. Signalled against predicted at two budgets, same checkpoint and test set, with one router trained against the oracle on the deployed table at a single λ.

The table above compares the two, and what it prices is exactness against bits.

Not signalling costs 3.1–10.5 points, roughly flat across rate, for zero added bits and a byte-identical file. The gap is smallest at q0 and widens toward both ends of the rate range.

It tracks |β|, the cost multiplier the bisection has to apply to move a router trained at one λ onto another operating point. At q0 the required β is 0, essentially none, since that is where the training λ lands, and there the gap is at its minimum. At q0 it is 0 and the gap is 10.5. A large β in either direction lets the cost term dominate the logits, which discards the content ranking the router learned. The remedy is a router per operating point, and a deployment would have one anyway: a single set of weights covers all rates, and a 144 K head per rate is 0.3% of the model each.

Loosening the budget closes the gap only where the budget saturates the ladder. At 0.3 dB the gap is 0.0 points at the three lowest rates. That is exactly the router's own 0.163% of decode, so its *prediction* is free there; both configurations send every tile to the cheapest exit, and nothing is left to predict wrongly. At the two highest rates 0.3 dB does not saturate, and the gap *widens* to 1.7 points. A looser budget gives the allocation more room to be wrong in as well as more room to be right.

For a system that trains once, the single-router number is the honest one, so that is what we report. It understates what B can do.

Retraining with the mask fixed raises agreement and does not simply raise the saving. The head above was trained against a suppression term sitting above its own logits (below). We retrained it with the mask fixed, same recipe and same λ. Held-out agreement rises from 0.718 to 0.808, and the deployed saving moves by +3.1 points at q0

and –3.4 at q63. We find the retrained head ahead where the required β is small and behind where it is large, which is the same axis the gap runs along. That is the second time in this paper that agreement with the oracle has moved opposite to the quantity that matters. We keep the original head for every other measurement so the comparisons stay on one system, and we read the retrain as evidence that the mask cost the head real capacity, and that agreement is not the objective.

An earlier version of this measurement put the gap at 1.7 points at q0, rising to 13.3 at q63. The exit mask was at fault. It suppresses exits below the split depth by assigning -1e4, the head's own logits had drifted to that scale, and the suppressed entries were therefore the largest in every row. A large share of every tile went to the cheapest exit for a reason unrelated to its content. The mask is now negative infinity. Fixing it costs 3.5 points at q0, where the accident happened to agree with the oracle, and buys 9.4 at q63, where it did not.

5.6 A router with no parameters

Before a 144 K head is worth its 0.163% of the decode, it has to beat what the decoder already knows. The entropy model has produced one number per tile before the trunk runs, and at no cost: how many bits that tile's latents took.

We turn it into a routing rule with no learned parameters. We model the per-tile distortion as rank-1 in the log domain

$$\log D(t,k) \approx \alpha \log b(t) + c + \log \varphi_k \tag{7}$$

where b is the tile's bit count normalised by the frame mean and ϕ is a K-vector saying what each exit costs on an average tile. We fit (α, c, ϕ) by least squares, leave-one-sequence-out, so that no sequence contributes to the profile that routes it. The same Lagrangian the oracle runs is then run on the surrogate. Nothing is signalled and nothing is trained; the arithmetic is one scalar per tile.

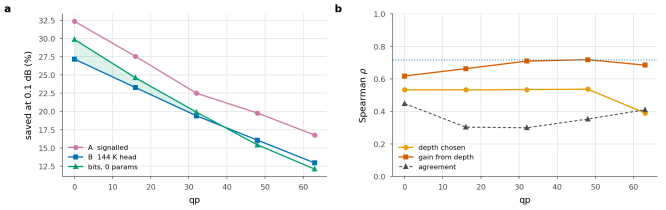

Figure 18. The calibrated bit rule. a Saving at 0.1 dB; shaded where the calibrated bit rule beats the trained head. **b** What a tile's bit count is correlated with, against how often the rule agrees with the oracle outright; dotted is the head's own held-out agreement.

q	rate-rank	B router	A signalled	ρ depth	ρ spread
0	**27.8**	26.8	29.9	+0.54	+0.61
16	**22.7**	18.4	25.6	+0.52	+0.66
32	**18.2**	13.2	20.9	+0.53	+0.71
48	**14.7**	8.3	18.3	+0.53	+0.71
63	**12.0**	5.1	15.6	+0.38	+0.68

Table 13. The calibrated bit rule, 0.1 dB, same test set; it is the "rate-rank" column. Bold where the rule beats the trained head. ρ are Spearman correlations between a tile's bit count and, respectively, the depth the oracle assigns it and the distortion it stands to gain from that depth.

It beats the trained head at every rate. The zero-learned-parameter rule is ahead of the 144 K router at all 5 measured rates, by margins that run from half a point at q48 to 6.9 points at q0. A head trained on this decoder against this oracle therefore returns nothing over a rule with no parameters at all, and it carries 0.163% of the decode that the calibrated bit rule does not.

Why it works is not that it agrees with the oracle. It agrees on 0.28–0.46 of tiles, well below the router's 0.718, and still saves more at every rate. Agreement weighs a disagreement on a tile where two exits are within a hair of each other exactly as heavily as one where the choice is most of the frame's error, and most tiles are the former. The ordering is what the rule gets right. Bits correlate with the *spread* across the ladder, that is, with how much a tile stands to gain from depth, at ρ_spread = 0.61–0.71 at every rate.

At a looser budget it stops being a baseline and becomes the answer. At 0.3 dB the calibrated bit rule matches the oracle exactly at the three lowest rates, where both reach the same architectural ceiling, comes within 0.2 points of it at q48, and gives up 34.3% against the oracle's 35.8% at q63. That is ahead of the trained head at every rate, by up to 0.7 points. The head is charged for itself and the rule is not. At a loose budget the head's ordering also has less left to contribute, because once the budget saturates the ladder there is little ordering left to get right, and the rule gets it. At 0.5 dB it reaches the ceiling at every rate and its assignment is *identical* to the oracle's on every tile.

What it cannot do is see anything beyond that ordering. A rank-1 model in the level assigns every tile the same relative profile over exits, so b only decides where on the ladder a tile falls, never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head should be earning its parameters on. Ours does not earn them at any rate we measured.

q	A signalled	B router	bits, 0 params
0	34.0	31.6	30.9
16	27.7	25.1	**27.1**
32	22.3	16.5	**22.4**
48	19.8	13.3	**19.1**
63	17.2	9.0	**16.2**

Table 14. The same comparison on a second training run (BEST), 0.1 dB, same test set. Bold where the calibrated bit rule beats that run's own trained head.

It replicates on a second training run. BEST is a separate recipe at a different epoch, with its own router trained the same way. There the calibrated bit rule beats the trained head at 4 of 5 rates, by up to 7.2 points, and comes within 3.1 points of the *oracle* everywhere. The margins there are wider than here, and the one rate it concedes is the only rate on either checkpoint where the head finishes ahead, by under a point. We do not read that as showing a learned head cannot be worth its parameters, only that neither of our training runs produced one that is. The calibrated bit rule stays close to the oracle on both.

Per-block bit allocation is a standard quantity in learned compression, where it is something to *choose*; block-level rate control sets it so that complex regions get more bits [42]. We read the same number in the other direction, after the fact and at the decoder, as a statement about how hard a region was. It costs nothing because someone else has already paid for it.

q	bits	0.1	0.25	0.5	0.75	0.9	head
0	**29.8**	27.9	27.9	27.9	28.0	27.9	28.0
16	**24.1**	23.3	23.0	23.0	22.8	22.7	22.8
32	**19.6**	19.5	19.4	19.3	19.3	19.3	19.3
48	14.9	16.9	**17.0**	16.8	16.7	16.7	16.6
63	12.4	**13.2**	12.4	11.7	11.6	11.4	11.4

Table 15. Blending the two decoder-side signals at 0.1 dB, both normalised to unit mean, weight w from the calibrated bit rule to the head's ordering. Bold is the best per rate.

Are the two signals complementary? Barely. We normalise both surrogates to unit mean and blend them with one weight w, where w=0 is the calibrated bit rule and w=1 the head's ordering. The best blend is w=0 at the three lowest rates and a small head weight at the two highest, worth +2.1 points at q48 and +0.9 at q63. The head reads the entropy model's scales, so it already has most of what the bit count carries; what it adds is confined to q48 and q63.

A learned component should be measured against the free alternative, and it rarely is. In adaptive inference the usual controls are a uniform

allocation and a random one. Both are much weaker than a decoder-side signal that is already lying around.

5.7 Signalling only what the router gets wrong

The encoder knows tile by tile where the router will be wrong, because it can run that router itself (Section 3.1), and nothing obliges it to correct every one of them. That is the room configuration C works in.

We choose the ρN overridden tiles by Lagrangian regret, Δ(t) = L(t,k-hat) − L(t,k*) with L(t,k) = D(t,k) + λc_k the objective of the Lagrangian in Section 3.5, so Δ(t) is exactly what that objective loses on tile t by staying silent. We hold β at B's value and bisect λ over the overrides to land back on the budget. The map costs an entropy-coded mask, N·H(ρ) bits with H the binary entropy, plus 3 per override.

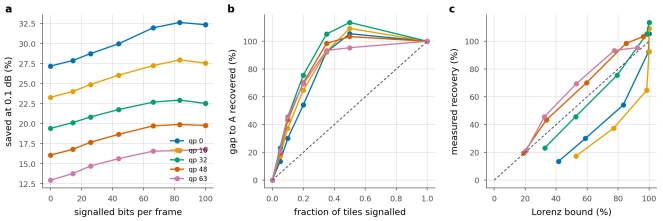

Figure 19. Partial signalling. a, saving against the bits spent on the map; each line runs from B on the left to A on the right. **b**, the same normalised by the A-to-B gap; dashed is proportional recovery. **c**, against the fixed-λ Lorenz bound; points above the dashed line are allocations one multiplier cannot reach.

q	B	5%	10%	20%	50%	A
bits/frame	0	14	26	44	83	100
0	24.4	25.1	26.0	27.3	30.1	29.9
16	20.9	21.7	22.5	23.7	25.7	25.6
32	17.4	18.1	18.8	19.8	21.1	20.9
48	14.1	14.9	15.8	16.8	18.3	18.3
63	11.1	11.9	12.9	13.9	15.3	15.6

Table 16. Configuration C at 0.1 dB. Columns are the fraction of tiles the encoder overrides; the two end columns reproduce B and A to within 0.1 points, which is the check that the interpolation is real.

The two ends check out. At ρ=0 the measurement reproduces configuration B to the second decimal at every rate, and at ρ=1 it reproduces A. Neither end is imposed; both fall out of the same code path run against independently measured files, so the columns between them are measuring something real.

Most of the gap is cheap. Overriding a tenth of the tiles for 26 bits per frame recovers 29–39% of the gap, and a fifth recovers 53–62%. Recovery is concave everywhere, so the first bits spent are the most useful ones.

And a partial map can beat a complete one. At 4 of 5 rates, signalling half the tiles for 83 bits saves *more* than signalling all of them, by up to 0.21 points, at the same distortion. Both land within the bisection's 5e-4 dB tolerance of the budget, and the local exchange rate is about 48 saving points per decibel, so the tolerance is worth 0.02 points against a margin ten to twenty times that. The margin is therefore real, and it does not come from a better predictor. The hybrid has *two* multipliers where A has one: the oracle's λ governs the overridden tiles and the router's fixed β the rest. A two-multiplier allocation reaches points on the distortion–compute plane that a single Lagrangian sweep cannot, for the same reason the sweep misses interior Pareto points at all. Configuration A is optimal among allocations reachable by one multiplier, and that is a smaller set than the word suggests.

Why the recovery is concave, and what bounds it. At a fixed λ the objective is separable, so overriding a set removes *exactly* the sum of that set's regrets. The recovery of the *objective* is then the Lorenz curve of the per-tile regret distribution, and its curvature is that distribution's Gini coefficient, 0.49–0.73 across rates. What the table reports is saving at fixed distortion, not the objective, and returning to the budget means re-bisecting λ. That re-bisection is also what lets the two-multiplier allocations above escape the bound.

At the loose budget it matters where the gap is. At 0.3 dB the three lowest rates are saturated, and there is nothing for a partial map to buy. At $q48$ and $q63$, where the gap is 1.7 points, half the map recovers 68–91% of it. The allocation follows the same rule here as everywhere else, which is to spend the bits where the ladder still has somewhere to go.

A better predictor concentrates its regret, which is what the Gini was for. We rebuild C on the retrained head of Section 5.5. The Gini of the per-tile regret rises at 4 of 5 rates, to 0.68–0.91. Its mistakes are rarer and larger. The same table shows the consequence. C converges on A faster from a better base and has less left to recover, and the half-beats-all effect disappears at q0, because a base close to A leaves little for the extra multiplier to exploit. The Lorenz reading is therefore worth having as a diagnostic and not as a decoration; it reports what *kind* of predictor one has as well as how good it is.

Which predictor should C be built on? Either will do, and the answer follows the same split as Section 5.6. Running the identical override rule over the calibrated bit rule instead of the head is worth up to 3.0 points at the lowest rate and costs up to 0.2 at the highest. Both converge on A at ρ=1 to the second decimal at every rate, which is the third independent check that the two code paths agree. The best single operating point we measured is the calibrated bit rule with half the map signalled, at 30.5% at q0. That is 0.61 points *above* full signalling, with no learned component anywhere in the decoder.

Configuration C is what we would ship where a small map is tolerable and a large one is not. It also reframes the A–B gap. What the gap measures is the price of *silence*, charged tile by tile, rather than the price of prediction.

5.8 Does the map have to be recomputed?

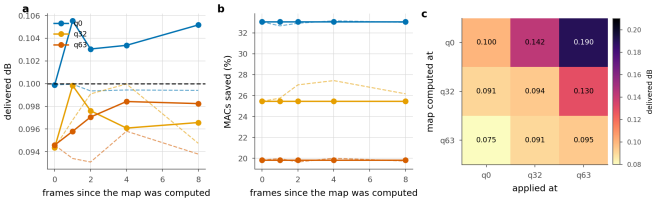

Figure 20. Reusing an exit map. Solid is the transferred map, dashed the one recomputed in place. **a**, **b**: reuse across frames of the same sequence. **c**: reuse across quality index; the entry is the delivered distortion against a 0.1 dB budget.

How much A's extra decode at the encoder matters depends on how often the map has to be recomputed, and we have not seen that measured anywhere.

Across frames. Reuse is close to free. We take the map found on frame 0 and apply it eight frames later; at q0 the delivered distortion rises by 0.005 dB, five percent of the budget, and the saving does not move at all: 33.05% transferred against 32.66–33.15% recomputed. The allocation is a property of where the content is hard, and that moves slowly. A codec would search once per group of pictures instead of once per frame, and divide the encoder cost by the group length.

Across rate. Here the map does have to be recomputed, and the direction of the reuse decides how badly. Found at q0 and applied at q63, it delivers 0.190 dB against a 0.1 dB budget, claiming the low-rate saving of 33.05% while spending nearly twice the quality it is allowed. The reverse is safe but wasteful: the q63 map applied at q0 delivers 0.075 dB and only 19.8% where 33.1% was available. Shallow exits are cheap in quality at low rate and expensive at high rate, so a map is calibrated to the rate it was found at, and reusing one upward breaks the quality guarantee without anything in the decode reporting that it has. In

our reuse probe, then, an encoder can search once per rate and reuse that search across the frames we tested. How far that carries beyond the offsets and sequences probed here we have not measured.

5.9 Complexity and wall-clock

q	Released	Routed	Saved (%)	
	(ms)	sorted (ms)	measured	MACs
0	111	78	**29.1**	35.3
32	111	86	**22.4**	28.0
63	111	94	**15.6**	20.1

Table 17. Wall-clock, 1080p, median of 40 interleaved iterations on one NVIDIA RTX A6000, 300 W board limit, PyTorch 2.6.0 and CUDA 12.4, single precision, at the 0.1 dB operating point. “MACs” is what the arithmetic predicts; “measured” is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time. Tiling costs 3.6% before anything exits early, measured as the deepest-exit tiled decode against the released full-frame one at the same arithmetic and the same weights. On top of that, the routed decode realises 27.9% at q0 where its operations predict 35.3%. Four fifths of the predicted saving arrives. The missing fifth is the tiling overhead together with the bookkeeping of a shrinking active set, and a MAC count sees neither. The shortfall scales with the saving instead of sitting at a fixed offset: at q63 we measure 15.6% realised against 20.1% predicted.

Seconds and joules. Multiply-accumulates are the right unit to optimise, because they do not depend on the machine, but they are not the unit a deployment cares about. We measured both on the padded 1080p frame, sampling board power from the driver at 100 ms and running each condition for a fixed wall time rather than a fixed iteration count, because the sensor updates too slowly for ten iterations of a 100 ms decode to mean anything. Idle power was measured before and after each condition so that a neighbouring process waking up would appear as drift rather than as a saving.

Energy follows time, not arithmetic. At q0, q32 and q63 the routed decode removes 33.0, 22.9 and 17.8% of the arithmetic; it removes 29.1, 19.1 and 14.1% of the wall clock and 29.6, 19.3 and 14.2% of the joules per frame. Board power barely moves, 298 W against 296 W at q0, because the later groups run on a shrinking set of tiles and a partly idle GPU still draws most of its static power. What early exit buys is a shorter decode, not a cooler one. Frame rate at 1080p goes from 9.0 to 11.2, and peak memory rises by 13.3% at every resolution we could measure: tiling turns one feature map into a batch of small ones, so the peak is set by the widest point of the ladder rather than by its deepest.

Sorting the tiles once by exit depth recovers part of it. The obvious implementation of a shrinking active set keeps a boolean mask, gathers the survivors and scatters the finished at every group boundary, and each of those boundaries forces a device-to-host synchronisation. Order the tiles by descending exit depth and “still active at group g” becomes a contiguous prefix: each group is then a slice instead of a gather, and one cumulative count supplies every boundary, so no per-group mask is built at all. The finished tiles come back through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. At q0 the saving goes from 27.9% to 29.1%, a gain of 1.2 points, or a fifth of the shortfall, for a change that touches no arithmetic.

The same gap appears in our own accounting. The router is charged at its share of the decoder's multiply-accumulates, 0.163% (Section 3.1). We also timed the head directly, on the padded 2048×1280 frame the decoder actually sees, interleaved against a deepest-exit decode so that a co-tenant's load falls on both equally. It costs 0.50%, which is 3.0× what its arithmetic predicts, and for the same reason as everything else in this section: a 144 K-parameter head is launch overhead, and a MAC count cannot see a launch. Charged at measured time instead of at operations, every B number in this paper would fall by a further 0.33 points. We leave them charged at MACs because that is the convention the rest of the literature reports in, and we record the correction here so that it does not sit unstated.

A paper that quotes only operations would report 35.3% where the same code, run as written, delivers 29.1%. That is why we give the shortfall as well. The error runs one way: operations are an *optimistic* bound on this method, and the optimism grows with how much of the frame exits early.

5.10 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.2 dB	0.3 dB	0.5 dB
RECIPE512	K6 j2 256px	41.9	22.1	**34.4**	38.2	39.1
BEST	K6 j2 256px	41.9	**24.2**	–	38.5	41.3
BEST128	K6 j2 128px	41.9	19.3	–	36.6	40.6
FINE12	K12 j4 128px	50.3	19.0*	–	**42.0**	**48.6**

Table 18. Ladder settings, mean saving (%) over the five rates, same test set and protocol. “Ceiling” is the architectural ceiling. * one rate is infeasible at that budget, because the ladder's floor exceeds it, so the mean is over the remaining four.

Section 5.4 argued that once a budget saturates a ladder, the only way to spend more is an exit that does not exist. The table measures that. A finer ladder (K=12, j=4) has a ceiling of 50.3% against 39.1%, and the two orderings cross somewhere between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate. Splitting later (j=4 against j=2) puts twice as many blocks in the per-tile section, which by the power law of Section 4 raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 39.1%, because the coarse one has been pinned at its ceiling since 0.3 dB and has nothing left to spend.

So the ladder is a choice made at the operating point, not a hyperparameter tuned once and fixed. The band of Section 5.4 tells you which side of the crossover a given budget sits on.

6. Limitations

The seam is reduced, and the exact remedy is not usable as it stands. The halo exchange removes 47–91% of the floor and is bit-exact at uniform depth (Section 4.4). It also destroys the routed saving, because routing puts neighbouring tiles at different depths. We do not know whether a decoder trained with the exchange can recover both at once, and that is the experiment we would run next. Until someone runs it, the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question we do not answer here, because an exit map propagates through the reference chain and a shallow tile in one frame is a worse reference for the next one.

Training is not converged. At every checkpoint we have measured more than once, the saving was still going up. We therefore read the numbers reported here as a lower bound on what a converged run would give.

A single decoder. All results are on DCVC-UF's intra decoder. Nothing in the method looks specific to it, since the ladder needs only a residual trunk with a shared head, but we have not measured a second decoder to check.

7. Conclusion

The intra decoder of DCVC-UF can be run at a depth chosen per tile from what that tile contains. At a 0.1 dB budget this removes 29.9% to 15.6% of its multiply-accumulates for a BD-Rate cost of 0.88%, with no change to the encoder and none to the coded payload.

Three lessons we would carry to any spatially adaptive decoder, and none of them is about early exit.

Tiling is the dominant cost and it is governed by depth. All of that cost traces back to a single 3×3 that is 0.29% of the arithmetic, and the

penalty grows as the square of how many such convolutions run per tile. The affected-area fraction usually quoted alongside it predicts that penalty badly.

The exact remedy is in tension with the thing it enables. Giving each convolution its real neighbour is bit-identical at uniform depth, and under routing it destroys the allocation, because routing is the deliberate violation of the condition that makes it exact. We expect any method that combines spatial adaptivity with tiled inference to walk into this.

A distortion budget is only a control variable inside a measurable band. Below the floor the budget admits nothing at all, and above saturation more of it buys nothing. A saving quoted without saying where in that band it sits has left out the part of the result a reader most needs.

We would attach three cautions to the measurements themselves. A saving in operations is an optimistic bound on a saving in time, and the optimism scales with the saving. A learned router should be compared against a free one. Routing on the bits already spent per tile needs no parameters and no training, it adds nothing to the stream, and it beats our trained head at every rate we measured, while agreeing with the Lagrangian oracle on fewer tiles than the head does. We read that as a warning about the metric quite as much as about the head. Finally, a timing harness will report numbers whether or not it is timing the right device. Ours timed the wrong one for months, and what gave it away was a decoder that appeared to slow down with the quality index.

References

[1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.

[2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.

[3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.

[5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.

[6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.

[7] W. Fedus et al. Switch Transformers. JMLR, 2022.

[8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.

[9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.

[10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.

[11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.

[12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.

[13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.

[14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.

[15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.

[16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.

[17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.

[18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.

[19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.

[20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.

[21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.

[22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.

[23] M. A. Yilmaz et al. Slimmable video codec. CVPRW, 2022.

[24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.

[25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.

[26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.

[27] T. Blard et al. Spatial competition for low-complexity learned image compression. arXiv:2605.13243, 2026.

[28] Y. Shoham, A. Gersho. Efficient bit allocation for an arbitrary set of quantizers. IEEE TASSP, 1988.

[29] A. Ortega, K. Ramchandran. Rate-distortion methods for image and video compression. IEEE SPM, 1998.

[30] Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization. arXiv:2604.14853, 2026.

[31] H. A. Alhaija et al. Local padding in patch-based GANs for seamless infinite-sized texture synthesis. arXiv:2309.02340, 2023.

[32] C. Innamorati et al. Overlap training to mitigate inconsistencies caused by image tiling in CNNs. arXiv:1812.02203, 2018.

[33] Z. Jia et al. Towards practical real-time neural video compression. CVPR, 2025.

[34] B. A. Hasan et al. Adaptive inference: theoretical limits and unexplored opportunities. arXiv:2402.04359, 2024.

[35] Y. Gao et al. Exploring the rate-distortion-complexity optimization in neural image compression. CVIU, 2024.

[36] Y.-H. Ho et al. On the rate-distortion-complexity trade-offs of neural video coding. arXiv:2410.03898, 2024.

[37] C. Zhang, W. Gao. Learned rate control for frame-level adaptive neural video compression via dynamic neural network. arXiv:2508.20709, 2025.

[38] Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. NeurIPS, 2017.

[39] D. Madras et al. Predict responsibly: improving fairness and accuracy by learning to defer. NeurIPS, 2018.

[40] H. Mozannar, D. Sontag. Consistent estimators for learning to defer to an expert. ICML, 2020.

[41] G. DeSalvo et al. Budgeted multiple-expert deferral. arXiv:2510.26706, 2025.

[42] M. Dong, M. Lu, and Z. Ma. Accelerating block-level rate control for learned image compression. arXiv:2409.01009, 2024.

[43] Y. Li et al. Learned image compression with hierarchical progressive context modeling. ICCV, 2025.

[44] D. He, Z. Yang, W. Peng, R. Ma, H. Qin, Y. Wang. ELIC: efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. CVPR, 2022.

[45] R. Zou, C. Song, Z. Zhang. The devil is in the details: window-based attention for image compression. CVPR, 2022.

[46] J. Liu, H. Sun, J. Katto. Learned image compression with mixed transformer-CNN architectures. CVPR, 2023.

[47] W. Jiang, R. Wang. MLIC++: linear complexity multi-reference entropy modeling for learned image compression. ICML Neural Compression Workshop, 2023.

[48] H. Li, S. Li, W. Dai, C. Li, J. Zou, H. Xiong. Frequency-aware transformer for learned image compression. ICLR, 2024.

[49] S. Qin et al. MambaVC: learned visual compression with selective state spaces. arXiv:2405.15413, 2024.

[50] H. Fu, J. Liang, Z. Fang, J. Han, F. Liang, G. Zhang. WeConvene: learned image compression with wavelet-domain convolution and entropy model. ECCV, 2024.

[51] D. Minnen, S. Singh. Channel-wise autoregressive entropy models for learned image compression. ICIP, 2020.

[52] J. Li, B. Li, Y. Lu. Neural video compression with diverse contexts. CVPR, 2023.

Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Supplementary Material

A. Implementation, training and reproducibility

This section is the recipe. It gives the released decoder the work starts from, how the training set was built, the optimiser and the schedule, what an epoch costs in hours, which checkpoint each number in the paper was measured on, the evaluation protocol down to the colour space, and what varies between two runs of the same command. One fact governs the rest and is stated here rather than buried: the checkpoint every headline number comes from has seen the training set exactly once. Training was still running when the paper was written, and A.4 reports how much the numbers move with a second pass.

A.1. The decoder we start from

The starting point is the released DCVC-UF intra codec [14], used unchanged as a codec and rearranged as a network. Its decoder is an opening upsampling block followed by 12 DepthConvBlocks at C = 384 channels and a head that pixel-shuffles back to RGB. We group the 12 blocks into K = 6 exits of b = 2 blocks each, and the first j = 2 groups run once over the whole frame before the decode splits into tiles. Exits 0 and 1 therefore lie below the split and cannot be selected; the four reachable exits are 2 to 5. The cost of each is derived in the complexity section and not repeated here.

scripts/warmstart_from_release.py performs the rearrangement. The encoder, hyperprior, entropy model and quantisation-step tables are copied verbatim; the decoder's weights are remapped into the ladder; every adapter and the seam-repair gate are zero-initialised, so each one is the identity at step zero. Two controls follow from that and both are asserted rather than assumed: the deepest exit reproduces the released decoder with $\max|\Delta| = 0.0$, and every untrained exit equals the released decoder truncated at that depth, also $\max|\Delta| = 0.0$. The transfer moves 398 tensors and creates 28 adapters at initialisation.

The encoder is frozen for the whole of training. That is what makes the comparison in this paper a decoder comparison: a trained decoder consumes byte for byte the stream the released encoder produces, and in every evaluation both decoders read the same latent from the same encode, so the bit rate is identical by construction and the whole of the measured difference is decoder-side.

quantity	value	source
trunk blocks N	12	adapter_cost
exits K	6	tile_definition
blocks per exit b	2	adapter_cost
split depth j	2	tile_definition
reachable exits	2 to 5	adapter_cost
trunk width C	384	adapter_cost
RGB tile	256 px	tile_definition
feature tile	32 px	tile_definition
latent tile	16 px	tile_definition
pixels per latent	16	tile_definition
tile padding	replicate	tile_definition
seam repair	grid	tile_definition
adapter kind	scaled	tile_definition
parameters, total	45,445,398	meta.json
parameters, adapters	3,104,640	meta.json

Table 1. The ladder and the tiling, as the checkpoint itself records them. Everything above the rule is read out of the two results files named in the last column, both on the pinned checkpoint. The two parameter counts are the exception and are read from the run directory, because nothing writes a parameter count into results/.

Source: results/adapter_cost.json and results/tile_definition.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar; parameter counts from runs/RECIPE512/meta.json.

A.2. The training data

Training uses Open Images [24], prepared by scripts/prepare_openimages.py into the flat layout DCVC-UF's own ImageFolder expects. Of 384,795 files scanned, 380,126 survive a minimum-side filter of 512 px and 4,669 are dropped as too small. The filter is not cosmetic: the loader zero-pads any image smaller than the current crop, and an image that is half black border teaches the decoder to reconstruct black borders. From the survivors, every 400th entry of the sorted list is held out, giving 512 validation images and 379,614 training images with no overlap. The split is deterministic so that every run and every control sees the same held-out set.

Each sample is a random crop at the schedule's patch size with a random horizontal flip, converted to YCbCr, scaled to [0,1] and shifted by −0.5. One quality index is drawn uniformly over the 64 levels per sample, and the matching λ comes from the log-spaced table running 10 to 2048, so a single set of weights covers the whole rate range rather than one point on it. Training on random crops and testing on whole frames is the usual arrangement in this literature, and the crop size in [2] is 256; ours is the size the released recipe's final phase asks for. It also happens to be the smallest crop that carries more than one tile: at 256 px tiles a 512 px crop holds 4, whereas a 256 px crop holds one, and a single tile has no border against a neighbour, so patched training at that crop size would train a configuration that cannot occur at inference.

A.3. The training run

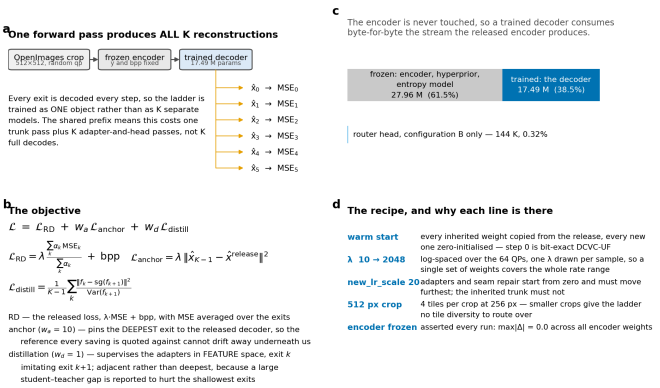

Figure 1. The training scheme. Panel (a): one forward pass produces all K reconstructions, so the ladder is trained as one object. Panel (b): the three loss terms. Panel (c): the encoder is never touched. Panel (d): why each line of the recipe is there. The figure is a schematic drawn from the model definition; the numbers it carries are parameter counts, and no file in results/ records those, which is why they are not quoted in the text.

The recipe is Microsoft's own, applied at the point in their schedule that matches what we inherit. Their schedule is a 105-entry table of learning rate and crop size indexed by epoch, and the released weights are the output of the last entry. Applying entry zero to them would kick a converged codec with the from-scratch learning rate, so the run enters at offset 99, which is inside their final 512 px phase at a learning rate of 1×10^{-5}. Every one of the 662 records in the run's log carries that learning rate and that crop, which is the check that the offset did what it was meant to.

One deviation from a plain fine-tune is deliberate. The adapters and the seam-repair gate start at zero and have to move furthest, while the inherited trunk must not move at all if the deepest exit is to stay the released decoder. Running both at one learning rate means choosing which of the two to sacrifice, so the new modules are given a separate parameter group at 20 times the schedule's rate.

setting	value
optimiser	AdamW
schedule	DCVC-UF's 105-entry table, entered at offset 99
learning rate	1×10^{-5}, constant for this run
lr multiplier, new modules	×20
crop	512 × 512
batch size	8
data workers	8
gradient clip	0.1, NaN steps skipped
precision	fp32 throughout
devices	1 × NVIDIA RTX A6000
λ range	10 to 2048, log-spaced over 64 quality indices
quality index	uniform over 64, drawn per sample
anchor weight	10
distillation weight	1.0, teacher = adjacent exit
auxiliary weight	1.0, constant
training path	through the deployed tiled decode
encoder	frozen
epochs requested	16
steps per epoch	47,451
seed	none set

Table 2. The training recipe in full. Take from it that nothing here is tuned: the optimiser, the schedule, the clip and the λ range are the released recipe's, and the four settings that are ours (the learning-rate multiplier for zero-initialised modules, the anchor, the distillation and the tiled training path) each exist to protect a property the measurement depends on.

Source: scripts/launch_recipe512.sh for the flags, train_flexuf_image.py for the optimiser, the schedule table and the clip, and runs/RECIPE512/train_log.jsonl for the learning rate and crop actually used. No file in results/ records any of it.

The loss has three terms. The rate-distortion term is the released loss, λ·MSE + bpp, with the MSE averaged over the exits under fixed weights. The anchor term pins the deepest exit to the released decoder, because every saving in the paper is quoted against that decoder and a reference that drifts underneath the measurement makes the measurement meaningless. The distillation term supervises each adapter in feature space against the exit one step deeper, following the adjacent-teacher form rather than the deepest-teacher form [17], since a large gap between student and teacher is reported to hurt the shallowest exits most.

An epoch is 47,451 optimiser steps, which is 379,614 images at batch 8. The run logs every 200 steps and the median interval is 299.7 s, so a step costs about 1.5 s and an epoch about 19.8 hours on one card. Across all 662 records the count of steps skipped for a non-finite gradient is 0.

The router head of configuration B is a separate and much smaller job, trained on top of a frozen decoder. Each variant of the head is 3,000 steps at batch 6 on 512 px crops, learning rate 1×10^{-3}, seed 0, 148,176 parameters, and takes about an hour on the same card. The six variants of the input ablation cost 6.5 hours between them. What the head learns and what each input is worth are the subject of the router section; only the cost of training it belongs here.

Source: results/router_ablation_stem.json and its five siblings, all on runs/RECIPE512/ckpt_PAPER.pth.tar.

A.4. How far training has gone

quality index	epoch 0	epoch 1	change	floor e0	floor e1
q0	29.90	30.84	+0.93	0.0326	0.0295
q16	25.62	27.06	+1.44	0.0429	0.0383
q32	20.93	21.97	+1.04	0.0528	0.0495
q48	18.33	19.92	+1.59	0.0599	0.0551
q63	15.64	17.20	+1.56	0.0659	0.0595
mean	22.08	23.40	+1.31		

Table 3. One more pass over the training set is worth about a point and a half. Saving at the 0.1 dB budget against the released decoder, in percent, on the pinned checkpoint and on the epoch-1 pin, measured with the identical protocol on the identical frames. The last two columns are the floor, the quality already given up before any tile exits early, in dB; it falls at every rate. The paper reports the left-hand column.

Source: results/signalled_RECIPE512_ctc53.json (runs/RECIPE512/ckpt_PAPER.pth.tar, epoch 0) and results/signalled_RECIPE512_e1.json (runs/RECIPE512/ckpt_PIN_e1.pth.tar, epoch 1), both 53 sequences at 1 frames per sequence.

The run had reached epoch 2, step 37,000 of 47,451, when this was written, against the 16 epochs its launcher was given. So the paper's numbers come from a decoder one pass into a schedule that is nowhere near finished, and the epoch-1 pin shows the direction of travel: every rate gains, the floor falls at every rate, and the mean at the 0.1 dB budget moves from 22.1% to 23.4% against an architectural ceiling of 39.1%. Two things follow. The reported saving is a lower bound on what this configuration reaches, and no comparison in this paper between two runs at different epochs can be read as a comparison between two configurations. Where such a comparison appears it is labelled with both epochs.

Nothing in results/ projects the trajectory to convergence, and this section does not either. scripts/convergence.py fits a saturating curve with the ceiling pinned, but it writes only a figure, so its asymptote is not a number this document can cite.

A.5. The pinned checkpoint, and what every table is measured on

Results are measured on runs/RECIPE512/ckpt_PAPER.pth.tar. It records epoch 0 inside itself and is byte identical, by md5, to that run's ckpt_epo0.pth.tar. The name exists because a filename is not a

checkpoint here: a per-epoch watcher overwrites runs/*/ckpt_eval.pth.tar as each epoch lands, so a results file naming that path names a file that no longer exists. scripts/make_paper_tables.py and scripts/check_paper.py both hold the pinned name and, among candidate files for the same quantity, prefer one whose own ckpt field records it, falling back to the most recently written. Hand-written order breaks ties and nothing else, which is a fix rather than a convenience: an earlier hand-ordered list kept preferring a superseded file after the pinned one had been remeasured.

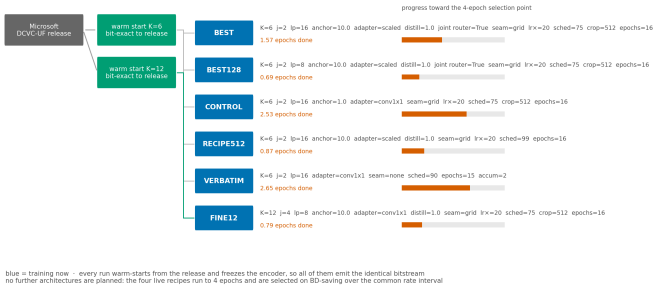

Figure 2. Lineage of the training runs. Every run warm-starts from the released decoder through one of two remaps, one per ladder size, and freezes the encoder, so all of them consume the identical bitstream. The flags shown are the ones that differ between runs. The progress bars are a snapshot taken on 2026-08-18 and the runs have advanced since; the current position of the run this paper reports is given in A.4.

results file	backs	checkpoint	n
signalled_RECIPE512_ctc53	main results, complexity, positioning	pinned, e0	53 seq × 1
saturation_RECIPE512_ctc53	operating range	pinned, e0	53 frames
static_RECIPE512_b01	static controls	pinned, e0	53 seq
per_class_RECIPE512	per class	pinned, e0	53 frames
router_RECIPE512_b01	A against B	pinned, e0	53 seq × 1
router_RECIPE512_b03	A against B, rate rank	pinned, e0	53 seq × 1
raterank_RECIPE512_b01	rate rank	pinned	53 seq × 1
raterank_RECIPE512_b03	rate rank	pinned	53 seq × 1
hybrid_RECIPE512_b01_fixed	partial signalling	pinned	53 seq × 1
hybrid_raterank_b01	rate rank, second run	pinned	53 seq × 1
latency_RECIPE512_sorted	latency, complexity	RECIPE512/eval	-
router_latency	operating range	RECIPE512/eval	-
adapter_ablation	adapter ablation	RECIPE512/eval	16 frames
combined_RECIPE512_b01	blend	RECIPE512/eval	53 seq × 1
raterank_RECIPE512_b05	rate rank	RECIPE512/eval	53 seq × 1
hybrid_v3_b01	rate rank, second run	RECIPE512/eval	53 seq × 1
hybrid_RECIPE512_b03_fixed	macros only	RECIPE512/eval	53 seq × 1
hybrid_lorenz_b01	macros only	RECIPE512/eval	53 seq × 1
coupling_ablation	macros only	RECIPE512/eval	16 frames
map_transfer	macros only	RECIPE512/eval	-
signalled_BEST_ctc53	ladder configurations	BEST/eval, e1	53 seq × 1
signalled_BEST128_ctc53	ladder configurations	BEST128/step, e1	53 seq × 1
signalled_FINE12_ctc53	ladder configurations	FINE12/step, e1	53 seq × 1
ctc_seam_p256	padding, tile size	*not recorded*	-
ctc_seam_p128	tile size	*not recorded*	-
bdrate	BD-rate	*not recorded*	-
band_collapse	macros only	*not recorded*	-
band_collapse_BEST	macros only	*not recorded*	-
raterank_BEST_compare	rate rank, second run	*not recorded*	-
router_retrain_compare	macros only	*not recorded*	-
check_paper	claim check	*not recorded*	-

Table 4. Provenance of every generated table in the main paper. These are the 31 results files scripts/make_paper_tables.py reads; the checkpoint column is read from each file at build time rather than typed. Take from it that 10 of the 31 are on the pinned checkpoint and the rest are not, so a reader comparing two rows of two different tables should check this column first. "eval" is the overwritten per-epoch file, and "not recorded" means the file carries no checkpoint field at all, which is a defect in the script that wrote it.

Three groups deserve separate comment. The pinned group carries the headline saving, the operating range, the per-class breakdown, the static controls and both routed configurations, so the paper's central claim rests on one epoch of one run measured consistently. The eval group is measured on a file that has since been overwritten, which means those numbers can be quoted but not reproduced; the latency table and the adapter ablation are in it. The group with no checkpoint field at all cannot even say which decoder it describes, and the seam and BD-rate tables are in that group.

A.6. The evaluation protocol

class	sequences	resolution	padded	tiles	added px
UVG	7	1920x1080	2048×1280	40	26.4%
MCL-JCV	30	1920x1080	2048×1280	40	26.4%
HEVC B	5	1920x1080	2048×1280	40	26.4%
HEVC E	3	1280x720	1280×768	15	6.7%
HEVC C	4	832x480	1024×512	8	31.3%
HEVC D	4	416x240	512×256	2	31.3%
all	53				

Table 5. The test set, and what tiling does to each resolution. Take from it that granularity is not constant across the set: a 1080p frame carries 40 tiles for the allocation to work with and a 416×240 frame carries two, and the small classes also pay the most padding. Padding is replicate, applied bottom and right only, to a whole number of tiles.

Sources: results/per_class_RECIPE512.json for the class membership and tile counts, results/tile_definition.json for the padded sizes; both on runs/RECIPE512/ckpt_PAPER.pth.tar. The sequences are UVG [25], MCL-JCV [26] and HEVC classes B, C, D and E [9]; the full list of 53 names is recorded in the measured field of results/signalled_RECIPE512_ctc53.json, with not_measured empty.

- **Frames.** The first frames of each sequence, consecutively, one or two per sequence depending on the measurement; every results file records frames_per_seq. The headline file uses two frames on 53 sequences, so 106 frames; the routed, rate-rank and partial signalling tables use one.
- **Colour.** Sources are YUV 4:2:0, 8 bit. They are converted to 4:4:4, divided by 255 and shifted by −0.5 for the network. PSNR is the released codec's own metric, (6·Y + U + V)/8, computed after converting the reconstruction back to 4:2:0 on the 0 to 255 scale and comparing against the source planes. Measuring in 4:4:4 instead would compare against a chroma upsample of the source rather than the source, and would report a higher chroma PSNR that no published DCVC-UF number could be compared with.

- **Padding.** Frames are padded by replication, bottom and right only, to a whole number of tiles. The stock codec aligns to 16, which is enough for the transform and not for a tile grid. The bit rate is charged on the true pixel count and the PSNR is computed after cropping back to the true frame, so padding can cost and cannot flatter.
- **Tiling.** One tile is 256 px of RGB, 32 px of feature map and 16 px of latent. The patchify step asserts divisibility rather than padding, so an unpadded 1080p frame fails loudly instead of producing a ragged last row.
- **Reference.** The released decoder re-expressed in the same ladder, selected by K so that a K = 12 run cannot be quoted against the K = 6 remap. Both decoders read the same latent from the same encode.
- **Finding the operating point.** For a target quality budget the multiplier λ is found by bisection, 60 halvings on [0,1], taking the largest λ whose predicted dB stays under the target. dB is averaged per frame and then over frames, which is the convention the released codec's own evaluation uses.
- **The map is billed.** The exit map is entropy coded and its cost is added to the bit rate before any saving is computed; every row records map_bits and the bpp it added.

Two limits of that protocol should be read alongside it. Sweeping the multiplier reaches only the lower convex hull of the achievable set [28, 29], so a budget falling in a gap of the hull is met at the nearest reachable point rather than exactly; every row therefore carries a budget_reachable flag and a floor, and rows whose floor already exceeds the budget are reported as unreachable rather than quietly dropped. And the 0.1 dB budget is a reporting convention rather than a perceptual threshold. Subjective work measures the smallest noticeable change in quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so nothing published licenses a claim that 0.1 dB is invisible; 0.3 dB and 0.5 dB are reported beside it so that the choice carries no weight.

A.7. Seeds, and where run-to-run variation comes from

No seed is set anywhere in the decoder's trainer. Four things vary between two runs of the same command: the sampler is reshuffled every epoch from the global generator, the quality index is drawn per sample, the tiled training path draws a fresh random exit for every tile at every step, and cuDNN selects kernels by autotuning. The honest statement is that the recipe reproduces and the run does not, and this work has no repeated run of one configuration from which a spread could be quoted.

The router experiments are the exception and were built to be. Every variant of the input ablation fixes seed 0 and shares architecture, parameter count, optimiser, image order and quality-index draws, so the only thing that differs between them is which inputs the head is allowed to see. That is what makes six numbers from six separate trainings comparable at all, and their agreement figures carry a standard error taken across 1,200 frames.

quality index	path 1	path 2	gap
q0	35.50	33.10	2.40
q16	30.32	27.58	2.74
q32	22.15	19.82	2.33
q48	16.88	15.14	1.74
q63	7.65	8.67	-1.01

Table 6. Two implementations of the same quantity do not agree. Saving in percent at the 0.1 dB budget, computed by the curve-sweeping path and by the signalling path, which differ in how they pool distortion across tiles and frames. Take from it a floor on protocol sensitivity: the gap reaches 2.74 points, which is larger than several of the differences the paper draws conclusions from, so numbers from the two paths are never mixed inside one table.

Source: results/crosscheck_paths.json. The file records no checkpoint field.

Evaluation itself is deterministic once a checkpoint is fixed: the sequence list, the leading frames and the tile grid are all fixed, and no crop is random. scripts/check_paper.py re-reads 85 numerical claims out of the paper's prose and checks each against the file it is supposed to come from. As recorded in results/check_paper.json, 84 of them pass and one does not: "hybrid: rho=1 reproduces A", which is an internal consistency identity the interpolation between the two configurations is expected to satisfy at its endpoint. It is open at the time of writing and is listed among the limitations rather than presented as passing.

A.8. What no file in results/ pins

The rule this supplementary follows is that a number is quoted only with the file it came from. Applying it honestly means listing the places where the file does not exist.

- **The decoder's training recipe.** Nothing that writes to results/ runs inside the trainer, so no results file records the learning rate, the schedule, the batch size, the crop, the loss weights or the number of steps. Table 2 is read from scripts/launch_recipe512.sh, train_flexuf_image.py and the run's own log.
- **Parameter counts and wall clock.** Read from runs/RECIPE512/meta.json and runs/RECIPE512/train_log.jsonl. The log is appended to by a live process, so the position quoted in A.4 is a snapshot taken at 2026-08-19 21:03.
- **The warm start's own controls.** runs/warmstart/warmstart_report.json holds the transfer counts and both zero-difference controls, but the copy on disk is the K = 12 rebuild of 17 August; the K = 6 report the pinned run descends from was overwritten. The controls are asserted inside the script at every build, so a failure would stop the run, but the K = 6 record is gone.
- **The intra decoder's total multiply-accumulate count.** It is a constant in scripts/make_paper_tables.py with a comment naming scripts/mac_audit.py, and that script prints to standard output and writes no JSON. Every row of the complexity table is normalised by it.
- **Any resolution above 1080p.** The queued 4K latency job returned without writing a file, and results/supp_footprint.json marks its 3840×2304 row oom, so nothing in this work is measured at 4K.
- **The qualitative figures' checkpoint.** Each of those scripts defaults to a per-epoch file that the watcher has since overwritten and none writes a provenance sidecar, so no qualitative figure in this work can have its checkpoint stated.
- **A repeated run.** With no seed and no configuration trained twice, the paper reports no run-to-run interval. Table 6 is the nearest available substitute and it measures something else: sensitivity to the measurement path, not to the training draw.

B. Architecture and complexity accounting

This section derives what the decoder costs, operator by operator, and then checks the derivation against a count taken off decodes that were actually run. It closes with what the saving is worth in the two units a deployment cares about, milliseconds and joules, which are not the unit the allocation is optimised in.

Some vocabulary first. A *multiply-accumulate* (MAC) is one multiply and one add; MAC/px is per pixel of the feature grid an operator runs on. The *trunk* is the released decoder's stack of N = 12 identical DepthConvBlocks at width C = 384. An *exit* is a point in that stack where a tile may stop; the ladder has K = 6 exits, one every b = N/K = 2 blocks. The *split depth* j = 2 is the number of exits' worth of blocks that run once for the whole frame before any tile is allowed to leave, so groups 0 and 1 are full-frame and groups 2 to 5 are per-tile. An *adapter* is a small zero-initialised residual module that stands in for the blocks an exit skips. A *tile* is 256 RGB px square. The *exit map* is the per-tile assignment. The *released decoder* is DCVC-UF's intra decoder [14] with none of this added, and it is the unit all costs below are quoted in.

B.1. One DepthConvBlock at C = 384

The block is five convolutions: a 1×1 projection, a 3×3 depthwise, a second 1×1, then a feed-forward pair whose first 1×1 expands to 4C and whose gated activation folds 4C back to C for free, so the closing 1×1 is C→C rather than 4C→C. Table 7 is the count, taken by forward hooks on the real modules at the real tensor shapes.

Operator	Kernel	Out ch.	MAC/px	in C²	Share
1×1 in	1×1	384	147,456	1.0000	0.1424
3×3 depthwise	3×3	384	3,456	0.0234	0.0033
1×1 out	1×1	384	147,456	1.0000	0.1424
FFN 1×1 C→4C	1×1	1536	589,824	4.0000	0.5695
FFN 1×1 C→C	1×1	384	147,456	1.0000	0.1424
Block			**1,035,648**	**7.0234**	**1.0000**

Table 7. One DepthConvBlock at C = 384, by operator. The block is 7C² + 9C = 1,035,648 MAC/px. Take from it that the 3×3 depthwise, the only operator in the whole trunk with a spatial footprint, is 0.33% of the block: everything else is 1×1 and has no neighbours to miss, which is why a tile boundary is cheap to repair and why the trunk can be cut into tiles at all.

Source: results/adapter_cost.json, forward hooks on the modules of runs/RECIPE512/ckpt_PAPER.pth.tar. Shares are of the block.

$$M_{\text{block}} = C^2 + 9C + C^2 + 4C^2 + C^2 = 7C^2 + 9C \tag{1}$$

The cost model shipped in flexuf/cost.py writes the block in closed form as 8C² + 9C = 1,183,104 MAC/px, one C² too many, which overstates it by 14.24%. Every adapter and the seam-repair pass are priced as fractions of a block, so the error propagates to all of them; Section B.5 measures exactly how far.

B.2. The two adapters

An exit that leaves early hands a feature map to the head that the remaining blocks would have refined. The adapter is what stands in for them. Two kinds are used. The 1×1 adapter is Ad(f) = f + Wf with W a C×C matrix, so it costs C² MAC/px. The FFN adapter repeats the block's feed-forward pair: a 1×1 expanding C→4C, the same gated activation, and a 1×1 C→C. Both are zero-initialised, so the ladder is the released decoder exactly at step zero and any quality the adapters add is attributable to them.

Module	Operators	MAC/px	in C²	Blocks	Of decode
1×1 adapter	one 1×1 C→C	147,456	1.0000	0.1424	1.061%
FFN adapter, 1×1 C→4C		589,824	4.0000	0.5695	
FFN adapter, 1×1 C→C		147,456	1.0000	0.1424	
FFN adapter, total		**737,280**	**5.0000**	**0.7119**	**5.306%**

Table 8. The two adapter kinds. Take from it that the FFN adapter costs 5C², not 2C²: the gated activation is free, so the pair is 4C² + C² and not 4C² + 4C². It is 0.71 of a whole block, which makes it much the largest single correction in this section. The 1×1 adapter is 0.14 of a block, nearer one seventh than the one eighth the closed form implies.

Source: results/adapter_cost.json, same hooks and same checkpoint as Table 7. The "of decode" column is the adapter priced against a released decode, using the per-block share derived in Section B.3.

The ladder assigns the FFN adapter to any exit skipping four or more blocks and the 1×1 adapter otherwise, which at K = 6 and b = 2 means the FFN at exits 0 to 3, the 1×1 at exit 4, and no adapter at exit 5, whose feature is the released decoder's own. Since the routed allocation lives at exits 2 and 3, the correction from 2C² to 5C² lands on precisely the exits the reported saving leans on.

B.3. From the block to the decoder

Write s_{up}, s_{trunk} and s_{head} for the shares of a released decode taken by the opening upsample, the 12-block trunk and the head; a_k for the adapter at exit k; and r for the seam-repair pass, which runs once over the stitched canvas. A tile assigned an exit shallower than j still runs group j, because the decoder clamps the map, so the cost of exit k in units of one released decode is

$$c_k = s_{\text{up}} + s_{\text{head}} + r + s_{\text{trunk}}\frac{(\max(k,j)+1)b}{N} + a_{\max(k,j)} \tag{2}$$

and the frame-level cost of an exit map a over T tiles amortises the shared part over the frame rather than over the tile,

$$C(a) = s_{\text{up}} + s_{\text{trunk}}\frac{jb}{N} + s_{\text{head}} + r + \frac{1}{T}\sum_{i=1}^{T} u_{a(i)} \tag{3}$$

with u_k the per-tile suffix, that is the blocks after the split plus the adapter. Only the last term depends on the map. That is the structural reason a ladder over a decoder of this shape has a ceiling well below 100%: the stem, the head and the repair pass run whatever the map says.

The shares themselves need not be taken on trust. results/static_RECIPE512_b01.json records, on the pinned checkpoint, the frame-level saving of a map that sends every tile to exit e, for e = 2, 3, 4 and 5. Four numbers give three differences and a constant, and those four quantities are the whole model. One trunk block is 0.074533 of a released decode, so the twelve of them are 0.8944; the always-on remainder, upsample plus head plus repair, is 0.413241; and the two adapters come out at 0.009289 and 0.046447, which are the prices the shipped model pays for them. Repricing those two and the repair pass on the measured block of Table 7 raises them to 0.010612, 0.053060 and 0.010861, which is the derived column throughout this section. Table 9 lays the stages out against the wall clock.

Stage	MAC %	1080p tiles	1080p ms	720p tiles	720p ms
Stem: upsample + groups 0, 1	37.97	frame	41.39	frame	16.30
Patchify	0.00		0.21		0.09
Group 2 (2 blocks, per tile)	14.91	40	15.98	15	6.19
Group 3 (2 blocks, per tile)	14.91	28	11.24	10	4.16
Group 4 (2 blocks, per tile)	14.91	14	5.72	5	2.23
Group 5 (2 blocks, per tile)	14.91	9	3.79	3	1.30
Unpatchify	0.00		0.21		0.09
Seam repair (full-frame)	1.09	frame	2.07	frame	0.83
Head	2.40	frame	4.36	frame	1.72

Table 9. Where a decode goes, in arithmetic and in milliseconds. The MAC column is each stage's share of a released decode at full occupancy, derived as above; it sums to 1.0109, which is the deepest exit's cost and is why the ladder is about 1% dearer than the release when nothing is skipped. Take from the table that patchify and unpatchify are free arithmetically and nearly free on the clock, that the repair pass is 1.09% of the arithmetic, and that the head takes 5.1% of the routed clock for 2.40% of the arithmetic, being bandwidth-bound rather than arithmetic-bound.

Milliseconds: results/supp_latency_1920x1080.json and results/supp_latency_1280x720.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000 taken under a lock so the card was not shared. The tile counts are that decode's own exit map, which leaves 28, 14 and 9 of 40 tiles alive in groups 3, 4 and 5. The head's 2.40% is the one share in the table that no file in results/ pins; see Section B.9.

Patchify and unpatchify are reshapes and cost no arithmetic at all. They require the feature map to divide by the 32 px feature tile, which the encoder-side padding guarantees: the RGB frame is replicate-padded at the bottom and right before encoding, so 1920×1080 becomes 2048×1280, a 128×80 latent, a 256×160 feature map and 40 tiles of 32×32; 1280×720 becomes 1280×768 and 15 tiles (results/tile_definition.json). One feature pixel covers 8×8 RGB pixels, so a 256 px tile is 32 feature px and 16 latent px.

The router head, which reads the shared stem map and predicts a per-tile exit, is 144 K parameters and 0.163% of decode MACs. On the clock it is 0.50% of decode time, 3.0× its arithmetic share, an extra 0.33 points of decode time bought by a launch of a small kernel over a large map. It is small enough on either measure to leave the accounting unchanged, and it does not run at all in the signalled configuration, where the encoder chooses the map.

results/router_latency.json: 40 iterations at 2048×1280 on an NVIDIA RTX A6000. Measured on runs/RECIPE512/ckpt_eval.pth.tar, not the pinned checkpoint; the router

head is the same size in both.

B.4. The per-exit cost vector and the ceiling

Exit k	Blocks run	Adapter	c (2C²)	c (5C²)	c derived	Saved %
0 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
1 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
2	6	FFN	0.5809	0.6088	0.6167	38.33
3	8	FFN	0.7300	0.7578	0.7658	23.42
4	10	1×1	0.8697	0.8697	0.8724	12.76
5	12	none	1.0095	1.0095	1.0109	-1.09

Table 10. The per-exit cost vector, in units of one released decode, under three prices for the same architecture: the first shipped table, which put the FFN adapter at 2C²; the table the paper reports, which corrected that to 5C² but kept the closed-form block; and the derivation of Tables 7 and 8, which prices both on the measured block. Take from it that the two corrections move the shallowest reachable exit by 2.79 and then 0.80 points, and that they leave exits 4 and 5 almost untouched, because those wear the cheap adapter or none.

Exits 0 and 1 are unreachable: the decoder clamps the map at j = 2, so a tile nominally assigned them leaves through exit 2 and wears exit 2's cost. The 2C² and 5C² columns are the vectors recorded in results/why_qp.json and results/static_RECIPE512_b01.json respectively.

The ceiling is $100(1 - c_j)$, the saving when every tile takes the shallowest reachable exit. It is a property of the architecture and no allocation can pass it. Three values are in circulation and they are the three columns of Table 10: 41.91% at 2C² (results/band_collapse.json), 39.1% at 5C² (results/saturation_RECIPE512_ctc53.json, on the pinned checkpoint, and the value the paper's tables use), and 38.33% on the measured block. The last of the three is the one a hook count agrees with, as the next subsection shows.

B.5. Predicted against measured

A cost model and the decoder it models drift. The check that catches it is a hook count: flexuf/measure.py attaches a forward hook to every convolution and linear layer and accumulates MACs from the output shape of each call. A group skipped by an early exit never fires its hook and so costs nothing; a group run on 28 of 40 tiles costs 28 tiles' worth, because the tiles are the batch dimension and the output shape carries them. There is no bookkeeping to get wrong, and the count is the ground truth against which the model is judged.

Condition	Map 2/3/4/5	Measured	at 5C²	diff	Derived	diff
720p q0	11/4/0/0	34.3524	35.1494	+0.797	34.3528	+0.0003
720p q32	6/4/2/3	23.0606	23.6547	+0.594	23.0608	+0.0002
720p q63	4/3/4/4	18.0177	18.4971	+0.479	18.0179	+0.0002
1080p q0	28/10/1/1	32.9763	33.7436	+0.767	32.9766	+0.0003
1080p q32	16/10/6/8	22.8829	23.4682	+0.585	22.8831	+0.0002
1080p q63	11/7/11/11	17.8489	18.3184	+0.469	17.8490	+0.0002

Table 11. Predicted against measured, on six decodes with six different exit maps at two resolutions. “Measured” is the hook count off the decode that was executed and timed. Take from it that the derivation of Tables 7 to 10 reproduces the count to four decimal places in every condition, while the model the paper reports is optimistic by 0.47 to 0.80 points, always in the same direction.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000. The map is the file's pooled exit histogram rounded to the frame's tile count, which is what scripts/power_profile.py decodes; the four columns of the map are the tiles leaving at exits 2, 3, 4 and 5.

Because the count is exactly linear in the exit map, six mixtures spanning four reachable exits over-determine the per-exit vector, and it can simply be solved for. The six equations turn out to be consistent to machine precision, the largest residual being 3×10^{-14} points of saving, so the recovered vector in Table 12 is a measurement and not a fit.

Exit k	c recovered	c derived	diff (10^{-6})	Saved %	Reported %
2	0.616725	0.616721	+3.6	38.3275	39.1245
3	0.765790	0.765788	+2.2	23.4210	24.2179
4	0.872407	0.872406	+1.2	12.7593	13.0270
5	1.010861	1.010861	-0.1	-1.0861	-0.9507

Table 12. The per-exit cost vector recovered from the six hook counts of Table 11 by solving the six linear equations, against the derivation. Take from it that the derivation and the measurement agree to a few parts in a million at every exit, and that the model the paper reports overstates the saving by 0.80 points at exits 2 and 3, 0.27 at exit 4 and 0.14 at exit 5.

Recovered by least squares from the six (map, measured saving) pairs of results/supp_power.json; the system is consistent, so the solution is exact rather than fitted. “Reported” is the 5C² column of Table 10.

Budget (dB)	q0	q16	q32	q48	q63
0.05	0.453	0.320	n/a	n/a	n/a
0.075	0.593	0.515	0.419	0.356	0.296
0.1	0.683	0.598	0.519	0.467	0.433
0.15	0.777	0.725	0.640	0.602	0.563
0.2	0.797	0.791	0.725	0.684	0.649
0.25	0.797	0.797	0.786	0.752	0.704
0.3	0.797	0.797	0.797	0.789	0.757
0.4	0.797	0.797	0.797	0.797	0.797
0.5	0.797	0.797	0.797	0.797	0.797

Table 13. Model minus hook count, in points of saving, over the whole budget grid: 9 budgets by 5 quality indices, 53 sequences, 2 frames each. Take from it that the residual is positive everywhere, that it grows with the budget, and that it saturates at exactly 0.797, which is the residual Table 12 predicts for an exit-2 map. Every entry lies inside the interval [0.135, 0.797] that the per-exit residuals bound it to. n/a marks a budget below the floor, where no allocation is admissible.

results/signalled_RECIPE512_grid.json, pinned checkpoint. Both columns are in the file: saving_pct_vs_release is the model, saving_pct_measured is the hook count, and model_minus_measured is their difference.

The residual is therefore not noise and not a tolerance. It is the one C² of block that the closed form counts twice, propagated through the adapters and the repair pass, and it is largest exactly where the saving is largest. Reading the paper's headline figures as hook counts means subtracting between 0.43 and 0.80 points: at the 0.1 dB budget on the pinned checkpoint the model reports 29.9% at the lowest rate against a measured 29.13%, and 15.6% at the highest against a measured 15.16% (results/signalled_RECIPE512_ctc53.json).

B.6. What the saving is worth in seconds

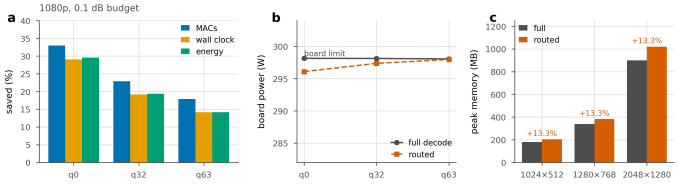

Figure 3. Three units for one saving. a, the same routed decode measured as arithmetic, as wall clock and as joules per frame. **b**, why they differ: the board draws the same power either way, because the later groups run on a shrinking set of tiles and a partly idle GPU still draws its static power. **c**, peak memory, which routing raises rather than lowers.

Arithmetic is the right unit to allocate in, because it is the only one independent of the machine, the driver and whatever else is resident on the card. It is not the unit a deployment cares about. DCVC-RT [33] makes the point for neural codecs generally, arguing that non-computational operational cost rather than arithmetic is the primary bottleneck, and the complexity-aware line of work [35, 36] treats decode cost as an axis to be measured rather than inferred. Table 14 is the stage evidence for why the two diverge here.

Group	1080p tiles	1080p ms/tile	720p tiles	720p ms/tile
2	40	0.3994	15	0.4127
3	28	0.4015	10	0.4157
4	14	0.4088	5	0.4458
5	9	0.4214	3	0.4340

Table 14. Milliseconds per tile in each per-tile group, as the group's tile set shrinks. Take from it that a tile costs more to decode in a group that has fewer tiles left: 5.5% more at 1080p from group 2 to group 5, 8.0% more at 720p from group 2 to group 4. The last groups launch kernels over a handful of 32×32 feature maps, and a GPU sized for the first group is partly idle inside them.

Derived from results/supp_latency_1920x1080.json and results/supp_latency_1280x720.json by dividing each group's measured time by the tiles alive in it, both recorded in the files.

Condition	MACs %	Time %	Gap	Energy %	Above idle %
720p q0	34.35	30.81	3.54	30.10	29.70
720p q32	23.06	18.13	4.93	18.45	18.63
720p q63	18.02	12.99	5.03	12.96	12.94
1080p q0	32.98	29.07	3.91	29.56	29.88
1080p q32	22.88	19.14	3.75	19.34	19.48
1080p q63	17.85	14.14	3.71	14.17	14.18

Table 15. The three units side by side, all as savings of the routed decode against the released decode on the same latent. Take from it the result of this subsection: energy saved tracks time saved to within 0.5 points in every condition, while arithmetic saved runs 3.5 to 5.0 points ahead of both. Subtracting idle power changes nothing, which is the check that the agreement is not an artefact of the static draw.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000 with a 300 W limit, taken under the evaluation lock. 720p is padded to 1280×768 and 15 tiles, 1080p to 2048×1280 and 40 tiles.

The gap between arithmetic and the clock is the shrinking tile set of Table 14, plus the fixed cost of the tiled path itself: patchify, unpatchify and the repair pass together are 2.9% of the routed clock at 1080p and cost nothing in the released decode. The gap is largest where the map is most lopsided, which is the low-rate end, and that is also where the arithmetic saving is largest.

B.7. What the saving is worth in joules

Board power was sampled from the driver at 100 ms, which is about the rate the sensor updates. Each condition therefore runs for a fixed wall time of 20 s rather than a fixed iteration count, so that a 30 ms decode yields two hundred power samples rather than three. Idle is measured for 4 s immediately before and after every timed run on the same card, so a neighbouring process waking up mid-measurement appears as a drift between the two rather than as a saving. Energy per frame is the mean board power over the run times the median decode time.

Condition	ms full	ms routed	W full	W routed	J full	J routed
720p q0	43.45	30.06	283.9	286.8	12.338	8.624
720p q32	43.50	35.61	293.7	292.6	12.774	10.418
720p q63	43.53	37.88	294.1	294.2	12.804	11.144
1080p q0	110.74	78.55	298.1	296.1	33.016	23.257
1080p q32	110.76	89.56	298.1	297.4	33.019	26.633
1080p q63	110.75	95.08	298.0	298.0	33.008	28.332

Table 16. The measurement behind Table 15, in absolute units. Take from it why energy tracks time: the routed decode draws the same board power as the released one, within 3 W in the worst case and within 1 W in four of the six conditions, and at 720p q0 it draws slightly *more*. Routing does not lower the instantaneous draw of the card. It shortens the job.

results/supp_power.json. Watts are the mean of about 200 driver samples per run; times are medians over 181 to 664 iterations. Idle over the six conditions in the order run: 74.6, 78.5, 91.2, 100.5, 106.5 and 98.6 W before, and 122.4, 120.0, 126.2, 128.4, 125.0 and 121.9 W after, on a card whose decode draw is 284 to 298 W.

That is the honest engineering result and it should be stated plainly: the energy saving of this method tracks its time saving, not its arithmetic saving. A partly idle GPU still draws most of its static power, so the joules follow the seconds almost exactly, and the seconds fall short of the multiply-accumulates for the occupancy reason of Table 14. At 1080p and the lowest rate the three read 32.98%, 29.07% and 29.56%; at the highest rate they read 17.85%, 14.14% and 14.17%. Anyone quoting the arithmetic figure as an energy figure overstates it by three and a half to five points.

Two limits on these numbers. The idle baseline drifts upward across the six conditions as the card warms, from 74.6 W before the first to 128.4 W after the fourth, so the above-idle column of Table 15 is the weaker of the two energy figures; that it agrees with the unadjusted column to within 0.4 points is the reason the conclusion survives. And power comes from the driver counter, not from an external meter, so the absolute joules inherit whatever bias that sensor has. The savings are ratios of two measurements taken the same way on the same card minutes apart, which is the quantity the bias mostly cancels in.

B.8. Memory, and what the encoder pays

Frame (padded)	Tiles	Peak MB, full	Peak MB, routed	Change %	Time saved %
1024x512	8	180.0	204.0	+13.33	12.00
1280x768	15	337.5	382.5	+13.33	18.18
2048x1280	40	900.0	1020.0	+13.33	19.26
3840x2304	135	out of memory			

Table 17. Peak decoder memory, released against routed. Take from it that routing costs memory rather than saving it, by a constant 13.33% at every resolution that fits, because the tiled path holds the whole tile batch and the stitched canvas alive together. The 4K row is a genuine failure and is reported as one: the job ran out of memory on a 48 GB card that four other processes were sharing, so no measurement above 1080p exists anywhere in this work.

results/supp_footprint.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000. The 4K stage profile queued alongside it failed the same way (results/supp_queue.log).

The signalled configuration moves the choice of exit map to the encoder, which has to price the tiles before it can allocate them. On one 1080p sequence at the highest rate, one decode takes 114.0 ms, building the cost table full-frame takes 156.9 ms (1.376× one decode), and building it the way the deployed encoder does, per tile, takes 515.4 ms (4.520×). The two searches agree on 0.850 of tiles and reach savings of 18.64% and 19.01% at the same budget. The asymmetry is the design: a decoder-side saving is bought with encoder-side work, which suits a compress-once, decode-many deployment and does not suit live encoding.

results/encoder_cost.json: one sequence (Bosphorus), q63, 40 tiles. The file records no checkpoint field, so this is the one measurement in this section whose checkpoint cannot be stated from the file.

B.9. What these files do not pin

Four gaps, listed so that they are visible rather than quietly absent.

- **The split of the always-on remainder.** Table 9 gives the head 2.40% and the opening upsample 8.16%. results/ pins their sum, 10.56%, and pins everything else in the model, but the split between the two is produced by scripts/mac_audit.py, which prints to stdout and writes no JSON. The same is true of 453.5 GMAC, the absolute figure every row of the main paper's complexity table is normalised by. Neither affects any saving, which is a ratio, but neither is reproducible from results/ as it stands.
- **The paper's tables are on the 5C² column.** Table 10 and Table 13 together say by how much: 0.14 to 0.80 points of saving, always optimistic, largest where the saving is largest. The measured-block column is the one a hook count agrees with, and the honest ceiling is 38.33% rather than 39.1%.
- **Nothing above 1080p.** Both 4K jobs ran out of memory (Table 17). The resolution trend in this section rests on three points, of which two

carry power measurements.

- **One card, one budget, one card-level power sensor.** Every millisecond and joule here is an NVIDIA RTX A6000 at a 0.1 dB budget; the stage profiles are q32 only, and the power runs cover three quality indices at two resolutions. Nothing here separates the decoder's energy from the board's, and no external meter was available to calibrate the driver counter against.

C. Derivations

This section states the allocation problem from its objects upward and proves what the paper asserts about it. Nothing here needs a result from elsewhere in the supplement, and a reader who has not met a Lagrangian rate-distortion argument before should be able to follow every line. Each statement is either proved in place or marked as checked numerically, in which case the file that checks it and the checkpoint it was measured on are named at that point.

C.1. Tiles, exits, and what is measured against what

A tile. The decoder is cut in two at a fixed depth. Everything above the cut runs once over the whole frame; everything below it runs separately on square patches of the trunk's feature grid. Those patches are the tiles. The frame is padded on the bottom and the right before the encoder sees it, by replication, so that the feature grid divides exactly into patches, and the tiling is then a partition: every feature position belongs to exactly one tile and no tile overlaps another. A tile is 32 feature positions on a side, which is 16 latent positions and 256 pixels. We index tiles by t and write N for their number in a frame.

frame	padded to	latent grid	tiles	N
1920x1080	2048×1280	128×80	5×8	40
1280x720	1280×768	80×48	3×5	15
832x480	1024×512	64×32	2×4	8
416x240	512×256	32×16	1×2	2

Table 18. The tiling, at every resolution measured. N is fixed by the frame size and the tile side, not chosen per frame. Read it as the granularity available to the allocation: 40 tiles at 1080p, and only 2 at 416×240, which is why the arguments below that improve as 1/N improve very little at the smallest class.

results/tile_definition.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar. The pad is applied to the RGB frame before the encoder, by replication, on the bottom and right only.

An exit. The per-tile part of the decoder is a stack of 12 residual blocks. An exit is a point in that stack at which a tile may stop and be handed to the reconstruction head. There are K = 6 of them, evenly spaced, so exit k runs (k+1)b blocks with b = 2. The split depth is j = 2: blocks 0 to jb-1 run full-frame for every tile whatever exit it takes, so exits below j are not distinguishable from exit j and the usable ladder has K - j = 4 distinct members. Throughout, k ranges over j, ..., K-1 unless stated otherwise. Because a tile leaving early hands the head a feature the head was not trained to read, each exit below the deepest carries a small learned adapter, zero-initialised so that the ladder starts life as the released decoder [31, 32].

Distortion, and the reference it is measured against. $D_{t,k}$ is the mean squared error tile t incurs when it leaves at exit k. Two choices in that definition do real work. The error is measured on the path actually deployed, a tiled decode with replicate padding at tile borders, so what tiling costs is already inside $D_{t,k}$ rather than being accounted for separately. And it is measured against the released decoder's full-frame decode of the same latent, not against the source image. The encoder, the hyperprior and the entropy model are therefore byte-identical on both sides of the comparison and synthesis is the only thing that differs. A consequence worth stating now: $D_{t,k} = 0$ would mean the exit reproduces the released decoder exactly, and nothing in the construction forces the deepest exit to achieve it. Section C.8 returns to that.

Decibels, and which convention. Distortion enters the optimisation as a mean squared error and is reported as 10 $\log_{10}$ of the ratio to the reference. The two are not interchangeable, because the ratio can be formed at two levels. Pooling every tile of every frame into one mean squared error and taking one logarithm is the form the Lagrangian below is exact for. Averaging a per-frame decibel instead is the codec convention and is what the budget is bisected against in every measurement this project reports. On an identical allocation the two read 0.023 to 0.033 dB apart, which is 23% to 33% of the headline 0.1 dB budget, so a decibel figure is meaningless without its convention.

Convention gap from results/db_convention.json, checkpoint runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar, 40 frames. That is an old checkpoint for a claim about arithmetic rather than about a model, but it is the only file in results/ that measures both conventions on one allocation.

C.2. Compute, and what an exit costs

The unit. Compute is counted in multiply-accumulate operations and then divided by the multiply-accumulates of one full-frame decode by the released decoder, 453.5 GMAC at 1080p. Everything below is in those units, so c = 1 means one released decode and a saving of S% means the frame was decoded for (1 - S/100) of one. Working in a ratio rather than in GMAC is what makes the cost of an exit resolution-independent: every operator in this decoder runs at a fixed multiple of the latent grid, so the shares below do not move with frame size.

453.5 GMAC is produced by scripts/mac_audit.py, which prints and writes no JSON, so it is the one quantity in this section with no file in results/ behind it. It enters only as a normaliser and cancels from every ratio quoted here.

$$c_k = s_{\text{up}} + s_{\text{trunk}} \frac{(k+1)\,b}{N_{\text{blk}}} + s_{\text{head}} + a_k + r \tag{4}$$

Equation (4) is the cost of one tile at exit k, and it is worth reading term by term. s_{up} = 0.0816 is the opening upsample, which runs whatever happens. The middle term is the trunk: s_{trunk} = 0.8944 spread evenly over N_{blk} = 12 blocks, of which exit k runs (k+1)b. s_{head} = 0.0240 is the reconstruction head, which also runs whatever happens. a_k is the adapter that exit k wears and r = 0.0095 is the full-frame deblocking filter that repairs tile borders after the tiles are stitched. Only the trunk term depends on k. That single fact is the reason a ceiling exists at all: however shallow the ladder gets, the stem, the head and the filter are still paid.

Two of those shares are worth pinning down. The adapter share a_k is measured with forward hooks on the real modules: one DepthConvBlock costs 1,035,648 MAC/px, the 1×1 adapter 147,456 or 0.142 of a block, and the FFN adapter 737,280 or 0.712 of a block. The exits skipping four or more blocks get the FFN and the rest the 1×1, and the deepest exit carries none, which is what makes it bit-identical to the released decoder in structure. The three whole-decoder shares are the output of the MAC audit and are not in results/, but they are not free parameters either: 0.0816 + 0.8944 + 0.0240 + 0.0095 = 1.0095072, which is exactly the deepest-exit cost recorded in every configuration file this section reads.

exit k	blocks	adapter	share	c_k	100(1-c_k)	earlier c_k
2	6	ffn	0.712	0.6088	39.12	0.5809
3	8	ffn	0.712	0.7578	24.22	0.7300
4	10	1×1	0.142	0.8697	13.03	0.8697
5	12	none	0.000	1.0095	-0.95	1.0095

Table 19. What each exit costs, in units of one released full-frame decode. The deepest exit costs more than 1 because a tiled decode still pays the deblocking filter and the tile borders. The last column is the earlier cost vector, built when the FFN adapter was billed at $2C^2$ instead of its measured $5C^2$; the numerical checks in C.7, C.9 and C.12 were run against it and are labelled where they appear.

c_k and 100(1-c_k) from the uniform-depth rows of results/static_RECIPE512_b01.json, and c_j independently from results/saturation_RECIPE512_ctc53.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar. Adapter shares from results/adapter_cost.json, same checkpoint. Earlier vector from results/tile_table.json. The complexity section of this supplement audits the block count itself.

The four usable exits therefore cost 0.6088, 0.7578, 0.8697 and 1.0095 of a released decode. The gaps between them are what the whole allocation trades in, and they are unequal: 0.1491, 0.1119 and 0.1398. The first gap is the largest because exit 3 gives up the FFN adapter's saving as well as two blocks.

C.3. An assignment, and the two things assumed about it

An *assignment* is a map a from tiles to exits, a(t) being the exit tile t takes. There are $(K\text{-}j)^N$ of them, which is 4^{40} at 1080p, so they are not going to be enumerated. The frame's compute and the frame's distortion are

$$C(a) = \tfrac{1}{N}\sum_{t=1}^{N} c_{a(t)}, \qquad D(a) = \tfrac{1}{N}\sum_{t=1}^{N} D_{t,a(t)} \tag{5}$$

Equation (5) carries two assumptions and it is worth being explicit about both, because everything after this rests on them and neither is a mathematical necessity. The first is that compute is additive over tiles: the frame costs the mean of what its tiles cost. That is exact when tiles are decoded independently, which is what a tiled decoder does, and it is why the stem and the head appear inside c_k rather than as separate terms. The second is that distortion is a tile mean, which is exact for any per-pixel loss averaged over a frame provided a tile's error does not depend on which exits its neighbours took. That second proviso is not free: the deblocking filter runs on the stitched canvas and therefore sees the whole exit map at once.

So the second assumption is checked rather than asserted. The per-tile table and the measured decode of the corresponding mixed map are both recorded for 60 allocations of one frame, spanning savings from 1.19% to 41.91%. Predicting the frame's decibels from the tile means and comparing with the decode, the largest disagreement over all 60 allocations is 5.5×10^{-5} dB, which is four orders of magnitude below the budget. Neither assumption constrains $D_{t,k}$ itself. In particular nothing requires $D_{t,k}$ to fall as k grows, and Table 21 below contains a tile for which it does not.

Additivity check computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPE512/ckpt_eval.pth.tar. That checkpoint has since been overwritten by the per-epoch watcher; the arithmetic it checks is a property of the decode path rather than of the weights.

C.4. A price on compute

The problem the encoder faces is to spend as little compute as possible while staying inside a distortion budget:

$$\min_{a} C(a) \quad \text{subject to} \quad D(a) \le D_{\text{budget}} \tag{6}$$

Constraint (6) couples the tiles. A tile cannot be given an exit on its own merits, because whether it can afford to leave early depends on how much of the budget the other 39 tiles have used. The standard way out is to buy the constraint off with a price. Introduce a number $\lambda \ge 0$, charge λ per unit of compute, and minimise the total bill:

$$L(a,\lambda) = D(a) + \lambda\, C(a) = \tfrac{1}{N}\sum_{t=1}^{N} \left[D_{t,a(t)} + \lambda\, c_{a(t)} \right] \tag{7}$$

It is worth being slow about what λ is. It is an exchange rate. Its units are squared error per unit of relative compute, so $\lambda\, c_k$ converts the compute an exit uses into the squared error that compute is deemed to be worth. In the numbers below λ runs from about 2×10^{-7} to about 1×10^{-4}, against squared errors of order 1×10^{-4}, because the reference images are on a 0 to 1 scale. Nothing depends on the scale itself; what matters is where λ sits relative to the differences between exits.

Two extremes fix the picture. At $\lambda = 0$ compute is free, the second term vanishes, and every tile takes whichever exit has the smallest error, whatever it costs. As λ grows, the compute term grows for every exit but grows fastest for the deepest one, because c_k is largest there. A tile therefore drops to a shallower exit at the moment the compute that exit saves, times the price, exceeds the error it adds. Past a large enough λ every tile has dropped as far as the ladder allows. Moving λ from 0 upward sweeps the decoder from most accurate and most expensive to cheapest and worst, and does so without ever turning back, which C.4 proves.

The Lagrangian separates

Nothing so far has removed the coupling; equation (7) is still a minimisation over $(K\text{-}j)^N$ assignments. What removes it is that the sum has no cross terms.

Proposition 1 (separation). For every $\lambda \ge 0$, an assignment minimises $L(a,\lambda)$ over all assignments if and only if it minimises the bracket tile by tile:

$$k_t^\star(\lambda) \in \arg\min_{k \ge j} \left[D_{t,k} + \lambda\, c_k \right] \tag{8}$$

Proof. Write $f_t(k) = D_{t,k} + \lambda\, c_k$. Then $N\,L(a,\lambda) = \sum_t f_t(a(t))$, a sum in which the t-th term depends on a only through a(t). For any assignment a, term by term $f_t(a(t)) \ge \min_k f_t(k)$, so $N\,L(a,\lambda) \ge \sum_t \min_k f_t(k)$, and the right-hand side is attained by any a that picks a minimiser in every tile. Conversely if a is optimal and some tile t had $f_t(a(t)) > \min_k f_t(k)$, replacing a(t) by that minimiser and leaving every other tile alone would strictly lower L, contradicting optimality.

The N-dimensional search has become N independent searches over 4 numbers each. Ties are broken toward the shallower exit throughout, which makes k^* a function rather than a set-valued map and costs nothing, because a tie means the two choices have identical Lagrangian value. We call $k^*_t(\lambda)$ the *oracle*'s choice, because it uses the true $D_{t,k}$, which only the encoder has. The construction is Shoham and Gersho's [28], with compute in place of rate, and Ortega and Ramchandran [29] made it standard in image and video coding.

The sweep is monotone, so bisection finds the budget

Proposition 2 (monotonicity). Let $\lambda_1 < \lambda_2$ and let a_1, a_2 be the corresponding oracle assignments. Then $C(a_2) \le C(a_1)$ and $D(a_2) \ge D(a_1)$.

Proof. Fix a tile and drop the index. Optimality at each price gives $D_1 + \lambda_1 c_1 \le D_2 + \lambda_1 c_2$ and $D_2 + \lambda_2 c_2 \le D_1 + \lambda_2 c_1$, where (D_i, c_i) is the pair chosen at λ_i. Adding the two and cancelling $D_1 + D_2$ leaves $(\lambda_2 - \lambda_1)(c_2 - c_1) \le 0$, hence $c_2 \le c_1$. Substituting that back into the first inequality gives $D_2 \ge D_1 + \lambda_1(c_1 - c_2) \ge D_1$. Averaging over tiles preserves both.

This is the standard exchange argument and it is what makes the budget searchable. $D(a^*_\lambda)$ is non-decreasing in λ, so bisecting λ against the measured decibels converges on the largest price whose allocation still meets the budget, and by Proposition 2 that is also the cheapest such allocation the family contains. Every λ quoted in this project is found that way, and so is the router's β. Monotonicity is also checked numerically, over 6001 multipliers on a measured tile table, in scripts/verify_theory.py.

tile	bits	D(2)	D(3)	D(4)	D(5)	exits as λ rises
34	10,446	1.608	1.638	1.679	1.655	2 throughout
0	12,996	9.346	9.249	9.207	9.180	5, 4 at 1.9, 3 at 3.0, 2 at 6.5
18	16,989	16.249	15.795	15.638	15.606	5, 4 at 2.3, 3 at 11.2, 2 at 30.6

Table 20. Three tiles of one frame, and what the price does to them. D is in units of 10^{-5} squared error; switch points are in units of 10^{-6} of λ. The cheapest tile prefers the shallowest exit at every price, including $\lambda = 0$, because its error does not fall with depth at all: depth is not always worth buying. The most expensive tile holds its deepest exit five times longer than the middle one. This is the whole mechanism, and it is why a single uniform depth cannot be right for both.

λ	n at 2	3	4	5	saved %	dB
2.00e-07	2	0	0	38	1.19	0.0392
6.45e-07	2	1	1	36	2.24	0.0395
1.65e-06	3	6	4	27	7.86	0.0439
3.32e-06	4	18	9	9	19.06	0.0614
6.71e-06	11	23	5	1	28.66	0.0909
1.07e-05	21	16	3	0	33.78	0.1192
1.71e-05	28	12	0	0	37.44	0.1482
2.73e-05	38	2	0	0	41.17	0.1975
4.37e-05	40	0	0	0	41.91	0.2112
8.81e-05	40	0	0	0	41.91	0.2112
2.00e-04	40	0	0	0	41.91	0.2112

Table 21. The same frame swept. Each row is one price and the exit histogram it produces over the frame's 40 tiles. The allocation moves continuously from almost everything deep to everything shallow, and both ends are flat: below the first row nothing more is gained and above the last row nothing more is available. The 60 prices in the file produce only 33 distinct savings, which is the granularity C.7 quantifies.

C.5. The achievable set

The pairs (C(a), D(a)) an allocation can produce form a set, and the shape of that set is what determines both what a sweep can reach and what content adaptivity is worth. There are two versions of it and they are genuinely different. Write

$$\bar{D}_k = \tfrac{1}{N} \sum_{t=1}^{N} D_{t,k} \tag{9}$$

for the mean distortion of exit k over the frame. We call it the *exit mean*, and it is the only place the tiles are averaged before anything is chosen.

Proposition 3 (fixed proportions). Suppose tiles are assigned to exits in proportions p, one probability per exit, independently of their content. Then the achievable set of expected pairs is exactly the convex hull of the K-j points whose coordinates are the cost c_k and the exit mean at k.

Proof. Under content-independent proportions the expected cost is $\sum_k p_k c_k$ and, by the tile mean, the expected distortion is $\sum_k p_k$ times the exit mean at k. The map from p to that pair is affine and its domain is the probability simplex, whose extreme points are the unit vectors. An affine image of a simplex is the convex hull of the images of its vertices, and those images are exactly the cost and exit mean of each exit.

q	exit 2	exit 3	exit 4	exit 5	best single
q0	0.1791	0.0932	0.0589	0.0361	3 at 24.2%
q16	0.2194	0.1181	0.0755	0.0452	4 at 13.0%
q32	0.2819	0.1472	0.0897	0.0543	4 at 13.0%
q48	0.3330	0.1757	0.1030	0.0622	5 at -1.0%
q63	0.3955	0.2057	0.1156	0.0716	5 at -1.0%

Table 22. The vertices of the fixed-proportion set, in dB below the released decoder, one uniform-depth decode per column. These four points and their hull are everything a content-blind allocation can reach. The last column is the shallowest uniform depth that still meets a 0.1 dB budget, and its saving: at the two highest rates that is the deepest exit, which saves nothing at all.

results/static_RECIPE512_b01.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. The mean over rates of the best single depth is 9.7%, against 22.1% for the per-tile allocation at the same budget.

Theorem 4 (per-tile assignment). The convex hull of the pairs achievable by per-tile assignment is the Minkowski average of the N per-tile hulls,

$$\mathcal{A}_{\text{tile}} = \tfrac{1}{N} \bigoplus_{t=1}^{N} \operatorname{conv}\{(c_k, D_{t,k}) : k \geq j\} \tag{10}$$

Proof. By the two assumptions, an achievable pair is the average of one point drawn from each per-tile set S_t, the set of pairs cost c_k against error $D_{t,k}$, so the achievable set is the Minkowski average of the S_t. Taking convex hulls commutes with Minkowski addition, the hull of a sum being the sum of the hulls, and with positive scaling; applying that N-1 times moves the hull inside the sum.

Proposition 3 is the special case in which every tile has the same row, so that each tile's error at k equals the exit mean there. Otherwise the per-tile set is strictly larger, and the difference is visible in the vertex count. The fixed-proportion hull has at most K-j = 4 vertices, so its lower boundary is a chain of 3 straight segments. The Minkowski average carries up to N(K-j-1) = 120 vertices at 1080p, and the sweep behind Table 21 finds 33 distinct cost levels on one frame. That is the difference between a frontier with three kinks and a frontier that looks smooth at any scale one plots, and C.10 shows why conflating them leads to a confident statement about a shape that is an artefact of sampling.

C.6. What the sweep reaches, and what it cannot

Proposition 5 (the sweep traces the lower convex hull). For every $\lambda \geq 0$ the point (C, D) produced by the oracle assignment lies on the lower convex hull of the achievable set, and no λ produces a point strictly inside it.

Proof. By Proposition 1 the oracle assignment minimises D(a) + λ C(a) over all assignments, so its pair minimises the linear functional (C, D) → D + λ C over the achievable set and hence over that set's convex hull. A minimiser of a linear functional over a compact convex set lies on its boundary, and because $\lambda \geq 0$ the supporting line has slope $-\lambda \leq 0$, which places the point on the lower boundary. Conversely a point strictly inside the hull is a strict convex combination of achievable points and is therefore strictly beaten, at that same C, by some point of the hull; it cannot minimise any such functional.

The converse has a practical edge and it cuts both ways. Everett's generalised multiplier method (1963) supplies the reassuring half: whatever compute the returned allocation happens to consume, it is optimal for that level, so a sweep never returns something dominated. Proposition 5 supplies the other half: a budget whose optimum sits between two hull vertices is not reachable by any price at all, and the bisection returns the nearest reachable point below it. The question is how much that costs, and it can be answered exactly, because with only four distinct exit costs the whole Pareto set can be enumerated by dynamic programming over tiles.

budget dB	by sweep	exact Pareto	gap
0.05	12.398	12.421	0.0230
0.08	25.443	25.489	0.0452
0.10	30.475	30.496	0.0209
0.12	33.782	33.803	0.0211
0.15	37.439	37.461	0.0211
0.18	40.048	40.046	-0.0018
0.20	41.166	41.165	-0.0016

Table 23. What convexity costs. Savings in per cent at seven budgets, on one frame. The sweep reaches 95 allocations against the 635 on the exact Pareto set, and the largest loss over these budgets is 0.045 saving points. The two negative entries are the sweep landing on a budget the enumeration's grid straddles, not the sweep beating the Pareto set, which Proposition 5 forbids.

results/hull_gap.json, Bosphorus q32, 40 tiles, on the earlier cost vector; the enumeration and the sweep use the same vector, so the gap is unaffected by the correction.

Why is the loss that small? Not because the images cooperate. The reachable costs form a lattice, and the spacing of that lattice is set by how far the mean cost moves when λ crosses a point at which some tile changes its mind.

Proposition 6 (granularity). If at most m tiles switch exit at any single breakpoint of the sweep, and each switch moves a tile by one rung, then consecutive reachable savings differ by at most

$$\text{spacing} \leq \frac{100\, m \max_k (c_{k+1} - c_k)}{N} \ \% \tag{11}$$

and the loss from convexity cannot exceed the largest spacing.

Proof. Between breakpoints the assignment is constant, so the reachable costs are exactly the costs at the breakpoints. At one breakpoint the mean cost changes by (1/N) times the sum of the individual changes, each of which is at most $\max_k(c_{k+1} - c_k)$ in magnitude, and there are at most m of them; multiplying by 100 puts it in saving points. For the second part, let a budget fall strictly between two reachable levels. By Proposition 2 the bisection returns the lower level, so the loss is at most the distance between them.

On the measured table m = 2, because tiles with identical rows switch together, the largest rung is 0.1491 and N = 40, so the bound is 0.7453 saving points. The measured spacing is 0.7453, sitting exactly on the bound, and the convexity loss of Table 23 is 0.0452, comfortably under it. Nothing in the expression depends on content or on rate, and it falls as 1/N: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine granularity, not a property of the pictures, is what makes the Lagrangian relaxation lossless in practice, and by Table 18 the smallest test class has N = 2, where it is not.

m, the largest rung, the bound and the measured spacing from results/theory_checks.json (Bosphorus q32, 40 tiles, runs/RECIPE512/ckpt_eval.pth.tar), written by scripts/verify_theory.py, which reports seven of seven propositions passing.

C.7. Floor, saturation and ceiling

The sweep has two ends and both are worth naming, because a budget outside them buys nothing.

Proposition 7 (the floor). At $\lambda = 0$ the allocation attains $D_{min} = (1/N) \sum_t \min_k D_{t,k}$, and no allocation attains less. *Proof.* Immediate from Proposition 1 with $\lambda = 0$, and from the fact that D(a) is a mean of terms each bounded below by its own tile minimum.

Because $D_{t,k}$ is measured on the deployed tiled path, D_{min} is not zero: it is what tiling costs before any tile exits early, and a budget below it admits no allocation at all.

Proposition 8 (saturation). Define

$$\lambda_{\mathrm{sat}} = \max_t \max_{k>j} \frac{D_{t,j} - D_{t,k}}{c_k - c_j} \tag{12}$$

Then for every $\lambda > \lambda_{sat}$ the oracle assigns exit j to every tile, the frame's cost is exactly c_j, and no larger price changes anything.

Proof. Exit j beats exit k on tile t precisely when $D_{t,j} + \lambda c_j \le D_{t,k} + \lambda c_k$, that is when $\lambda \ge (D_{t,j} - D_{t,k})/(c_k - c_j)$, the denominator being positive for $k > j$. Taking the maximum over k and then over t gives a single price beyond which exit j wins everywhere. The cost of the constant map is c_j by equation (5).

On the worked frame that closed form gives $\lambda_{sat} = 3.045 \times 10^{-5}$, which is exactly the last switch of the most expensive tile in Table 20: the price at which the frame saturates is the price at which its most stubborn tile gives up.

Proposition 9 (the ceiling). The largest saving any allocation can reach is

$$S_{\max} = 100(1 - c_j)\% \tag{13}$$

Proof. C(a) is a mean of costs each at least c_j, because j is the shallowest usable exit, so $C(a) \ge c_j$ with equality only for the constant map, which Proposition 8 shows is attained.

S_{max} = 39.1% for the shipped ladder. It is a function of the split depth alone, so it does not move with content, with rate or with training, and that is checkable: over the 18 class-and-rate cells measured at a saturating budget the reported ceiling spreads by 6e-06 saving points, which is single-precision noise accumulated through 18 independent decodes.

Per-class invariance computed from results/per_class_RECIPE512.json at budgets of 0.5 dB and above, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar. The ceiling itself is 39.1%, from results/saturation_RECIPE512_ctc53.json.

q	floor dB	saturation dB	band dB	0.1 dB uses
q0	0.0361	0.1791	0.1431	45%
q8	0.0402	0.1945	0.1543	39%
q16	0.0452	0.2194	0.1741	31%
q24	0.0503	0.2514	0.2010	25%
q32	0.0543	0.2819	0.2276	20%
q40	0.0586	0.3049	0.2463	17%
q48	0.0622	0.3330	0.2707	14%
q56	0.0665	0.3653	0.2988	11%
q63	0.0716	0.3955	0.3239	9%

Table 24. The band a budget works in. Below the floor no allocation exists; at or above saturation every allocation is the same one and reaches 39.1%. The last column places the headline budget inside that band: it uses 45% of it at the lowest rate and 9% at the highest, so the same 0.1 dB is a loose budget at low rate and a tight one at high rate.

results/saturation_RECIPE512_ctc53.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. Floor and saturation are 0.036 to 0.072 dB and 0.179 to 0.396 dB across the five reported rates.

The floor is also the one place where the derivation is exposed to training. Every quantity here is measured against a reference decoder and the deepest exit is assumed to reproduce it; nothing in Propositions 1 to 9 enforces that. If the deepest exit drifts from the reference by δ then $D_{t,K-1}$ is raised for every tile, the whole achievable set shifts upward by δ before any compute is saved, and a budget below δ becomes unreachable at every allocation. Routing has not failed there; the achievable set simply does not meet the constraint. This is why the training objective carries an anchor term tying the deepest exit to the released decoder, and the size of the effect is measurable: with the anchor the deepest-exit drift is 0.005 to 0.036 dB over q0 to q63, and on a run trained without it the same drift is 0.110 to 0.159 dB, which on its own consumes more than the headline budget.

Anchored drift from results/anchor_RECIPE512_ctc53.json, checkpoint runs/RECIPE512/ckpt_eval.pth.tar, 53 frames. Unanchored drift from results/anchor_VERBATIM.json, checkpoint runs/VERBATIM/ckpt_eval.pth.tar, 40 frames, a different training run at three rates. Neither is the pinned checkpoint and the two are not a controlled pair; the direction is what the argument needs, and the interval is quoted as measured rather than as a difference.

The floor is not an abstraction either. Over the 45 budget-and-rate cells of the dense sweep, 3 are unreachable, all of them at the 0.05 dB budget and all at rates whose floor already exceeds it.

results/signalled_RECIPE512_grid.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences, 2 frames per sequence, 9 budgets by 5 rates.

C.8. The value of adaptivity, and a bound on it

Everything so far describes what a content-aware allocation does. The next question is what content awareness is worth, and it can be answered without building a predictor at all. Compare the best per-tile allocation with the best content-blind one at the same price:

$$J_{\mathrm{ad}}(\lambda) = \frac{1}{N}\sum_t \min_k [D_{t,k} + \lambda c_k], \quad J_{\mathrm{fix}}(\lambda) = \min_k [\bar{D}_k + \lambda c_k] \tag{14}$$

J_{ad} takes the minimum inside the average and J_{fix} takes it outside. That is the entire difference, and by Propositions 1 and 3 the first is the best value the per-tile family can attain at price λ and the second is the best the fixed-proportion family can attain.

Theorem 10 (adaptivity gain). For every $\lambda \ge 0$, $\Delta(\lambda) = J_{fix}(\lambda) - J_{ad}(\lambda) \ge 0$, with equality if and only if some single exit k^* attains the per-tile minimum for every tile.

Proof. Fix λ and let k be any exit. Then the exit mean at k plus λc_k = $(1/N) \sum_t [D_{t,k} + \lambda c_k] \ge (1/N) \sum_t \min_{k'}[D_{t,k'} + \lambda c_{k'}] = J_{ad}(\lambda)$, because each summand dominates the corresponding minimum. The right-hand side does not depend on k, so taking the minimum over k on the left

preserves the inequality and gives $J_{fix} \geq J_{ad}$. For equality, suppose some k^* attains the per-tile minimum everywhere; then choosing it makes every summand equal to its minimum and the inequality is tight. Conversely suppose $\Delta = 0$ and let k^* attain J_{fix}. Then $(1/N) \sum_t ([D_{t,k^*} + \lambda c_{k^*}] - \min_{k'}[D_{t,k'} + \lambda c_{k'}]) = 0$. Every summand is non-negative, so every summand is zero, which says exactly that k^* attains the minimum on every tile.

Three things follow that are worth separating. First, Δ depends only on the table $D_{t,k}$ and the costs, so it is computable before any router exists and it upper-bounds what any router can gain over a content-blind allocation at that operating point. Second, the equality condition says when adaptivity is worthless, and it is a condition on the data, not on the method: when every tile agrees on the best exit there is nothing to adapt to. Third, and less comfortably, Δ is a property of the ladder it is measured on. Improving the shallow exits compresses the differences between exits, more tiles then agree on the argmin, and the measured value of adaptivity falls even as the saving rises.

q	$\lambda=10^{-5}$	3×10^{-5}	10^{-4}	3×10^{-4}
q0	0.30%	0.39%	0.45%	0.03%
q16	0.65%	0.74%	0.38%	0.01%
q32	0.62%	1.41%	0.37%	0.00%
q48	0.62%	2.00%	0.30%	0.00%
q63	0.68%	2.09%	0.27%	0.00%

Table 25. The adaptivity gain in scale-free form, $\Delta(\lambda)$ as a percentage of $J_{ad}(\lambda)$. At the two smaller prices it rises with rate, which says adaptivity is worth most where the deep blocks are doing real work. At the largest price it is essentially zero at every rate, which is Theorem 10's equality condition arriving: the oracle itself has collapsed onto one exit, so there is nothing left for a router to recover.

results/theory_check.json, checkpoint runs/BEST/ckpt_eval.pth.tar, 40 sequences (UVG, MCL-JCV and HEVC class E; classes B, C and D not measured), on the earlier cost vector of Table 19. This is the one table in the section that is not on the pinned checkpoint, and nothing on the pinned checkpoint currently reproduces it.

q	blind %	oracle %	gap	blind dB	shuffled dB
q0	25.38	29.90	4.52	0.1262	0.1248
q16	19.45	25.63	6.18	0.1278	0.1335
q32	15.02	20.93	5.91	0.1304	0.1411
q48	12.00	18.28	6.27	0.1373	0.1451
q63	8.05	15.62	7.57	0.1367	0.1413

Table 26. The same gap in the units the paper reports. Columns 2 to 4 are savings at a 0.1 dB budget: the best content-blind mixture of uniform depths, the per-tile oracle, and the difference. Columns 5 and 6 are decibels at the oracle's own compute: what the best blind mixture would read there, and what a random shuffle of the oracle's exit map actually reads. Adaptivity is worth 4.5 to 7.6 saving points, rising with rate, against an oracle reading 0.100 dB and a shuffle reading up to 0.141 dB.

Computed from results/static_RECIPE512_b01.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. The blind columns are the lower hull of the four uniform-depth points of Table 22 under Proposition 3, evaluated at the budget and at the oracle's compute; the shuffle column is a measured decode. At q0 the shuffle reads slightly better than the hull predicts, which bounds the interaction the additivity assumption ignores at under 0.002 dB.

C.9. Convexity in the units the frontier is plotted in

Theorem 4 gives D convex in C. The frontier is almost never plotted in those units: it is plotted as decibels against percentage saving, and convexity there is a different statement, because the logarithm is concave and increasing and so preserves neither convexity nor concavity. With $S = 1 - C/c_{K-1}$,

$$\mathrm{dB}(S) = \tfrac{10}{\ln 10} \ln D(C), \qquad C = (1-S)\, c_{K-1} \tag{15}$$

Proposition 11. Let D be twice differentiable along the frontier. Then dB(S) is convex if and only if D is log-convex in C:

$$D D'' \geq (D')^2 \quad \Longleftrightarrow \quad \tfrac{d}{dC}\left(\tfrac{1}{\lambda}\right) \geq \tfrac{1}{D} \tag{16}$$

Proof. C is affine in S and convexity is preserved under affine reparametrisation, so by (15) dB is convex in S if and only if ln D is convex in C. Differentiating twice, $(\ln D)'' = D''/D - (D'/D)^2 = [D\,D'' - (D')^2]/D^2$, and $D > 0$, so $(\ln D)'' \geq 0$ if and only if $D\,D'' \geq (D')^2$. For the second form, the slope of the frontier is $D' = -\lambda$ by the supporting-line argument of Proposition 5, so $D'' = -\lambda'$; substituting gives $-\lambda' D \geq \lambda^2$, and dividing by $\lambda^2 > 0$ gives $-\lambda'/\lambda^2 \geq 1/D$, whose left-hand side is $d(1/\lambda)/dC$. Read the second form as a rate condition: distortion has to fall at least exponentially in compute for the plotted curve to be convex.

This is a property of the decoder and not a consequence of anything above; a convex D can fail it. It is also directly checkable, so it is checked rather than assumed. Two remarks make the differentiability hypothesis harmless in one case and misleading in the other. Under fixed proportions D is piecewise linear with K-j vertices, so ln D is piecewise *concave* with upward jumps in its derivative at the vertices: neither convex nor concave, and the shape one sees depends entirely on where one samples. Under per-tile assignment the hull carries up to N(K-j-1) vertices and is smooth at any plotted scale. The two cases are genuinely different, and treating a fixed-proportion curve as though it were the per-tile one is how a confident claim about frontier shape turns out to be an artefact of vertex spacing.

q	points	condition holds	margin	quartic fit
q0	16	100.0%	0.0024	84.0%
q16	14	100.0%	0.0050	76.3%
q32	13	100.0%	0.0088	75.3%
q48	13	100.0%	0.0145	73.7%
q63	13	100.0%	0.0230	74.7%

Table 27. Testing log-convexity on the measured frontier. The direct test asks whether the secant slopes of ln D against S are non-decreasing; the margin is the smallest increase attained. The condition holds at every point of every frontier, and the margin grows tenfold from the lowest rate to the highest. Fitting a quartic and differentiating it twice instead reports 73.7% to 84.0%, and the difference is the fit rather than the data: near the ceiling the frontier is close to vertical and a polynomial overshoots, whereas the secant test has no fitted degree to choose.

results/logconvexity.json. That file records no checkpoint, so it cannot be attributed from disk; its five frontiers have 13 to 16 points each. The quartic column is the file's own quartic_frac_ok field.

C.10. How much of the oracle a partial signal recovers

The decoder cannot run the argmin of Proposition 1, because $D_{t,k}$ is an error against a source the decoder never receives. A predictor can guess it. Writing $p_{t,k}$ for a head's probability that tile t belongs at exit k, its rule is

$$\hat{k}_t = \arg\max_k [\log p_{t,k} - \beta c_k] = \arg\min_k \left[\tfrac{-\log p_{t,k}}{\kappa} + \lambda c_k\right] \tag{17}$$

The second form of (17) is the same rule with $\beta = \lambda\,\kappa$, where κ is a fixed calibration turning nats into squared error. It is written that way to make the point that a router is the oracle of (8) with a surrogate in place of $D_{t,k}$, and that the same price governs both. Given the two, define the per-tile *regret* at a fixed price:

$$r_t(\lambda) = [D_{t,\hat{k}_t} + \lambda c_{\hat{k}_t}] - [D_{t,k_t^\star} + \lambda c_{k_t^\star}] \geq 0 \tag{18}$$

Non-negativity is immediate from (8). The regret is exactly what the frame's Lagrangian loses on tile t by trusting the prediction, and by Proposition 1 those losses add. That gives the encoder an option the two extremes do not have: signal the exits of a subset S of the tiles and let the head decide the rest. Signalling S removes exactly $\sum_{t \in S} r_t$ from the objective, no more and no less, which is the whole reason a partial signal is analysable at all.

Proposition 12 (Lorenz bound). Fix λ and let $r_{(1)} \geq r_{(2)} \geq \ldots \geq r_{(N)}$ be the regrets in decreasing order with total R. For any subset S of size s, the fraction of the total regret that signalling S removes is at most

$$L(\rho) = \frac{1}{R}\sum_{i=1}^{\lceil \rho N \rceil} r_{(i)}, \qquad \rho = s/N \tag{19}$$

with equality if and only if S is a set of s largest regrets. L is the Lorenz curve of the regret distribution: it is non-decreasing, concave, runs from 0 to 1, and lies above the diagonal by an amount the Gini coefficient summarises.

Proof. The sum of any s of the r_t is at most the sum of the s largest, since replacing an element of S by a larger one outside S cannot decrease the sum, and repeating that exchange terminates at a set of s largest values. Dividing by R gives the bound. Concavity holds because the increments $r_{(i)}$ are non-increasing by construction.

The practical content is that a highly unequal regret distribution can be repaired cheaply. If a tenth of the tiles carry half the regret then signalling a tenth of them removes half of what the predictor gives up, at a tenth of the bits. Whether that holds is a measurement, and the Gini coefficient of the regret is the one number that says so.

q	Gini	L(.1)	rec(.1)	L(.2)	rec(.2)	L(.5)	rec(.5)
q0	0.53	35%	29%	51%	53%	87%	103%
q16	0.49	27%	33%	46%	59%	87%	102%
q32	0.61	37%	40%	59%	68%	95%	106%
q48	0.67	45%	39%	65%	63%	96%	100%
q63	0.73	53%	39%	72%	62%	99%	94%

Table 28. The Lorenz curve of the regret, and what signalling actually recovers. L(ρ) is the share of the total regret carried by the ρ worst tiles, which Proposition 12 says is the most any signal of that size can remove at a fixed price; rec(ρ) is the share of the predictor-to-oracle gap the measured sweep recovers. Regret is concentrated at every rate, Gini 0.49 to 0.73: a tenth of the tiles carries 27% to 53% of the total regret and recovers 29% to 40% of the gap.

results/hybrid_RECIPE512_b01_fixed.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget; the head it is measured against was trained on runs/RECIPE512/ckpt_eval.pth.tar. rec(ρ) is computed here from the same file as the saving at ρ, relative to the ρ = 0 and ρ = 1 endpoints.

The measured recovery is not bounded by L, and the reason is not a failure of Proposition 12. Over every reachable cell of the file, three ρ of which appear in Table 28, the ratio rec/L runs from 46% to 127%. Proposition 12 is a statement at a *fixed* price, whereas the measurement re-bisects λ at every ρ so that the frame lands back on its budget. Re-bisecting enlarges the feasible set: the overridden tiles and the predicted ones then effectively carry two multipliers, and pairs of multipliers reach allocations that no single one does. That is also why signalling half the tiles beats signalling all of them at 4 of 5 rates. The Lorenz curve is the right prediction to hold in mind and the wrong thing to call a bound on the deployed system.

ρ	map bits per frame	recovery at q0	at q63
0.05	14.4	13%	18%
0.1	25.8	29%	39%
0.2	44.0	53%	62%
0.35	66.2	82%	88%
0.5	83.3	103%	94%
1	99.9	100%	100%

Table 29. What the signal costs. The map is an entropy-coded mask over tiles plus an index for each override. A fifth of the tiles costs 44 bits per 1080p frame and recovers over half the gap; the ρ = 1 endpoint, which reproduces the oracle's own allocation, costs 100 bits under this coder. For scale, a fixed-length code over the K exits would be 3 bits per tile, 120 bits per frame at 1080p, and the usable alphabet of K-j = 4 exits needs only 2, so the full map is already coded below its fixed-length size.

results/hybrid_RECIPE512_b01_fixed.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar. Configuration A's own map is 79 to 95 bits per frame across the five rates (results/signalled_RECIPE512_ctc53.json, same checkpoint), or 0.021% of a typical bitrate.

C.11. The rate-rank rule, and the model it comes from

A trained head is one way to guess the oracle. There is a cheaper one, and it is worth deriving rather than presenting as a heuristic, because the derivation says exactly when it must work and exactly how it fails. The decoder holds, before the trunk runs, the number of bits the entropy coder spent on each tile's latents. Write b_t for that count divided by the frame's mean, so b_t is around 1.

The model is separability, in log space: a tile has one scalar difficulty, and the ladder has one profile that applies to every tile in proportion.

$$\log D_{t,k} \approx u_t + \log \varphi_k, \qquad u_t = \alpha \log b_t + c \tag{20}$$

Equivalently $D_{t,k} = g_t\,\phi_k$ with $g_t = \exp(u_t) > 0$. The exit profile ϕ is K-j numbers fitted once per rate; α and c are two more. Nothing is learned per frame and nothing is transmitted, which is why the rule costs the decoder exactly zero. α, c and the profile are all fitted leave-one-sequence-out, so no sequence contributes to the profile used to route it.

Proposition 13 (a rank-1 table gives a threshold rule). If $D_{t,k} = g_t\,\phi_k$ with $g_t > 0$ then

$$\arg\min_k [g_t \varphi_k + \lambda c_k] = \arg\min_k \left[\varphi_k + \tfrac{\lambda}{g_t} c_k\right] \tag{21}$$

so a tile's exit depends on the tile only through the single scalar λ/g_t, and by Proposition 2 the chosen exit is non-increasing in it. Sorting tiles by g_t and cutting the sorted list at K-j-1 thresholds therefore reproduces the oracle exactly.

Proof. Divide the bracket by $g_t > 0$, which does not move the argmin. The reduced problem is the one-tile problem of (8) with distortion ϕ and price λ/g_t, so Proposition 2 applies to it and its solution is non-increasing in the price. Since λ/g_t is decreasing in g_t, the exit is non-decreasing in g_t: harder tiles go deeper. The thresholds are the breakpoints of that one common problem and there are at most K-j-1 of them.

Proposition 14 (a rank-1 rule uses only the profile's hull). Under the same model, an exit k is chosen for some tile at some price only if (c_k, ϕ_k) lies on the lower convex hull of the profile. *Proof.* By Proposition 13 the reduced problem is a single Lagrangian sweep over the points (c_k, ϕ_k), and by Proposition 5 such a sweep reaches only hull points.

q	α	ϕ(2)	ϕ(3)	ϕ(4)	ϕ(5)	on hull
q0	9.98	1.0242	1.0009	0.9912	0.9841	2,3,4,5
q16	4.58	1.0251	1.0021	0.9911	0.9822	2,3,4,5
q32	2.73	1.0297	1.0013	0.9893	0.9804	2,3,4,5
q48	1.94	1.0350	1.0004	0.9884	0.9771	2,3,4,5
q63	1.43	1.0494	0.9963	0.9829	0.9731	2,3,4,5

Table 30. The fitted separable model. α is the exponent relating a tile's coded bits to its difficulty and ϕ is the exit profile, both fitted leave-one-sequence-out. α is positive at every rate, so a tile the entropy coder spent bits on is a tile every exit reconstructs worse, and it falls sevenfold from the lowest rate to the highest. The profile spans only about 4% from its shallowest to its deepest entry, which is the margin the rule has to work in. All four exits survive on the profile's lower hull at every rate, so Proposition 14 costs nothing here.

results/raterank_RECIPE512_b01.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget. Hull membership computed against the shipped cost vector of Table 19. On the single-sequence table of results/tile_table.json the same construction drops exit 4 from the hull, so the property is not automatic.

q	rule %	oracle %	agree	ρ level	ρ spread	ρ exit
q0	27.79	29.90	0.458	0.832	0.609	0.537
q16	22.72	25.62	0.319	0.845	0.660	0.520
q32	18.24	20.93	0.280	0.857	0.711	0.531
q48	14.67	18.33	0.350	0.840	0.715	0.528
q63	11.99	15.64	0.390	0.882	0.684	0.384

Table 31. What the free rule achieves, and what the bit count carries. Savings at a 0.1 dB budget against the oracle's, and the fraction of tiles on which the two agree. The last three columns are Spearman correlations of the tile's coded bits with the mean distortion level across the ladder, with the spread from shallowest to deepest, and with the oracle's own exit index. Bits track the level well, the spread almost as well, and the exit least well, which is the ordering the separable model predicts.

results/raterank_RECIPE512_b01.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences. The file stores the last column as spearman_bits_vs_exit against the negated exit index; it is reported here with the sign flipped, so a positive value means more bits and a deeper exit.

So where does the rule lose its 6.9 points? Not in the separability. On the measured tile table the separable model absorbs 99.96% of the variance of log D, which by any ordinary standard is an excellent fit. The trouble is that the quantity it has to resolve is small. The whole exit profile spans 0.0449 in log units, while the non-separable residual has a root mean square of 0.0144, which is 32% of it. A model can explain almost all of a table and still be unable to order the differences that decide the argmin, because the argmin reads only the differences. Layered on that, predicting the tile's difficulty from its bit count alone leaves $R^2 = 0.84$, and by Proposition 13 an error in log g_t is a proportional error in the effective price that tile faces.

Residual analysis computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPE512/ckpt_eval.pth.tar; α fitted there is 4.69 against 2.73 for the 53-sequence fit at the same rate, the difference being one sequence against 53.

Two consequences are worth stating plainly. The rule is not a weaker router: it is the exact oracle for a decoder whose distortion table happens to be rank-1, and its shortfall measures how far from rank-1 the real table is. And it is the reason a partial signal is attractive even without a trained head, since Proposition 12 applies to any predictor whose regret can be computed by the encoder, this one included.

C.12. What is proved and what is checked

statement	proof	checked in	checkpoint
1 separation	C.4	verify_theory P1	eval
2 monotonicity	C.4	verify_theory P2	eval
3 fixed-proportion hull	C.6	static_RECIPE512_b01	PAPER
4 Minkowski average	C.6	not measured	-
5 hull, and interior unreachable	C.7	hull_gap	eval
6 lattice spacing	C.7	theory_checks	eval
7 floor	C.8	saturation_RECIPE512	PAPER
8 saturation	C.8	verify_theory P5	eval
9 ceiling	C.8	per_class_RECIPE512	PAPER
10 adaptivity gain	C.9	theory_check	BEST
11 log-convexity	C.10	logconvexity	none recorded
12 Lorenz bound	C.11	hybrid_RECIPE512_b01_fixed	PAPER
13 rank-1 threshold rule	C.12	raterank_RECIPE512_b01	PAPER
14 rank-1 uses hull exits	C.12	raterank_RECIPE512_b01	PAPER

Table 32. Every statement in this section, and its evidence. PAPER is runs/RECIPE512/ckpt_PAPER.pth.tar, eval is runs/RECIPE512/ckpt_eval.pth.tar, since overwritten by the per-epoch watcher, and BEST is runs/BEST/ckpt_eval.pth.tar. The seven propositions checked by scripts/verify_theory.py all pass, on one sequence at one rate. Theorem 4 is proved and not measured: it describes a set of 120 vertices that no experiment enumerates.

Files named without a directory are under results/ with a .json extension. verify_theory refers to scripts/verify_theory.py, whose output is results/theory_checks.json.

Three gaps in that table are worth naming rather than leaving to be noticed. Theorem 10, the one result the paper leans on hardest, is measured only on runs/BEST/ckpt_eval.pth.tar over 40 sequences and on the earlier cost vector; the pinned-checkpoint version in Table 26 measures the same quantity in saving points but not the scale-free ratio. The convexity check of Table 27 records no checkpoint at all. And the granularity and hull results of C.7 rest on a single sequence at a single rate, which is enough to check arithmetic and not enough to characterise a codec.

D. Complete results

This section prints in full the measurements the main paper quotes at one or two points. The per-class table there reports one budget at three of the five quality indices; here every class appears at every index and at three budgets. The frontier is then given at every budget measured, and the integrated figures that summarise it are given with the interval and the averaging convention each was taken under, because both move the number by more than the difference between the systems being compared.

D.1. Every class at every rate, at the 0.1 dB budget

Class	q0	q16	q32	q48	q63
UVG	30.6	28.9	23.1	18.7	15.5
MCL-JCV	33.7	30.4	26.2	23.0	20.2
HEVC B	31.1	25.8	18.2	12.4	9.7
HEVC E	29.1	25.2	18.4	15.1	15.5
HEVC C	18.0	6.3	1.7	2.8	2.5
HEVC D	8.8	2.5	2.5	2.5	2.5
All 53, pooled	29.7	25.5	20.9	17.9	15.6

Table 33. Compute saved against the released decoder, per class, at the 0.1 dB budget. The main paper prints the q0, q32 and q63 columns of this table. Two effects are visible and they compound: the saving falls with rate, from 29.7% to 15.6% on the pooled row, and it falls with decreasing resolution, from 33.7% on MCL-JCV to 8.8% on HEVC D at q0. At q63 both of the two smallest classes sit at 2.5%, which is one tile in four at the second-deepest exit and every other tile run to full depth.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at one frame each. This file extends results/per_class_RECIPE512.json from three rates to five; on the nine cells the two share, every saving, every decibel and every histogram bin is bit-identical, so the extension is a continuation of the same measurement and not a re-run. The pooled row sits under a tenth of a point from the main paper's 29.9% and 15.6%, which are the same budget measured over two frames per sequence rather than one.

Class	q0	q16	q32	q48	q63
UVG	0.101	0.108	0.098	0.096	0.095
MCL-JCV	0.116	0.113	0.115	0.111	0.108
HEVC B	0.080	0.078	0.072	0.070	0.067
HEVC E	0.105	0.120	0.120	0.108	0.118
HEVC C	0.062	0.052	0.062	0.068	0.083
HEVC D	0.028	0.040	0.047	0.065	0.090
All 53, pooled	0.099	0.099	0.100	0.098	0.100

Table 34. Quality actually given up, per class, at the same operating points. The budget is met on the set and on nothing else. At q0 the allocation spends 0.116 dB on MCL-JCV and 0.028 dB on HEVC D, a factor of four across classes at a single multiplier. The small classes are cheap in quality for the same reason they are poor in saving: with two or eight tiles there is no fine-grained way to spend the budget, so the bisection stops short of it.

Same file and same rows as the previous table. Decibels are the per-frame convention, averaged over the sequences of the class.

The resolution effect is a property of the tiling and not of the content. A 256 px tile is 40 tiles on a 1080p frame, 15 on 720p, eight at 832x480 and two at 416x240, so the allocation has two orders of magnitude fewer choices at the bottom of the range. It is also the argument against reading a single mean over a test set that mixes resolutions: the pooled row is dominated by MCL-JCV, which is 30 of the 53 sequences and all of

them 1080p.

D.2. Where the tiles exit

Rate	e2	e3	e4	e5	Blocks skipped	Saved
q0	67.4	20.7	8.3	3.6	5.04	29.7
q16	58.2	18.6	13.1	10.0	4.50	25.5
q32	43.5	19.5	18.8	18.2	3.77	20.9
q48	34.4	16.8	24.3	24.5	3.22	17.9
q63	22.3	23.1	28.1	26.6	2.82	15.6

Table 35. The exit histogram at the 0.1 dB budget, as a percentage of the 1765 tiles in one pass over the test set, with the mean number of trunk blocks skipped. At the lowest rate two thirds of tiles take the shallowest selectable exit and the ladder is barely used above it. At the highest rate the mass is spread almost evenly over the four exits and the mean depth skipped has fallen from 5.04 blocks to 2.82. The same budget buys less at high rate because the residual the shallow exits fail to reconstruct is larger relative to the quality being protected.

Histograms summed over the six classes from results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Blocks skipped per exit from results/adapter_cost.json.

Class	e2	e3	e4	e5	e2	e3	e4	e5
UVG	54.3	31.1	14.3	0.4	5.7	31.4	44.6	18.2
MCL-JCV	73.8	17.2	5.7	3.4	29.8	23.4	23.5	23.3
HEVC B	64.0	18.5	12.5	5.0	6.5	11.0	37.5	45.0
HEVC E	51.1	35.6	4.4	8.9	15.6	35.6	8.9	40.0
HEVC C	3.1	53.1	31.2	12.5	0.0	0.0	25.0	75.0
HEVC D	0.0	25.0	25.0	50.0	0.0	0.0	25.0	75.0

Table 36. The same histogram split by class, at the two ends of the rate range: the first four columns are q0 and the second four are q63, each as a percentage of that class's tiles. HEVC C and HEVC D never reach the shallowest exit at q63; three quarters of their tiles run to full depth, which is the 2.5% saving of the previous table seen tile by tile.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

D.3. The 0.3 dB and 0.5 dB budgets

Budget	Class	q0	q16	q32	q48	q63
0.3 dB	UVG	39.1	39.1	39.1	38.9	38.7
0.3 dB	MCL-JCV	39.1	39.1	39.1	38.8	38.1
0.3 dB	HEVC B	39.1	39.1	39.1	38.3	36.2
0.3 dB	HEVC E	39.1	39.1	39.1	39.1	38.8
0.3 dB	HEVC C	39.1	39.1	39.1	33.6	28.8
0.3 dB	HEVC D	39.1	39.1	39.1	31.7	16.9
0.5 dB	UVG	39.1	39.1	39.1	39.1	39.1
0.5 dB	MCL-JCV	39.1	39.1	39.1	39.1	39.1
0.5 dB	HEVC B	39.1	39.1	39.1	39.1	39.1
0.5 dB	HEVC E	39.1	39.1	39.1	39.1	39.1
0.5 dB	HEVC C	39.1	39.1	39.1	39.1	39.1
0.5 dB	HEVC D	39.1	39.1	39.1	39.1	39.1

Table 37. Compute saved per class at the two looser budgets. Almost every cell is 39.1%, which is the architectural ceiling: the largest saving the ladder can reach, obtained by putting every tile at the shallowest selectable exit. Where a cell is below the ceiling the budget is still binding and the allocation is still making a choice; that happens at 0.3 dB only at q48 and q63.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. The ceiling is 39.1% from results/saturation_RECIPE512_ctc53.json, same checkpoint, which records it as 100(1 minus the cost of exit e2) with that cost measured at 0.6088 of a released decode.

Budget	Class	q0	q16	q32	q48	q63
0.3 dB	UVG	0.200	0.218	0.253	0.290	0.320
0.3 dB	MCL-JCV	0.173	0.202	0.260	0.279	0.299
0.3 dB	HEVC B	0.153	0.191	0.245	0.282	0.272
0.3 dB	HEVC E	0.226	0.305	0.393	0.408	0.380
0.3 dB	HEVC C	0.191	0.260	0.359	0.335	0.322
0.3 dB	HEVC D	0.174	0.279	0.383	0.375	0.221
0.5 dB	UVG	0.200	0.218	0.253	0.298	0.338
0.5 dB	MCL-JCV	0.173	0.202	0.260	0.296	0.346
0.5 dB	HEVC B	0.153	0.191	0.245	0.308	0.365
0.5 dB	HEVC E	0.226	0.305	0.393	0.408	0.400
0.5 dB	HEVC C	0.191	0.260	0.359	0.462	0.573
0.5 dB	HEVC D	0.174	0.279	0.383	0.516	0.727

Table 38. Quality given up at the two looser budgets. In the saturated cells the delivered decibel is well under the budget, because once every tile is at the shallowest exit there is nothing further to spend and the multiplier has run to the top of its bracket. The direction reverses at the bottom of the resolution range: a 0.5 dB set budget costs 0.727 dB on HEVC D at q63 and 0.573 dB on HEVC C, so the small classes overshoot a budget that the set as a whole undershoots.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Where the bisection saturates it returns the top of its bracket, $\lambda = 1$, which is the marker for a budget the ladder cannot reach rather than a price that was chosen.

Saturation is worth stating as a property of the design rather than of these two budgets. The set-level decibel at which every tile arrives at the shallowest exit is 0.179 dB at q0 and 0.396 dB at q63, so a budget above that number buys nothing further at any rate, and a budget below 0.036 dB at q0 or 0.072 dB at q63 cannot be met at all, because the deepest exit already differs from the released decoder by that much. The 0.1 dB budget sits inside that window at every rate, which is why it is the one the main paper reports.

D.4. Per-sequence spread

A set mean is not what any single clip receives. Because one multiplier prices compute for the whole set, each sequence lands wherever its own content puts it, and the spread of those landings is the number a deployment would care about.

Rate	n	Mean	SD	Min	p25	Median	p75	Max
q0	40	39.0	4.0	28.1	36.6	40.3	42.1	42.5
q16	40	36.2	5.3	23.5	33.7	37.0	40.6	42.5
q32	40	32.4	6.5	18.6	28.2	32.9	37.4	42.1
q48	40	29.5	6.9	12.2	24.8	30.1	35.3	42.5
q63	40	26.4	6.5	11.1	22.4	26.3	30.0	42.5

Table 39. Compute saved per sequence at the 0.1 dB budget: the distribution behind the mean. The interquartile range is 5.5 points wide at q0 and 7.6 at q63, and the gap between best and worst sequence is 14.4 points at q0 and 31.4 at q63. Quoting the mean alone would promise the hardest content something it does not get.

results/per_sequence.json. Not the pinned checkpoint: the file records runs/BEST/ckpt_eval.pth.tar, 40 sequences. Those 40 are UVG, MCL-JCV and HEVC E only, so the three classes where the saving collapses are absent and the minimum column is not the test set's minimum. The file also names results/curve_BEST.json as its source, and that file has since been regenerated with 53 sequences and different per-sequence values, so this table cannot be reproduced from anything now on disk. The next table asks the same question on a file that can.

Rate	n	Mean	SD	Min	p25	Median	p75	Max	Under 10
q0	53	35.7	6.0	12.5	34.6	38.7	39.7	39.7	0
q16	53	32.1	9.0	0.0	28.9	35.8	39.1	39.7	1
q32	53	27.7	11.5	0.0	22.8	31.1	36.7	39.7	5
q48	53	25.3	11.6	0.0	17.4	27.9	35.4	39.7	7
q63	53	22.8	11.4	0.0	14.7	24.2	32.0	39.7	8

Table 40. The same question over all 53 sequences, including the classes the previous table omits. The last column counts sequences saving under 10%. At q0 there are none; at q63 there are seven, and four of those save nothing at all, which against the released decoder is minus 0.95% because our deepest exit is the more expensive of the two. The standard deviation rises from 6.7 to 12.2 points across the rate range, so the saving at high rate is not only smaller but harder to predict.

Recomputed here from the per-sequence arrays in results/curve_RECIPE512.json at its 0.1 dB operating point. Not the pinned checkpoint: that file records runs/RECIPE512/ckpt_eval.pth.tar at epoch 1, and it was measured under an earlier compute-cost model whose ceiling is 42.5% rather than 39.1%, so the levels are not comparable with the per-class tables above. The shape of the distribution is what this table is for.

Sequence	q0	q63	q63 vs release	q63 dB
RaceHorses_832x480_30	25.9	0.0	-1.0	0.076
BlowingBubbles_416x240_50	26.8	0.0	-1.0	0.066
BQSquare_416x240_60	12.5	0.0	-1.0	0.162
RaceHorses_416x240_30	19.4	0.0	-1.0	0.070
PartyScene_832x480_50	22.2	1.7	0.8	0.107
BQMall_832x480_60	28.6	6.6	5.7	0.095
videoSRC21_1920x1080_24	39.7	39.7	39.1	0.207
videoSRC29_1920x1080_24	39.7	39.7	39.1	0.259

Table 41. The six hardest sequences at q63 and the two easiest, with their q0 value for comparison and the decibel each actually received. All six of the hard ones are 832x480 or 416x240. The delivered decibel is not the budget on any of them: across the 53 sequences at q63 it runs from 0.002 dB to 0.248 dB against a 0.1 dB target, so one multiplier for the set means individual clips overshoot the budget by a factor of two while others use a fiftieth of it.

results/curve_RECIPE512.json, operating point 0.1 dB, on runs/RECIPE512/ckpt_eval.pth.tar at epoch 1 rather than the pinned checkpoint. The q63 saving against the release is the same allocation counted against the released decoder rather than against our deepest exit.

D.5. Integrated figures

A budget is one sample of a curve, and a conclusion drawn from one sample is fragile. Video coding answers this for rate against quality with Bjontegaard's construction [4], integrating one axis over a stated interval of the other, and the same construction applies with compute in place of rate. Two integrals are reported: BD-saving, the mean compute saved over an interval of decibels, and BD-quality, the mean decibel paid over an interval of saving. Neither means anything without its interval, and both are given with theirs.

Configuration	Budget	BD-Rate	MACs saved	Map bits
A signalled	0.1 dB	0.88%	22.1%	79
A signalled	0.3 dB	2.38%	38.2%	79
A signalled	0.5 dB	2.62%	39.1%	79
B router	0.1 dB	0.85%	19.8%	0
B router	0.3 dB	2.35%	38.4%	0
B router	0.5 dB	2.59%	41.7%	0

Table 42. BD-Rate cost of the two configurations at three budgets. This is the rate a reader would have to spend to buy back the quality the compute saving costs, integrated over the five rate points, and it is the number that makes the trade-off comparable with a published codec result. Configuration A signals the exit map in the bitstream and pays about 80 bits a frame for it; configuration B infers it and pays none. Their BD-Rate costs differ by at most 0.03 points at every budget, so the map is close to free, and at the 0.1 dB budget A converts that into four more points of compute saved.

results/bdrate.json. The file records no checkpoint. It was written before the current results/signalled_RECIPE512_ctc53.json and its saving column is on the superseded compute-cost model: it reads 41.9% at 0.5 dB where the pinned file now saturates at 39.1%. The BD-Rate column is an integral of rate against quality and does not depend on that model; the saving column does.

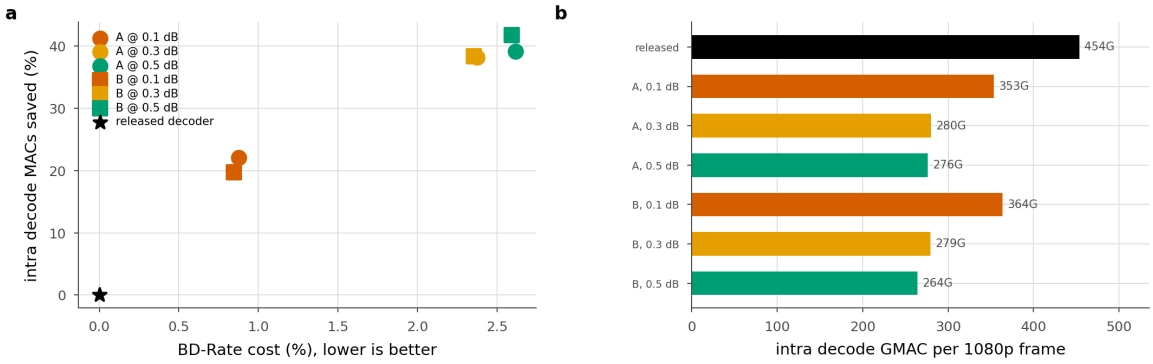

Figure 4. The same measurement drawn. Left, BD-Rate cost against compute saved, with the released decoder at the origin: the two configurations sit almost on top of each other in BD-Rate, and loosening the budget moves both up and to the right together. Right, decode cost per 1080p frame in GMAC. Rendered from the same run as the previous table, so the compute axis carries the same superseded cost model and the BD-Rate axis does not.

docs/figures/paper_rdc.png, written by scripts/paper_metrics.py in the same pass that wrote results/bdrate.json.

Rate	BD-saving	BD-quality	Floor	Points
q0	n/a	0.051	0.033	15
q16	n/a	0.065	0.043	14
q32	n/a	0.081	0.053	13
q48	33.3%	0.093	0.060	13
q63	30.7%	0.106	0.066	13
Mean of the two that span it	32.0%	0.099		

Table 43. BD-saving and BD-quality per rate, integrated over 0.066 to 0.3 dB and over 10% to 30% saving respectively. Three of the five rates return no BD-saving at all, because their measured frontier does not span the interval: the floor at q0 is 0.033 dB but the sweep does not reach 0.3 dB before saturating. Reporting a mean over the two rates that do span it, as the file does, is a mean over the two hardest rates and reads higher than a mean over five would.

results/bd_RECIPE512_ctc53.json, on runs/RECIPE512/ckpt_eval.pth.tar at epoch 0 rather than the pinned checkpoint, 53 sequences, per-frame convention. Points is the number of measured frontier points inside the interval.

dB interval	Mean	q0	q63	q0 minus q63
0.064 to 0.20	22.8%	31.3%	14.4%	16.8 pt
0.064 to 0.25	25.1%	33.3%	16.9%	16.4 pt
0.064 to 0.30	27.0%	35.0%	19.1%	15.9 pt
0.084 to 0.30	28.2%	35.9%	20.4%	15.6 pt
0.104 to 0.30	29.2%	36.8%	21.5%	15.3 pt
0.104 to 0.25	27.4%	35.3%	19.6%	15.7 pt

Table 44. What the interval does to the answer. The level moves by 6.3 points across six defensible intervals, which is why an interval is quoted beside every integrated number in this work. The rate effect does not move: the lowest rate exceeds the highest by 15.3 to 16.8 points on every interval tried, so the statement that the saving falls by roughly fifteen points across the quality range is about the decoder and not about the integration.

results/bd_sensitivity.json. The file records the convention but not the curve; scripts/bd_sensitivity.py names it as results/paper_curve_grid128.json, which is runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar at 128 px tiles over 40 sequences. That is neither the pinned checkpoint nor the tile size this work reports, so the levels here are not the levels elsewhere in this section; the robustness conclusion is what it is for. Intervals wide enough that a rate's frontier does not span them are excluded rather than averaged over fewer rates.

Rate	Per frame	Pooled	Difference
q0	31.1%	38.4%	7.3 pt
q16	28.0%	36.0%	8.0 pt
q32	22.0%	30.9%	8.9 pt
q48	17.7%	27.3%	9.6 pt
q63	14.2%	23.6%	9.4 pt
Mean	22.6%	31.2%	8.6 pt

Table 45. The averaging convention costs more than the systems being compared differ by. The two columns are the same allocation on the same curve, integrated under the two decibel conventions, and they disagree by 7.3 to 9.6 points of BD-saving. Part of that is the interval, which differs between the two because the per-frame convention raises every floor and so moves the lower limit the integration can start from; the two effects cannot be separated from these two files. Either way, no comparison across papers is meaningful unless both state which convention they average under.

results/bd_saving.json, per-frame convention over 0.064 to 0.196 dB, against results/bd_saving_pooled.json, pooled convention over 0.057 to 0.300 dB. Both on runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar over 40 sequences, which is neither the pinned checkpoint nor 256 px tiles.

D.6. What this section does not measure

Four gaps are worth naming so that they are visibly absent rather than quietly missing.

- No integrated figure in this work is on the pinned checkpoint. results/bdrate.json records no checkpoint and predates the current compute-cost model, results/bd_RECIPE512_ctc53.json is the epoch-0 evaluation checkpoint, and the interval and convention studies are on a 128 px run over 40 sequences. The per-class and frontier tables above are pinned; the BD tables are not.
- No per-sequence measurement is on the pinned checkpoint either. results/per_sequence.json is a second training run over a 40-sequence subset and cannot be regenerated from the curve it names, and results/curve_RECIPE512.json is the epoch-1 evaluation checkpoint under a superseded cost model.
- The 0.1 dB budget is a convention this work adopts and not a perceptual threshold. Subjective work measures the smallest noticeable change in quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so no published result licenses a claim that 0.1 dB is invisible. What it buys is a number a codec reader can compare, namely 0.88% of BD-Rate, and 0.3 dB and 0.5 dB are reported beside it so that nothing rests on the choice.
- Every measurement here is on intra frames of video sequences, one frame per sequence except in the frontier grid, which uses two. Nothing here measures the inter-frame path, and nothing here measures a second checkpoint, so the class and sequence effects are reported without a replication.

E. The quality budget and its working range

A budget D is the peak-signal-to-noise ratio the decode is allowed to give up, and an allocation is admissible if the loss it incurs stays at or below it. Everything the paper reports is one point on a grid of such budgets. This section gives the grid whole, together with the two ends of the range over which a budget changes anything, and the rescaling that removes most of the rate dependence from the trade-off.

E.1. Two decibel conventions

Two separate choices have to be pinned before a decibel figure means anything, and the project makes both choices in two ways in different files. The first is what the tiled decode is compared against. Under the *release-referenced* convention it is compared against the released decoder's full-frame decode of the same latent, so the cost of cutting the frame into tiles at all sits inside the number. Under the *self-referenced* convention it is compared against this model's own deepest exit decoded tile by tile, so the tiling cost cancels and the budget pays only for early exiting.

The two conventions give the ceiling two values. With c_k the cost of exit k in units of one released decode, j the split depth and K the number of exits,

$$S_{\max}^{\mathrm{rel}} = 100\,(1 - c_j), \quad S_{\max}^{\mathrm{self}} = 100\,(1 - c_j / c_{K-1}) \tag{22}$$

With c_j = 0.6088, recorded in results/saturation_RECIPE512_ctc53.json, and c_(K–1) = 1.0095, recorded as deepest_exit_cost in results/router_RECIPE512_b01.json, the first reads 39.1% and the second 39.70%. The deepest exit costs more than one released decode because it carries the deblocking filter, which the released decoder does not run, and that difference is the whole of the gap between the two ceilings.

The second choice is how decibels are averaged over frames. Table 46 gives the same allocation read both ways: a per-frame mean of the per-frame loss, and one loss computed from mean-squared errors pooled over all frames first. Pooling is the flattering reading at every rate, and the gap is between a quarter and a third of a 0.1 dB budget. The project reports the per-frame convention throughout.

q	per-frame dB	pooled dB	difference	saving %
0	0.1000	0.0669	-0.0331	28.29
16	0.1000	0.0670	-0.0330	24.78
32	0.1000	0.0741	-0.0259	18.73
48	0.1000	0.0746	-0.0254	14.34
63	0.1000	0.0767	-0.0233	10.82

Table 46. The averaging convention is worth more than the third digit of the budget. One allocation targeted at 0.1 dB, read two ways. Pooling the squared errors before taking the logarithm reports 0.023 to 0.033 dB less loss than the per-frame mean of the same decodes, so a result quoted without its convention is uncertain by about a third of the budget it claims to meet.

results/db_convention.json. This is the one table in this section not measured on the pinned checkpoint: it is runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar over 40 frames, an early 128 px run. The two conventions are definitions rather than properties of a checkpoint, so the size of the gap is what transfers, not the digits.

Nothing in the published literature fixes 0.1 dB as a tolerance. Subjective work measures the smallest noticeable change in quantisation parameter or in a video-quality metric rather than in tenths of a decibel of peak signal-to-noise ratio, and one of our own test sets, MCL-JCV [26], is a just-noticeable-difference dataset built that way. We treat 0.1 dB as a reporting convention, report 0.3 dB and 0.5 dB beside it in the main paper, and give the whole grid here so that nothing in the argument rests on the choice.

E.2. The floor

The floor D_min is the loss of a tiled decode with every tile at full depth. No early exiting has happened, so it is pure tiling penalty: the border of every tile has been convolved against padding instead of against its neighbour. It is measured by one forward pass per rate, with the exit map held at K − 1, against the released decoder's full-frame decode of the same latent.

A budget below the floor admits no allocation at all. This is a different failure from the budget falling in a gap of the Lagrangian hull, where the sweep returns the nearest reachable point and Everett's argument still makes it optimal for the compute it consumes [28, 29]. Below the floor the feasible set is empty and there is nothing to return. Three cells of the release-referenced grid are in that state and are recorded as such: at a 0.05 dB budget, results/signalled_RECIPE512_grid.json carries budget_reachable false at q32, q48 and q63, whose floors there are 0.053, 0.060 and 0.066 dB.

q	floor D_min	saturation D_sat	band	0.1 dB at
0	0.036	0.179	0.143	45%
8	0.040	0.194	0.154	39%
16	0.045	0.219	0.174	31%
24	0.050	0.251	0.201	25%
32	0.054	0.282	0.228	20%
40	0.059	0.305	0.246	17%
48	0.062	0.333	0.271	14%
56	0.066	0.365	0.299	11%
63	0.072	0.396	0.324	9%

Table 47. The working range, at all nine measured rates. The main paper's operating table shows five of these columns. Both ends widen with the rate and the band widens faster than the floor rises, so the 0.1 dB budget that sits 45% of the way up the band at q0 sits 9% of the way up at q63. That single quantity, and not the rate, is what the rescaling of Section E.5 turns out to need.

results/saturation_RECIPE512_ctc53.json, 53 CTC frames on the pinned checkpoint. The band and the position of 0.1 dB in it are arithmetic on the two measured columns.

The floor also appears in results/signalled_RECIPE512_grid.json, measured over 106 frames rather than 53, and the two estimates differ by 3 to 6 thousandths of a decibel: 0.033 against 0.036 dB at q0, and 0.066 against 0.072 dB at q63. The rescaling below uses the saturation file's values for both ends, so the two ends are on one measurement.

E.3. The saturation point

The saturation point D_sat is the loss when every tile takes exit j, the shallowest the split depth allows. Saving is then pinned at the ceiling and a looser budget buys nothing, so it is not a quantity to search for: it is one forward pass with the exit map held at j. Both ends of the band are therefore closed-form or one-pass quantities, which is what makes the rescaling cheap enough to be worth doing in deployment.

Table 48 checks that claim against a measurement that never sets an exit map by hand. Sweeping the Lagrange multiplier upward, the achieved loss stops rising at a value that agrees with the one-pass saturation point to within 1.5 thousandths of a decibel at all five rates, and the saving there is the architectural ceiling to four decimal places.

q	D_sat, one pass	grid stops at	difference	saving there
0	0.1791	0.1785	-0.6 mdB	39.1246
16	0.2194	0.2205	+1.1 mdB	39.1246
32	0.2819	0.2834	+1.5 mdB	39.1246
48	0.3330	0.3341	+1.1 mdB	39.1246
63	0.3955	0.3950	-0.5 mdB	39.1246

Table 48. Two independent routes to the same saturation point. The middle column is the largest loss any budget in the release-referenced grid actually incurs; past that point a looser budget changes neither the loss nor the saving. The two routes share a checkpoint and a test set but no code path.

results/saturation_RECIPE512_ctc53.json (53 frames) and results/signalled_RECIPE512_grid.json (106 frames), both on the pinned checkpoint. The difference column is arithmetic on the two.

One consequence is narrow enough to be worth stating. Budgets in [0.179, 0.194) dB saturate the lowest rate and no other, a window 15 thousandths of a decibel wide. Above 0.396 dB every rate has saturated and the ladder, not the budget, is the binding constraint.

E.4. The grid

Table 49 is the main grid: eight budgets across five rates, on the pinned checkpoint, one frame from each of the 53 test sequences. It is measured in the self-referenced convention, so its ceiling is 39.70% rather than 39.1%.

budget dB	q0	q16	q32	q48	q63
0.05	23.9	21.4	20.0	18.0	18.2
0.10	31.3	29.1	28.0	26.2	24.8
0.15	34.7	33.3	32.7	31.2	29.2
0.20	36.8	35.8	35.3	34.2	32.5
0.30	38.6	38.0	37.9	37.4	36.0
0.50	39.6	39.5	39.5	39.3	38.6
0.75	39.7	39.7	39.7	39.6	39.4
1.00	39.7	39.7	39.7	39.7	39.6

Table 49. Compute saved against budget, the whole grid. Oracle allocation, so an upper bound on any router. Reading down a column, the return on a looser budget falls away long before the ceiling is reached: at q0 the first 0.05 dB buys 23.9 points and the next 0.95 dB buys 15.8 more. Reading across a row, the rate dependence is 5.9 points at 0.05 dB and vanishes entirely once every rate has saturated.

results/supp_budget_grid.json, pinned checkpoint, one frame per sequence, measured by scripts/budget_table.py on an uncontended card.

budget dB	q0	q16	q32	q48	q63
0.05	0.049	0.049	0.050	0.049	0.049
0.10	0.098	0.099	0.099	0.099	0.099
0.15	0.149	0.149	0.149	0.150	0.148
0.20	0.200	0.200	0.200	0.198	0.199
0.30	0.300	0.299	0.298	0.300	0.299
0.50	0.496	0.495	0.495	0.499	0.498
0.75	0.522	0.649	0.682	0.742	0.742
1.00	0.522	0.649	0.799	0.893	0.976

Table 50. What the same allocations actually spend. Up to 0.3 dB the achieved loss tracks the budget to the third decimal, which is the bisection working. Above it the entries stop moving, and at q0 they stop at 0.522 dB however loose the budget gets: there is no allocation left that spends more, which is saturation seen from the other side.

results/supp_budget_grid.json. The entries above 0.3 dB are an upper bound on the loss rather than the loss a deployed decoder would incur; see the paragraph below.

Two properties of that grid need stating rather than leaving to be discovered. Its exit histogram has mass in the two columns below the split depth, which the deployed decoder cannot use: forward() clamps any assignment below j to j, and flexuf/cost.py applies the same clamp, which is why the ceiling is exactly 39.70% and not something larger. Those tiles are billed at exit j's cost, correctly, but the loss recorded for them is the shallower exit's, which is worse than what the clamped decode would produce. The effect is negligible at 0.05 dB, where 93 of 1,765 tiles are nominally below j, and large at 1.0 dB, where most of them are. Saving is unaffected at every budget; the achieved-loss column is pessimistic above about 0.3 dB.

budget dB	q0	q16	q32	q48	q63
0.050	16.4	9.3	none	none	none
0.075	24.3	19.7	14.6	11.1	7.6
0.100	29.8	25.5	20.7	18.2	15.6
0.150	36.6	33.0	28.6	26.5	24.1
0.200	39.1	37.8	33.7	31.6	29.6
0.250	39.1	39.1	37.4	35.2	33.0
0.300	39.1	39.1	39.1	37.9	35.8
0.400	39.1	39.1	39.1	39.1	39.1
0.500	39.1	39.1	39.1	39.1	39.1

Table 51. The same sweep in the release-referenced convention. Nine budgets, five rates, two frames from each of the 53 sequences. *none* marks a budget below that rate's floor, where no allocation is admissible. The 0.1 dB row is the main paper's headline: 29.9% at q0, 15.6% at q63, 22.1% averaged over the five rates. Every entry at 0.4 dB and above has saturated at 39.1%; at 0.3 dB all but q48 and q63 have.

results/signalled_RECIPE512_grid.json, pinned checkpoint, two frames per sequence. The budget_reachable flag is recorded in the file, not inferred here.

Table 52 puts the two conventions side by side at the budget the paper leads with. They disagree by 1.5 points at q0 and 9.3 points at q63, and the disagreement is very nearly the floor: the release-referenced number has to pay the tiling penalty out of the same 0.1 dB, and that penalty grows with the rate. Neither number is wrong. A reader comparing against a full-frame codec wants the first; a reader asking what early exiting alone contributes wants the second.

q	self-ref.	release-ref.	difference	floor dB
0	31.27	29.81	1.46	0.033
16	29.12	25.48	3.64	0.043
32	28.01	20.72	7.29	0.053
48	26.20	18.20	8.00	0.060
63	24.85	15.59	9.25	0.066

Table 52. The 0.1 dB column under both conventions. The difference between them tracks the floor, which is what the release-referenced number spends before it has saved anything. A saving figure quoted without its convention is ambiguous by more than the gap between the two configurations the main paper compares.

results/supp_budget_grid.json and results/signalled_RECIPE512_grid.json, both pinned, one and two frames per sequence respectively. The difference column is arithmetic on the two.

E.5. Rescaling onto the band

With both ends of the range measured, a budget can be quoted as a position in the range rather than as a decibel figure. Write

$$u = \frac{D - D_{\min}}{D_{\text{sat}} - D_{\min}} \tag{23}$$

so that u = 0 is the floor and u = 1 the saturation point. At a matched decibel the five rates are 15.5 points apart in saving. At a matched u they are 1.7 points apart on average and 2.2 at worst, so the band accounts for most of the rate dependence of the trade-off and what is left over is a couple of points.

u	q0	q16	q32	q48	q63	spread
0.1	16.5	14.6	15.1	15.2	16.3	1.91
0.2	21.0	20.9	20.7	20.9	21.8	1.13
0.3	25.2	24.9	24.3	25.5	26.2	1.91
0.4	28.3	27.7	27.9	28.6	29.7	1.96
0.5	30.8	30.4	30.4	31.4	31.9	1.54
0.6	32.8	33.0	32.8	33.4	33.9	1.13
0.7	34.7	34.7	34.7	35.3	35.7	1.00

Table 53. The five curves at matched positions in their own bands. Read across a row: five rates whose raw savings at a matched decibel differ by 15.5 points agree here to about a point and a half. The spread is largest at the bottom of the band, where the floor subtraction is a large fraction of a small number and the two files disagree about the floor by a few thousandths of a decibel.

Computed in the build from results/signalled_RECIPE512_grid.json and results/saturation_RECIPE512_ctc53.json by the rescaling in scripts/tradeoff_figure.py, linearly interpolated between measured points. Every rate has measured points spanning u = 0.10 to 0.70, so no entry is an extrapolation. Mean spread over these seven positions 1.51 points, worst 1.96.

The collapsed curve is described by one free number. With C the architectural ceiling, which is not fitted,

$$S(u) \approx C\,u^{\beta} \tag{24}$$

and fitting ln(S/C) against ln u by least squares through the origin gives 0.38 on the pinned checkpoint, with $R^2 = 0.989$ and a worst residual of 2.0 points. On the second training run the same procedure gives 0.33 with $R^2 = 0.984$. Table 54 gives both fits in full, and Figure 5 shows the second one as a picture.

	pinned	BEST
source grid	signalled_RECIPE512	signalled_BEST
checkpoint	ckpt_PAPER	ckpt_eval, epoch 1
rates	5	5
points fitted	26	31
ceiling C used	41.91	41.91
exponent β	0.3767	0.3253
R^2	0.9890	0.9843
worst residual	2.03	1.89
mean residual	0.90	0.95
raw spread at 0.1 dB	15.50	16.92
spread after rescaling, mean	1.70	1.56
spread after rescaling, worst	5.85	6.81
worst excluding first point	2.16	3.17

Table 54. The one-parameter fit, on both checkpoints. The exponent is the only fitted quantity; the ceiling is architectural. Both fits describe their own data well, and they describe it with different exponents. The two ceilings differ because the BEST files predate the correction to the adapter cost, and the row is kept in the table rather than harmonised because the exponent depends on it.

results/band_collapse.json and results/band_collapse_BEST.json. Neither file records a checkpoint; the rows above name the checkpoint of the grid each was fitted to, results/signalled_RECIPE512_grid.json and results/signalled_BEST_grid.json. The BEST grid is runs/BEST/ckpt_eval.pth.tar at epoch 1, a path the per-epoch watcher has since overwritten.

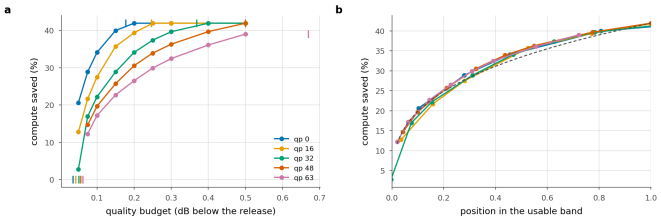

Figure 5. The collapse on the second training run. a Saving against budget, one line per rate, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own band; dashed is the one-parameter power law. The main paper shows this pair for the pinned checkpoint; this is the replication.

docs/figures/budget_band_BEST.png, drawn by scripts/tradeoff_figure.py from results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json.

E.6. How weak the replication is

The main paper says the collapse replicates and the exponent does not. Two checkpoints is a thin basis for either half of that sentence, and the arithmetic below is meant to show how thin.

q	β, pinned	R^2	β, BEST	R^2
0	0.368	0.9957	0.303	0.9905
16	0.393	0.9966	0.351	0.9854
32	0.402	0.9930	0.337	0.9824
48	0.395	0.9857	0.321	0.9855
63	0.359	0.9883	0.315	0.9908

Table 55. The exponent fitted to one rate at a time. Within a checkpoint the five rates want exponents spanning 0.043 on the pinned run and 0.048 on BEST. The difference between the two pooled exponents is 0.054, which is the same size. A quantity that varies this much across the rates of one checkpoint cannot be shown to differ between two checkpoints by measuring it twice.

Computed in the build from results/signalled_RECIPE512_grid.json with results/saturation_RECIPE512_ctc53.json, and results/signalled_BEST_grid.json with results/saturation_BEST_ctc53.json, by the fit in scripts/tradeoff_figure.py applied one rate at a time.

The exponent also moves with the ceiling constant, which is not fitted but assumed. Holding the pinned grid fixed and changing C from 39.1 to 41.91, the constant the pre-correction cost model gave, moves the pooled exponent from 0.379 to 0.409. That shift is of the same order as the difference between the two checkpoints, so the comparison the main paper draws between 0.38 and 0.33 is partly a comparison between two cost models.

One file needs regenerating. Re-running the fit on the files results/band_collapse.json names as its inputs, as they stand now, gives β = 0.3792 with $R^2 = 0.9895$, against the 0.3767 and 0.9890 the file records; both of its inputs were rewritten after it was written, and it still carries the pre-correction ceiling. The two digits the paper quotes, 0.38 and 0.989, survive the regeneration unchanged, so nothing in the main text moves, but the file should be rebuilt before submission. results/band_collapse_BEST.json reproduces exactly from its inputs: 0.3253 and 0.9843 against the 0.3253 and 0.9843 recorded.

A second description of the same frontier disagrees with the first. results/frontier_law.json fits the inverse relation, budget as a function of saving, in the form D = f + aS^b with the floor f free rather than measured. If both descriptions were exact, b would be 1/β. Table 56 shows that it is at the lowest rate and is not at the other four.

q	b, inverse fit	R^2	1/β, band fit
0	3.06	0.988	3.30
16	2.33	0.996	2.85
32	2.18	0.990	2.96
48	2.21	0.982	3.11
63	2.35	0.977	3.18

Table 56. Two power laws for one frontier. Both fits report R^2 above 0.97 on their own data and they imply exponents that differ by up to 40% at the middle rates. The inverse fit gives its floor three free parameters where the band fit measures the floor and fits one, so the agreement at q0 and the disagreement elsewhere both belong to the fitting, not to the decoder.

results/frontier_law.json, fitted to results/curve_BEST.json on runs/BEST/ckpt_eval.pth.tar over 40 sequences. The last column is computed in the build from results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json, so both columns describe the same training run but not the same sweep.

What the grids in this section do establish, and what they leave open:

- The floor and the saturation point are measured, cheap and unambiguous. Each is one forward pass per rate, both agree with an independent sweep, and the ceiling between them is closed form.
- The collapse is a strong empirical regularity on both checkpoints. Rescaling takes the spread across rates from 15.5 points to under two.
- The exponent is a description and not a law. It varies across rates within a checkpoint by as much as it varies between the two checkpoints, it moves when the assumed ceiling moves, and an inverse fit to the same frontier implies a different value.

- Everything here is the oracle allocation, obtained by sweeping the Lagrange multiplier with the per-tile losses known. It is an upper bound on any router and says nothing about how much of the band a decoder-side predictor can reach.
- The grids are intra frames at 1080p on the CTC set. Nothing here is measured on inter frames, and the resolution dependence of the band is not measured at all.

F. The router

An exit map assigns one exit index to every tile of a frame. This work produces such a map in three ways. Configuration A searches all K exits for every tile at the encoder and signals the answer. Configuration B predicts it at the decoder, from data the decoder already holds, and signals nothing. A third way computes it at the decoder from one number the entropy coder has already produced, with nothing learned at all. This section is about the second and the third.

Write D(t,k) for the mean squared error of tile t decoded at exit k, and c_k for the cost of exit k in units of one released decode. At a price λ on compute, the oracle gives each tile the exit that minimises the per-tile Lagrangian.

$$\ell(t,k) = D(t,k) + \lambda c_k \tag{25}$$

$$k^*(t) = \arg\min_{k \geq j} \ell(t,k) \tag{26}$$

The split depth j is the number of trunk groups every tile runs before any tile may leave, so exits below j do not exist and the minimisation starts there. λ is bisected once per rate so that the resulting map lands on the quality budget. *Agreement* below always means the fraction of tiles on which a predictor picks the same exit as this oracle at the same λ. It is the quantity the head is trained on, and part of what this section reports is that it is not the quantity that matters.

F.1. What the head is trained to do

The decoder is frozen throughout. The objective has two terms: a cross-entropy to the oracle's own choice, weighted per tile, and the expected regret against that choice.

$$\mathcal{L} = \frac{1}{|T|} \sum_{t \in T} w_t \, \mathrm{CE}(z(t), k^*(t)) + \alpha_r R \tag{27}$$

$$w_t = \frac{s_t}{\bar{s}}, \quad s_t = \max_k \ell(t,k) - \min_k \ell(t,k) \tag{28}$$

$$R = \frac{1}{|T|} \sum_{t \in T} \sum_k P_k(t) \left[\ell(t,k) - \min_{k'} \ell(t,k') \right] \tag{29}$$

P(t) is the softmax of the scores z(t), and α_r is 1 in every run reported here. The weight w_t is the spread between the best and the worst option on that tile, normalised to unit mean, so a tile where two exits are within a hair of each other counts for little and a tile where the choice is most of the frame's error counts for much. R is non-negative, is zero exactly when all the mass sits on k*(t), and its value is the excess Lagrangian cost the head is paying against the oracle, in the units of the frontier. It is evaluated on a hard Gumbel straight-through sample [10], so training decides one exit exactly as inference does while gradients still reach the probabilities.

The lineage is ClassSR's routing losses [12], with two of its three terms dropped for stated reasons. Its Class-Loss exists to repair the mismatch between a soft blend in training and an argmax at inference, and the straight-through sample removes that mismatch at source, so the repair has nothing left to do. Its Average-Loss exists because ClassSR's branches are separate networks that receive no gradient when unused; our exits share the trunk, and the trunk is frozen here, so an unused exit degrades nothing and forcing usage would mean routing tiles to exits the objective says are wrong.

- AdamW at learning rate 0.001 on a cosine schedule to zero, weight decay 10^{-4}, gradients clipped at unit norm (scripts/train_router2.py).
- 3,000 steps at a batch of 6 crops of 512×512 from OpenImages [24], with one quality index drawn uniformly per image.
- One λ for all rates, 1.3×10^{-5}, which puts the oracle near the 0.1 dB operating point.
- Half the tiles of every batch are hidden from the optimiser, and agreement is only ever reported on that half. In-sample agreement on 144 K parameters and a few thousand tiles will climb to anything at all.

F.2. What each input is worth

The head reads four per-tile signals and one per-frame signal, and this paper's own result is that a rule reading a single number beats it. That makes it fair to ask which of the five inputs carries anything. The ablation trains the same head once per input group, with every other group zeroed after its projection. Architecture, parameter count, optimiser, seed, image order and quality draws are identical across the six runs, so the only thing that differs is how much the head is allowed to know. Zeroing after the projection rather than deleting the projection is what makes that true.

The quality index q stays live in every single-group run. It is one number per frame, the same for every tile in that frame, so it can say which operating point the decoder is at and cannot, even in principle, tell two tiles of one frame apart. Holding it live keeps the runs comparable on per-tile information alone. The run with q on its own is then the floor that choice implies, which is the agreement reachable with no per-tile information whatever.

live inputs	agreement	± s.e.	last 250	over floor
stem,qp	0.7750	0.0074	0.7607	+0.2744
stem,latent,scales,bits,qp	0.7721	0.0075	0.7543	+0.2715
latent,qp	0.7448	0.0080	0.7433	+0.2442
scales,qp	0.7156	0.0083	0.7170	+0.2150
bits,qp	0.6202	0.0091	0.6137	+0.1196
qp	0.4923	0.0114	0.4900	-0.0083

Table 57. What each router input is worth. Agreement with the oracle's exit choice on 1,200 frames and 4,800 tiles the head never trained on, with one standard error taken across frames rather than across tiles, since tiles of one image are not independent draws. *last 250* is the running held-out agreement over the final 250 training steps, on 12 tiles a step, and is shown only to confirm that the end-of-run measurement is not a fluctuation. *over floor* is the margin over the best single constant exit, which scores 0.5006 on these tiles. The stem alone reaches the agreement of all five inputs together, and q alone does not clear the floor.

results/router_ablation.json, and the six per-variant training records results/router_ablation_stem.json and its five siblings. All on the pinned checkpoint, all at $\lambda = 1.3\times10^{-5}$, all 3,000 steps from seed 0, about an hour of one NVIDIA RTX A6000 each.

The stem alone is the whole head. Reading the stem and q gives 0.7750; reading the stem, the latent, the scales, the bit count and q gives 0.7721. The difference is 0.0029, against a standard error of 0.0074 on each measurement and 0.0105 on their difference, so it is under a third of one standard error and its sign is not determined. Four of the five inputs add nothing this measurement can resolve. That is a negative result about those four inputs, and it is worth stating what it does and does not say: it does not show that the latent, the scales or the bit count are uninformative about a tile, only that whatever they carry is already carried by the stem, which is computed downstream of all of them.

The single-group ordering is clear and the separations are larger than the noise. The stem at 0.7750 is 0.0302 above the latent, against a standard error of 0.0109 on the difference; the latent is 0.0292 above the scales; and the scales are 0.0954 above the bit count, which is more than seven standard errors. The bit count is the weakest per-tile input the head has. A.8 shows a rule that reads the bit count and nothing else beating the head at every rate, which is the sharpest form of the point that agreement

is the wrong objective: the input the head learns least from is the input that wins when it is used differently.

The quality index alone sits at the floor. It reaches 0.4923 ± 0.0114 where the best single constant exit scores 0.5006. A head with no per-tile information cannot beat the best constant in expectation, and this one does not. The floor is worth quoting beside every other row, because an agreement of 0.62 cannot be read at all until it is known that 0.50 is free.

exit map	exit 2	exit 3	exit 4	exit 5
oracle	19.6	16.2	14.1	50.1
stem,qp	19.8	14.2	12.9	53.1
stem,latent,scales,bits,qp	19.9	13.6	13.5	53.0
latent,qp	20.1	13.4	12.8	53.6
scales,qp	20.4	11.5	11.9	56.1
bits,qp	20.1	4.8	8.7	66.4
qp	10.5	0.0	0.0	89.5

Table 58. Where each variant sends its tiles, as a percentage of the 4,800 evaluation tiles. Exits 0 and 1 are below the split depth and cannot be chosen. Every variant reproduces the oracle's use of the shallowest exit to within a point, and what degrades as inputs are removed is the separation of the middle of the ladder: the bit count alone nearly empties exit 3 and piles the mass on the deepest exit, and q alone uses two of the four exits available to it.

results/router_ablation.json, fields pred_hist and oracle_hist.

F.3. What the head costs

q	decode (ms)	stem (ms)	router (ms)	share (%)
0	114.29	41.25	0.571	0.500
32	114.41	41.32	0.568	0.497
63	114.44	41.31	0.568	0.496

Table 59. The head against the decode it decides for. Three timings on one latent, interleaved so that a co-tenant's load drift lands on all of them equally. *decode* is the tiled decoder at the deepest exit, which is what the head's share is a share of; *stem* is the first j groups, which the head does not pay for because the decoder runs them anyway before the split. The head takes 0.50% of the decode where its operation count predicts 0.163%, a factor of 3.0.

results/router_latency.json: 1920×1080 padded to 2048×1280, 40 iterations after warm-up, one NVIDIA RTX A6000, the 144,024 parameter head runs/RECIPE512/routers2/v2_lam1.3e-5.pth. Measured on runs/RECIPE512/ckpt_eval.pth.tar rather than on the pinned checkpoint; the quantity is a ratio of two timings of the same weights, so it does not depend on which epoch they came from.

The extra 0.33 points are launch overhead. A head of 144,024 parameters evaluated on 40 vectors is small enough that its wall-clock is dominated by the cost of starting its kernels, and an operation count does not contain that. Every configuration-B saving in this work charges the head at its operation share, which is the smaller of the two numbers and therefore the more flattering; the honest reading is that the head costs about half a percent of the decode in time. It remains small against what it decides about, since one trunk block is 7.5% of the decode by the same cost vector.

F.4. A rule with no parameters

The entropy model produces one number per tile before the trunk runs, at no cost, and then discards it: how many bits that tile's latents took. Per-block bit allocation is a standard quantity in learned compression, where it is something to choose, and block-level rate control sets it so that complex regions get more bits [42]. Here it is read in the other direction, after the fact and at the decoder, as a statement about how hard the region was. What follows is the whole rule, and it can be implemented from this subsection alone.

Step 1, the per-tile statistic. Sum the entropy coder's estimated bits r_i over the latent positions i falling inside tile t, and divide by the mean over the N tiles of that frame. Normalising per frame rather than globally is deliberate. The absolute rate level is a property of the frame, and what decides a tile is how it compares with the rest of its own frame.

$$b(t) = N\frac{\sum_{i\in t} r_i}{\sum_i r_i} \tag{30}$$

Step 2, a rank-1 model of the distortion table. Assume every tile has the same shape of decay across the ladder, up to a scale that depends only on b(t).

$$\log D(t,k) \approx \alpha \log b(t) + c + \log \varphi_k \tag{31}$$

\varphi has one entry per reachable exit and says what that exit costs on an average tile; α is an exponent fitted rather than assumed. Fitting it is what lets the rule discover the direction of the relation instead of asserting one, and an earlier version of this surrogate that fixed the exponent at 1 was wrong about that direction.

Step 3, the fit. Take the mean of log D(t,k) over the reachable exits as the tile's difficulty, regress it on log b(t) by ordinary least squares to obtain α and c, and set log \varphi to the mean residual per exit. Do this leave-one-sequence-out, so no sequence contributes to the profile that routes it. The result is an offline calibration constant rather than a per-frame quantity: six numbers per rate, thirty for the five rates reported here.

Step 4, route. Run the oracle's own Lagrangian on the surrogate table in place of the true one.

$$\hat{k}(t) = \arg\min_{k \geq j} \left[e^{c}\, b(t)^{\alpha} \varphi_k + \lambda\, c_k \right] \tag{32}$$

Step 5, hit the budget. Bisect λ as configuration A does, in two levels: on the cheap per-tile table first, then correcting the target against a real decode of the resulting map, because the table and the decode differ slightly at the tile seams. Nothing is signalled, nothing is trained, and the arithmetic per tile is one power and K products. Sweeping λ traces the lower convex hull of the achievable set in the sense of [28], so the point returned is the best one on that hull at the budget it consumes.

The costs c_k on the pinned checkpoint are 0.6088, 0.7578, 0.8697, 1.0095 for exits 2 to 5, recovered from the frame-level saving of maps that send every tile to one fixed exit (results/static_RECIPE512_b01.json). The shallowest of them is what sets the architectural ceiling of 39.1% (results/saturation_RECIPE512_ctc53.json, c_j = 0.6088).

q	λ	α	φ_2	φ_3	φ_4	φ_5
0	3.14e-05	9.98	1.0242	1.0009	0.9912	0.9841
16	1.4e-05	4.58	1.0251	1.0021	0.9911	0.9822
32	7.25e-06	2.73	1.0297	1.0013	0.9893	0.9804
48	4.21e-06	1.94	1.0350	1.0004	0.9884	0.9771
63	2.49e-06	1.43	1.0494	0.9963	0.9829	0.9731

Table 60. The rule's entire state, at the 0.1 dB budget: the bisected price λ, the fitted exponent α, and the four-entry exit profile \varphi, per rate. α is positive everywhere, so a tile whose latents cost more bits is modelled as harder at every exit and is sent deeper, which is what the correlations below confirm. \varphi is nearly flat, spanning 4.1% at q0 and 7.8% at q63 between its shallowest and deepest entry, against costs that span 0.61 to 1.01, so what moves a tile along the ladder is its own b(t) rather than the shape of the profile.

results/raterank_RECIPE512_b01.json, pinned checkpoint, 53 CTC sequences at one frame each.

F.5. The rule against the trained head

q	0.1 rule	0.1 head	0.3 rule	0.3 head	0.5 rule	0.5 head
0	27.79	24.41	39.12	38.96	41.91	38.96
16	22.72	20.95	39.12	38.96	41.91	38.96
32	18.24	17.43	39.12	38.96	41.91	38.96
48	14.67	14.15	37.73	33.40	41.91	38.96
63	11.99	11.14	34.34	27.73	41.91	38.96

Table 61. The parameter-free rule against the 144 K head, at three budgets and five rates, as the percentage of the released decoder's operations saved. Both columns are charged for what they cost: the head pays its 0.163% compute share and the rule pays nothing. The rule is ahead in every cell, though the 0.5 dB pair is arithmetic on two different cost vectors rather than a comparison of allocations, for the reason given below.

Rule: results/raterank_RECIPE512_b01.json, b03 and b05. Head: results/router_RECIPE512_b01.json, b03 and b05. All on the pinned checkpoint over 53 sequences at one frame each, except results/raterank_RECIPE512_b05.json, which is on runs/RECIPE512/ckpt_eval.pth.tar.

At the working budget the rule wins at every rate. The margin is +3.38 points at q0 and +0.53 at its narrowest, which is q48, and it is positive at all 5 measured rates, the largest being 6.9 points. A head trained on this decoder against this oracle therefore returns nothing over a rule with no parameters, while carrying 0.163% of the decode that the rule does not.

At 0.3 dB the two saturate at the low rates and separate at the high ones. At q0, q16 and q32 both configurations send every tile to the shallowest exit and reach the ceiling, and the +0.16 point difference between them is exactly the head's own 0.163% compute share, which the rule does not pay. At q48 and q63 the budget still binds and the margins widen to +4.32 and +6.61 points, the latter being 0.7. The rule reaches 34.3% at q63 where the head reaches 27.7%. A looser budget gives the allocation more room to be wrong in as well as more room to be right.

The 0.5 dB pair measures a cost model rather than an allocation. At 0.5 dB every rate saturates for both configurations, so both send every tile to the shallowest exit and the two maps are identical. The reported margin of 2.95 points is arithmetic on two different cost vectors. The rule's file at that budget is on runs/RECIPE512/ckpt_eval.pth.tar, where the shallowest exit is priced at 0.5809 against the pinned 0.6088, a difference worth 2.79 points, and the remaining 0.163 is the head's compute share. We report the row rather than dropping it, and read nothing from it beyond the saturation.

q	rule agrees	ρ depth	ρ level	ρ spread	oracle
0	0.458	+0.54	+0.83	+0.61	29.90
16	0.319	+0.52	+0.85	+0.66	25.62
32	0.280	+0.53	+0.86	+0.71	20.93
48	0.350	+0.53	+0.84	+0.71	18.33
63	0.390	+0.38	+0.88	+0.68	15.64

Table 62. Why the rule works, and it is not by agreeing, at 0.1 dB. *rule agrees* is the fraction of tiles on which the rule picks the oracle's exit, far below the head's held-out 0.718 and still saving more at every rate. The three Spearman correlations are between a tile's bit count and, respectively, the depth the oracle assigns it, its mean distortion across the ladder, and the spread between its shallowest and its deepest exit. *oracle* is the oracle allocation measured inside the same run, which is the ceiling the rule is chasing.

results/raterank_RECIPE512_b01.json. The file stores spearman_bits_vs_exit against the negated exit index, as scripts/raterank_curve.py computes it, so the correlation with depth quoted here is its negative. The oracle column is measured at one frame per sequence and is not charged for the exit map; deployed configuration A, which carries the map, reads 29.9, 25.6, 20.9, 18.3, 15.6% over the same five rates at two frames per sequence (results/signalled_RECIPE512_ctc53.json).

The rule agrees with the oracle on 0.28 to 0.46 of tiles, well under the head's 0.718, and saves more at every rate. Agreement counts a disagreement between two exits that are within a hair of each other exactly as heavily as one that costs most of the frame's error, and most disagreements are of the first kind. What the rule gets right is the ordering. A tile's bit count correlates with the spread across the ladder, which is how much that tile stands to gain from depth, at 0.61 to 0.71 at every rate, and with the depth the oracle actually assigns at +0.38 to +0.54.

What the rule cannot do is see past that ordering. A rank-1 model gives every tile the same relative profile over exits, so b(t) decides where on the ladder a tile falls and never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head would have to earn its parameters on. Neither of our heads does.

q	oracle	head	rule	margin	rule agrees
0	34.02	31.56	30.92	-0.64	0.479
16	27.67	25.07	27.14	+2.07	0.370
32	22.29	16.47	22.39	+5.92	0.399
48	19.75	13.27	19.10	+5.83	0.431
63	17.19	8.99	16.16	+7.17	0.479

Table 63. The same comparison on a second training run. BEST is a separate recipe at a different epoch with its own head, trained the same way. The rule is ahead at 4 of 5 rates, by up to 7.2 points, and stays within 3.1 points of the oracle everywhere. The one rate it concedes is the only rate on either run where a trained head finishes ahead, and it does so by under a point.

results/raterank_BEST_compare.json: the BEST run, 53 sequences, 0.1 dB, not the pinned checkpoint. The margins there are wider than on the pinned run, so the pinned numbers are the conservative ones.

q	before	after	change	β before	β after
0	27.18	30.27	+3.10	136.6	19.0
16	23.27	25.82	+2.54	88.8	7.7
32	19.40	19.43	+0.03	9.1	-19.1
48	16.06	15.54	-0.51	-39.7	-35.9
63	12.94	9.58	-3.36	-50.3	-42.2

Table 64. Agreement rises and the saving does not follow. The head retrained after the exit mask was fixed, on the same recipe at the same λ. Held-out agreement goes from 0.718 to 0.808 while the deployed saving moves by +3.1 points at q0 and −3.4 at q63. β is the cost multiplier the bisection applies to move a head trained at one λ onto this operating point, and the retrained head is ahead where that multiplier is small and behind where it is large.

results/router_retrain_compare.json. The file records no checkpoint, and its configuration-B column reads 27.18% at q0 where the pinned measurement in results/router_RECIPE512_b01.json reads 24.41%, so the two are not on one basis. Only the comparison of the two heads within the file is read here.

F.6. What this section does not measure

- The ablation is one checkpoint, one λ, one seed and 3,000 steps per variant; results/router_ablation.json records the first of those as one_checkpoint. Nothing here says the ordering of the input groups would survive a second decoder or a second seed.
- The ablation reports agreement and nothing else. No variant was carried through a budget bisection to a deployed saving, so this section cannot say what a stem-only head would save, and the argument that agreement and saving come apart applies to the ablation as much as to the head.
- The head measured in configuration B was trained against runs/RECIPE512/ckpt_eval.pth.tar and then evaluated on the pinned checkpoint (results/router_RECIPE512_b01.json, field router2_meta), while the ablation heads were trained against the pinned checkpoint itself. Its 0.718 and the ablation's 0.7721 are therefore not two measurements of one thing.
- One head covers all five rates. A head per rate is the obvious remedy for the cost multiplier and is not measured here.
- The rule's 0.5 dB file is off the pinned checkpoint, which is why that pair of columns is read only for saturation.